Test-Time Scaling in the Wild: Why Exploitation, Not Exploration, Is the Bottleneck

Davide Romano[1,*], **Kanak Raj**[1,*], **Jerrod Parker**[1], **Daniele Giofrè**[1]

[1]Thomson Reuters
[*]Equal contribution

Correspondence: {davide.romano2,kanak.raj,jerrod.parker,daniele.giofre}@thomsonreuters.com

Abstract

Test-time scaling (TTS) improves language model outputs by spending additional inference compute — generating multiple candidates, searching over partial sequences, or iteratively refining drafts. These techniques yield large gains on mathematics and code, but have been developed and stress-tested almost exclusively on tasks where verification is straightforward. We conduct the first compute-normalised comparison of five TTS families across five open-ended generation benchmarks spanning medicine, law, finance, general chat, and creative writing — grounded in a unified framework that decomposes the effectiveness of each method's token budget into exploration and exploitation. The answer depends on which side of that decomposition you examine. Scaling exploration works: the best candidate in the pool improves steadily with compute across all settings. What breaks is exploitation — the step that converts a rich candidate pool into a final output. With state-of-the-art generators, reward models correlate at only $\hat{\rho}_v \approx 0.12$ with true quality, rendering selection near-random regardless of budget. Tree search amplifies this failure through diversity collapse. Refinement helps on one of five benchmarks; its apparent gains elsewhere are confounded. Only synthesis across candidates (Fusion) consistently improves over single-sample baselines, yet still recovers only ∼40% of available quality. The candidate pool is not the bottleneck — choosing from it is.

1 Introduction

Test-time scaling (TTS) improves language model performance without updating weights, by investing additional compute at inference time. Methods such as Best-of-N sampling with reward models [3, 25], tree search [20, 29], sequential refinement [16] and extended thinking [19] achieve large accuracy gains on mathematical reasoning, code generation, and multiple-choice tasks [14].

TTS methods are split by how they exploit additional compute: some select among candidates with an external verifier—typically a reward model or process supervisor—while others critique or synthesise without one. On the benchmarks where TTS was developed—competition mathematics, program synthesis, multiple-choice QA [11, 26, 34]—verifier-based methods are high-performing because the verifier itself works: outcomes are binary, labelled supervision is abundant, and reward models trained on it rank candidates reliably. Open-ended generation breaks this premise. Quality on medical consultations, legal analyses, and creative writing is graded against nuanced rubrics rather than a single correct answer [6, 23, 24], and supervision at comparable scale and reliability does not exist. The best off-the-shelf Reward Models (RMs) collapse to near random on open-ended tasks. Algorithms that depend on the verifier inherit this failure; methods that bypass it—synthesis across candidates or self-critique by the generator—fare unevenly across benchmarks and model families.

We provide the first systematic evaluation of TTS on open-ended generation, benchmarking five TTS families—Best-of-N, Beam Search, Particle Filtering, Refinement, and Fusion—across five open-ended generation benchmarks at matched compute.

The central finding is that **exploitation—not exploration—is the bottleneck**. Oracle quality—the true quality of the best candidate in the pool, corrected for judge-noise inflation—rises with compute, just as on deterministic tasks. This surface similarity conceals a critical difference: realised quality stagnates or

regresses for verifier-based methods, and even the strongest method Fusion captures only ∼40% of available headroom—the gap between a single-sample baseline and the oracle. None of the three scaling axes we test—compute, reward model size, or generator size—closes this gap. The candidate pool contains high-quality answers; converting them into a strong final output is the open problem.

Beyond this central finding, we contribute:

1. **Bias-corrected oracle estimator for TTS evaluation.** The naive oracle—the maximum judge score over a candidate pool—systematically overstates true pool quality, because taking the maximum of noisy scores selects partly on noise. This inflation grows with pool size and judge noise. We derive a closed-form per-entry bias-corrected estimator and estimate per-benchmark judge variance empirically from repeated scoring; the resulting gap between naive and corrected oracle is material on every benchmark.
2. **Verifier correlation predicts and explains selection failure.** We show analytically that for BoN, headroom capture approximately equals the verifier's correlation with ground truth (Eq. 2). Empirically, the Spearman correlation $\hat{\rho}_v$ between RM scores and true quality collapses to ∼0.12 on open-ended generation; both tested Outcome Reward Models fail similarly, indicating a structural rather than model-specific failure.
3. **Per-method diagnosis of exploitation failure across all TTS families.** We provide the first head-to-head comparison of generative exploitation strategies (Fusion, Sequential Refinement) against selection-based methods at matched compute on open-ended tasks. The comparison reveals that no exploitation strategy scales reliably: Beam Search and Particle Filtering perform strictly worse than parallel Best-of-N due to RM-guided diversity collapse (40–60% of independent-sampling diversity), and Sequential Refinement (SR) yields genuine improvement on only one of five benchmarks. Fusion is the sole method that improves over the single-sample baseline on every benchmark for Qwen3.5—yet yields mixed results on OLMo3, indicating that synthesis quality is a capability not all model families possess.

2 Related Work

Test-time scaling methods and verifiers. TTS methods fall approximately into five families: parallel, search, refinement based, fusion based, and extended thinking. *Parallel methods* generate N independent candidates and select via a verifier. *Search algorithms* structure exploration over partial sequences via a Process Reward Model (PRM), from deterministic beam search [3, 29] to stochastic variants [20]. *Sequential refinement* (or feedback) iteratively critiques and rewrites a single candidate [16, 17]. *Fusion* synthesises across multiple candidates [10]. Extended thinking allocates the full budget to a single reasoning trajectory [19]. Finally, many hybrid methods exist that combine strategies [4, 9, 22] or improve weaknesses of the basic methods [5, 15, 25, 32].

The success of the first two families depends on the external verifier. ORMs evaluate a (query, completion) pair with a single score, and PRMs [12] are trained to score intermediate Chain of Thought (CoT) reasoning steps. Existing ORMs and PRMs are trained predominantly on preference data or math-heavy datasets and one of the most common evaluation methods is Best-of-N sampling [14, 18]. Currently, there is no systematic comparison of these different families, and no framework to characterize how they trade off exploration (generating candidates) against exploitation (scoring, selecting, or synthesising a final output) under a shared token budget, or how that tradeoff degrades when verifier quality drops.

Evaluation beyond deterministic tasks. The evaluation of TTS methods is mostly performed on math-oriented tasks, while recently moving to multi-domain benchmarks, but still limited to deterministic and verifiable tasks [25, 29, 32]. Our work fills the remaining gap: the first systematic, compute-normalised comparison of the main TTS families on open-ended generation.

3 Theoretical Framework

We organise TTS methods along a single axis: how each method partitions a fixed token budget between *exploration* and *exploitation* (Figure 1). This decomposition yields two implications that structure the analysis

Table 1 Token cost accounting per TTS family under budget T. $\bar{\ell}$: mean generation length; $\bar{\ell}_c$: mean critique length; $\bar{\ell}_f$: mean fusion output length; K: refinement steps; N: candidates; B: beam width; W: branching factor; d: depth.

Method	T_x	T_e	N	Exploitation type
BoN + ORM	≈ 0	$\approx T$	$\lfloor T/\bar{\ell} \rfloor$	External scoring
Tree search	≈ 0	$\approx T$	$B \times W^d$ (collapses)	External scoring
SR	$K\bar{\ell}_c$	$T - K\bar{\ell}_c$	$K = T/(\bar{\ell} + \bar{\ell}_c)$	Generative (sequential)
Fusion	$\bar{\ell}_f$	$T - \bar{\ell}_f$	$N - 1$	Generative (synthesis)
Budget Forcing	0	T	1	None (single trajectory)

in Section 6.

3.1 Budget Decomposition and Quality Metrics

Let π_θ denote the generator, $\mathcal{V}$ a verifier assigning a scalar quality score to a candidate y given prompt x, and $\mathcal{E}$ the exploitation step mapping a candidate pool $\mathcal{P}$ to a final response $\hat{y} \sim \mathcal{E}(\mathcal{P})$. We distinguish three verifier types: the **ground-truth verifier** $\mathcal{V}^*$; **external verifiers** ($\mathcal{V}_{\neq\theta}$), i.e. ORMs or PRMs, with negligible cost relative to generation; and **self-verifiers** ($\mathcal{V}_{=\theta}$), which use π_θ to generate new tokens as part of exploitation (critiques in SR, synthesised responses in Fusion).

Every TTS method partitions a fixed budget T into exploration tokens T_e and exploitation tokens T_x ($T = T_e + T_x$); per-method accounting is in Table 1.

Quality metrics. Given x, let $\mu = \mathbb{E}_{y\sim\pi_\theta(\cdot|x)}[\mathcal{V}^*(y)]$ denote single-sample expected quality. The **oracle quality** $Q^*(T) = \mathbb{E}_{\mathcal{P}}[\max_{y\in\mathcal{P}} \mathcal{V}^*(y)]$ is the expected score of the best candidate in the pool. The **realised quality** $Q(T) = \mathbb{E}_{\mathcal{P}}[\mathcal{V}^*(\mathcal{E}(\mathcal{P}))]$ is the gold score of the algorithm's actual output. These decompose as:

$$Q(T) - \mu \;=\; \underbrace{(Q^*(T) - \mu)}_{\text{exploration headroom}} \;+\; \underbrace{(Q(T) - Q^*(T))}_{\text{net exploitation effect}}, \tag{1}$$

where the first term measures the quality available in the pool and the second measures how effectively the method converts that pool into a final output. $Q^*(T)$ is defined as the maximum over the *full* candidate pool (including synthesised or refined outputs), so the net exploitation effect is at most zero for every method by construction.

We operationalise exploitation quality through **headroom capture**:

$$h \;=\; \frac{Q(T) - \mu}{Q^*(T) - \mu}, \tag{2}$$

the fraction of exploration headroom that the method realises. $h = 1$ means oracle quality is returned; $h = 0$ means no improvement over a single sample; $h < 0$ means test-time compute hurts performance.

Oracle estimation. A noisy judge $J(y) = \mathcal{V}^*(y) + \epsilon_J$, $\epsilon_J \sim \mathcal{N}(0, \sigma_J^2)$ inflates $\max_i J(y_i)$ above $Q^*(T)$. Assuming an i.i.d. Gaussian pool, we derive (Appendix F) the entry-wise correction $\hat{O}^{\text{ideal}} = \max_i J(y_i) - a_N[\sqrt{\hat{\tau}^2 + \sigma_J^2} - \hat{\tau}]$, where $\hat{\tau}^2 = \max(0, s_X^2 - \sigma_J^2)$, s_X^2 is the empirical candidate variance, and $a_N = \mathbb{E}[\max_{i\le N} Z_i]$ for $Z_i \overset{\text{i.i.d}}{\sim} \mathcal{N}(0,1)$ — all oracle figures use $\hat{O}^{\text{ideal}}$.

3.2 Implication 1: The Verifier Bottleneck

For RM-based methods, headroom capture reduces to a single measurable quantity. Under i.i.d. sampling, oracle quality grows as:

$$Q^*(N) \;\approx\; \mu \;+\; \sigma\,\Phi^{-1}\!\left(\frac{N}{N+1}\right), \tag{3}$$

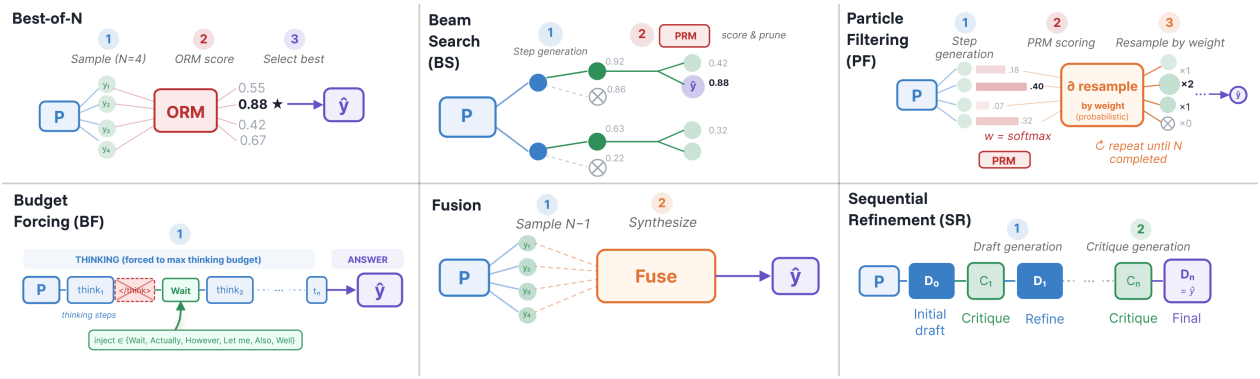

Figure 1 Schema for the six TTS methods in this paper. PF is the only method reproduced as it is, the other methods are variations from previous papers

at rate $O(\sigma\sqrt{\log N})$. When the external verifier correlates imperfectly with true quality at rate $\rho_v = \mathrm{Corr}[\mathcal{V}(y), \mathcal{V}^*(y)]$, dividing the expected BoN quality by oracle headroom yields (derivation in Appendix E):

$$h^{\mathrm{BoN}} \approx \rho_v. \tag{4}$$

Headroom capture for BoN *is* the verifier correlation. When $\rho_v = 0$, additional compute yields no benefit; when $\rho_v < 0$, more compute actively harms performance.

Robustness to judge noise. Equation 4 is robust to judge noise ϵ_J by construction: as proven in Appendix F.2, noise enters both $\hat{h}$ and $\hat{\rho}_v$ through the same attenuation factor, so the two quantities cancel and the identity $h \approx \rho_v$ holds at the level of raw measured scores without any correction. Practitioners can therefore use $\hat{\rho}_v$ as a direct predictor of correct headroom capture.

3.3 Implication 2: Oracle Ceilings Differ by Method

For BoN the candidate pool is N i.i.d. samples, so $Q^*_{\mathrm{BoN}}(T)$ is the standard order statistic over N draws (Eq. 3) — monotonic in T with diminishing returns. Fusion displaces one i.i.d. candidate with a synthesised output, giving $Q^*_{\mathrm{Fusion}}(T) \geq Q^*_{\mathrm{BoN}}(T)$ whenever the synthesis matches or exceeds the displaced candidate. SR's pool spans all intermediate and final drafts, so $Q^*_{\mathrm{SR}}(T)$ rises above the first-draft baseline only when conditioning on prior drafts genuinely improves quality. Per-method ceilings are studied empirically in Section 6.

4 Experimental Setup

Benchmarks. We evaluate on five open-ended generation benchmarks spanning medicine, law, finance, general chat, and creative writing (Table 2). None admits exact-match or binary verification. Full details in Appendix B.

Table 2 Benchmark characteristics.

Benchmark	Domain	n	Evaluation	Source	Scoring	Avg source items
LEXam [7]	Law	516	Reference-guided	Professor solutions	Holistic 0–1	1
HealthBench [2]	Medicine	5000	Rubric-only	Physician rubrics	Binary per criterion	11.4
PRBench [1]	Finance, Law	1650	Rubric-only	Expert rubrics	Binary per criterion	17.7
WildBench [13]	Chat/General	1024	Rubric-only	LLM-generated checklist	Holistic 1–10	∼11
WritingBench [30]	Writing	555	Rubric-only	LLM-generated rubric	1–10 per criterion	5

TTS Methods. We evaluate one representative per TTS family (Figure 1), with two tree search variants to disentangle deterministic pruning from stochastic diversity: **Best-of-*N*** (BoN), parallel sampling, external ORM; **Beam Search** (BS), deterministic tree search, PRM-guided; **Particle Filter** (PF) [20], stochastic

Table 3 Realised quality for **Qwen3.5-35B-A3B** across five benchmarks and four compute levels. Cell colour encodes delta from the single-sample baseline (BoN@1): green = improvement, red = regression. Bold = gain ≥ 0.03 over baseline.

	HealthBench					PRBench					LEXam				
	N=1	Low	Mid	High	XHigh	N=1	Low	Mid	High	XHigh	N=1	Low	Mid	High	XHigh
BoN+Skywork	**0.536**	0.538	0.539	0.539	0.539	**0.292**	0.293	0.294	0.293	0.292	**0.525**	0.527	0.529	0.531	0.530
BoN+Llama70B	**0.536**	0.539	0.541	0.541	0.541	**0.292**	0.293	0.295	0.296	0.298	**0.525**	0.528	0.532	0.533	0.534
Fusion	—	—	**0.574**	**0.587**	**0.588**	—	—	0.315	**0.324**	**0.331**	—	—	0.546	0.546	0.547
Beam Search	—	—	0.534	0.526	0.532	—	—	0.289	0.285	0.285	—	—	0.516	0.526	0.525
Particle Filter	—	—	0.532	0.532	0.530	—	—	0.286	0.282	0.282	—	—	0.531	0.527	0.526
SR	—	0.521	0.517	0.516	0.513	—	0.302	0.311	**0.327**	**0.342**	—	0.505	0.504	0.488	0.480
Budget Forcing	—	0.539	0.539	—	—	—	0.296	0.295	—	—	—	0.526	0.523	—	—

	WildBench					WritingBench					Overall				
	N=1	Low	Mid	High	XHigh	N=1	Low	Mid	High	XHigh	N=1	Low	Mid	High	XHigh
BoN+Skywork	**0.812**	0.825	0.834	0.839	**0.843**	**0.702**	0.708	0.712	0.715	0.718	**0.574**	0.578	0.581	0.584	0.584
BoN+Llama70B	**0.812**	0.820	0.825	0.828	0.830	**0.702**	0.707	0.710	0.714	0.717	**0.574**	0.578	0.580	0.582	0.584
Fusion	—	—	**0.844**	**0.857**	**0.864**	—	—	0.715	0.712	0.720	—	—	0.599	**0.606**	**0.610**
Beam Search	—	—	0.820	0.811	0.816	—	—	0.695	0.696	0.694	—	—	0.571	0.569	0.571
Particle Filter	—	—	0.802	0.801	0.797	—	—	0.694	0.694	0.689	—	—	0.569	0.567	0.565
SR	—	0.825	0.830	0.831	0.836	—	**0.738**	**0.750**	**0.770**	**0.775**	—	0.578	0.583	0.586	0.589
Budget Forcing	—	0.824	0.832	—	—	—	0.672	0.665	—	—	—	0.571	0.571	—	—

resampling, PRM-guided; **Sequential Refinement** (SR), sequential refinement, self-verifier; **Budget Forcing** (BF), extended thinking, single trajectory; and **Fusion**, generative synthesis over N candidates. More details on the algorithms, hyperparameters and prompt templates in Appendix C.

Models. Generators: **OLMo3-7B-Think** and **OLMo3.1-32B-Think** [28], and the **Qwen3.5** family (9B, 35B-A3B) [27]. ORMs for BoN: **Skywork-Reward-V2-Llama-3.1-8B** [14] and **Llama-3.1-70B-Instruct-RM-RB2** [18]. PRM for tree search: **VersaPRM-8B** [31]. ORMs were selected based on their performance on RewardBench 2 [18] while VersaPRM was selected for its unique multi-domain training.

Compute Normalisation. We normalise by *total generator output tokens* across all generative steps, excluding the cost of discriminative reward models whose cost is negligible compared to autoregressive decoding. We define four compute levels: **Low**, **Mid**, **High**, and **XHigh**, which match the average BoN output at $N \in \{2, 4, 8, 16\}$ within 25% tolerance (Table 5, Appendix A). Total compute across all experiments amounted to approximately 23,800 GPU-hours on B200 GPUs (Table 11, Appendix C).

Evaluation. Evaluating 27B generated tokens with each benchmark's native judge is cost-prohibitive; we instead use **Qwen3.5-397B-A17B** [27] as our judge across all benchmarks (Appendix C). We validated the agreement against each benchmark's native judge on stratified 5–15% samples (Appendix D). On HealthBench, our judge achieves Macro F1 of 0.679 with human annotations, above the mean inter-annotator agreement; on PRBench, criterion-level $\kappa = 0.679$, while WildBench and WritingBench show respectively QWK 0.564 and 0.408 per-pair agreement. The lower WritingBench agreement reflects the benchmark's known verbosity bias rather than judge miscalibration (Section 6.3): on identical BoN samples, our judge and the authors' official 7B Critic model [30] yield the same headroom capture (0.19 vs. 0.19, more in Appendix O).

5 Results

Table 3 reports compute-normalised results for Qwen3.5-35B-A3B across all methods and benchmarks. Results for Qwen3.5-9B and OLMo3 are in Appendices H and I.

The defining pattern is stagnation. BoN with either ORM yields negligible gains across an 8× budget increase. Tree search never improves over the single-sample baseline and actively degrades on several benchmarks. Budget Forcing is flat or regresses even at low compute — these reasoning models exhaust useful thinking within modest token budgets, and we do not scale it further. Yet oracle quality rises steadily with

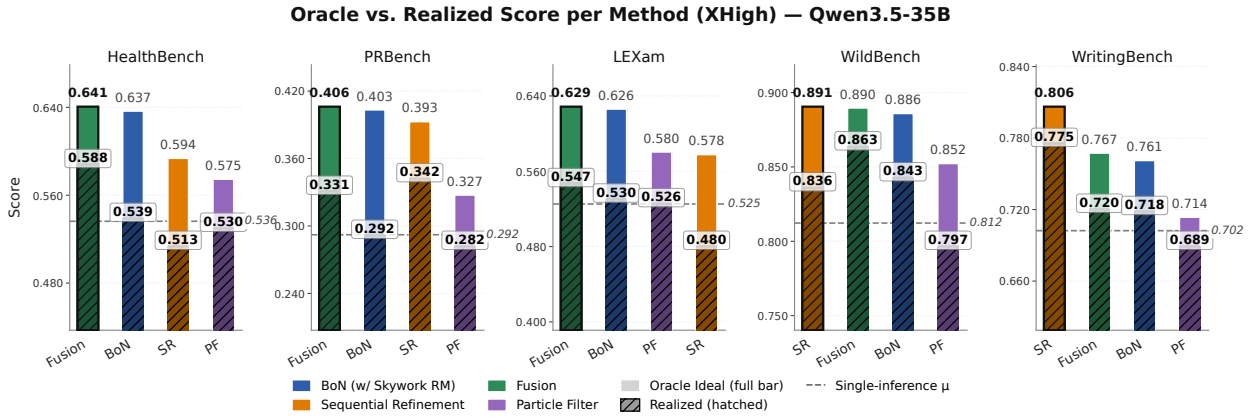

Figure 2 Oracle (full bar) vs. realised (hatched) quality per method at matched compute (**XHigh**), Qwen3.5-35B-A3B. All qualitative patterns above replicate under the naive oracle (Appendix G). In that appendix figure, the naive estimator systematically overestimates oracle quality, so the exploitation gaps shown there are conservative lower bounds on the true exploitation gap.

compute on every benchmark (Section 6): better candidates exist but no selection or search method identifies them.

Both ORMs fail almost identically. Skywork-Reward-V2 and Llama-3.1-70B-RM produce near-identical realised scores across all benchmarks (overall at XHigh: both 0.584), pointing to a structural rather than model-specific failure quantified in Section 6.1.

Fusion is the only method that consistently scales. It improves over the single-sample baseline on every benchmark, reaching 0.610 overall at XHigh compute (vs. 0.584 for the best RM-based method and 0.574 baseline), a pattern that holds across model scales (Section 6.4).

Sequential Refinement is unreliable. It achieves the largest gain of any method on WritingBench (+7.3pp) compared to single inference baseline, and outperforms Fusion on PRBench (0.342 vs. 0.331), but regresses on HealthBench (−2.3pp) and LEXam (−4.5pp). As Section 6.3 shows, the WritingBench gains are confounded by verbosity and the WildBench gains are driven almost entirely by a single subtask.

6 Analysis

The explanation reduces to one observation: **the candidate pool is not the problem—exploitation is**. Figure 2 makes this concrete. At the highest compute budget, all four methods produce candidate pools with high oracle quality across all five benchmarks. Fusion and BoN generate nearly identical oracle pools (typically within 0.01–0.04 oracle difference), yet their realised scores diverge consistently: averaged across benchmarks (Figure 3), Fusion captures ∼40% of available quality while BoN captures only ∼15%. Sequential Refinement is erratic, ranging from −0.86 to 0.70 across benchmarks, and Particle Filter averages around −40%, actively degrading quality. The bottleneck is not generating good candidates but selecting or synthesising the right answer from them. Table 4 summarises the failure mode for each method family.

6.1 Verifier Correlation Explains Selection Failure

Current reward models are insufficient to improve strong generators via BoN. The effectiveness of BoN depends critically on the generator-verifier gap. When the generator is weak relative to the RM, BoN yields clean positive scaling: our reproduction of the original Skywork-RM experiment, swapping in the much weaker generator from that paper, recovers strong TTS scaling (Appendix M). When the generator is strong, however, we find that current reward models struggle to discriminate reliably among candidate outputs.

Table 4 Diagnosis per method family. Oracle quality reflects exploration; headroom capture reflects exploitation.

Method	Oracle quality	Headroom capture	Failure mode
BoN + ORM	High	∼15% ($\hat{\rho}_v \approx 0.12$)	RM miscalibrated (§6.1)
Tree search	Collapsed (diversity loss)	Negative on most tasks	PRM destroys diversity (§6.2)
Fusion	≥ BoN	∼40% (best, still low)	Residual gap (§6.3)
SR	Lower than BoN on 3/5 benchmarks	Genuine gain on 1/5 (PRBench)	Refinement mostly hurts; where it helps, gains trace to subtask composition or verbosity (§6.3)

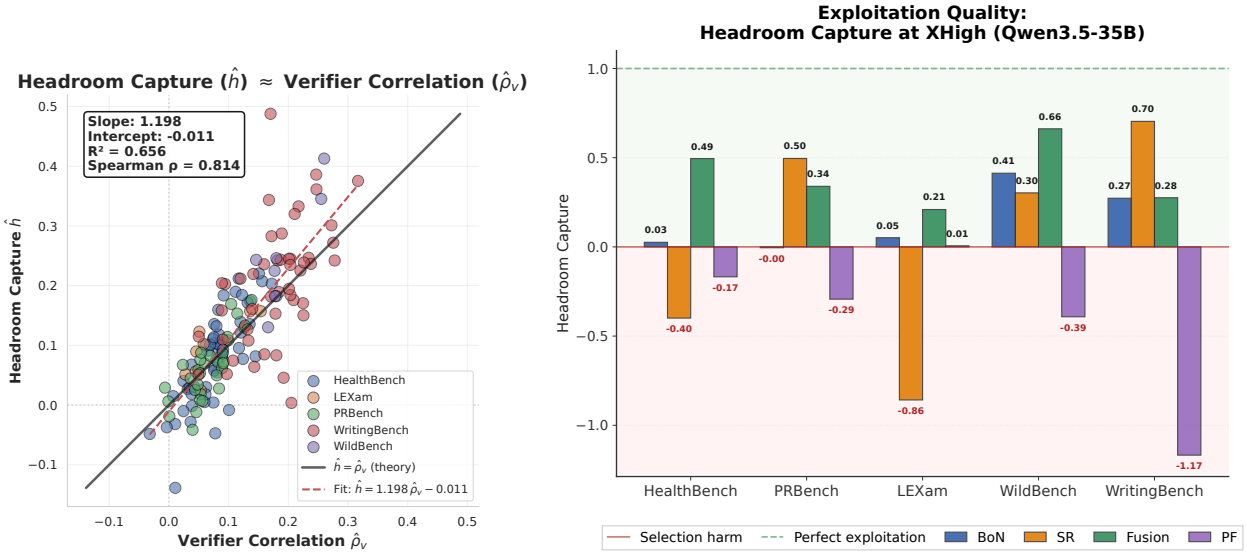

Figure 3 **Left:** $\hat{h}$ vs. $\hat{\rho}_v$ across $N = 152$ task-level points. Identity line $y = x$ is the theoretical prediction (Equation 4); empirical fit $y = 1.198\,\hat{\rho}_v - 0.011$ confirms it ($R^2 = 0.66$, $\rho = 0.81$, $p < 10^{-36}$). **Right:** Headroom capture per method at XHigh compute. The headrooms for all models are in Appendix J

On MATH-500 and GPQA Diamond, our generators gain only ∼2pp over single-sample (Appendix L); on open-ended generation, Spearman correlations between RM scores and judge scores average just 0.12 (Skywork) and 0.11 (Llama) across 152 (generator, RM, benchmark) combinations (Appendix K), reducing BoN to near-random selection.

$\hat{\rho}_v$ quantitatively predicts headroom capture. Theory predicts the identity $h^{\text{BoN}} \approx \rho_v$ (Equation 4). Regressing $\hat{h}$ on $\hat{\rho}_v$ across all $N = 152$ combinations yields slope 1.198 and intercept -0.011: slope near 1 and intercept near 0 directly validate the prediction ($R^2 = 0.66$, $\rho = 0.81$, $p < 10^{-36}$) Figure 3 (left)). Figure 3 (right) shows the concrete consequence: the same ∼15% vs. ∼40% headroom capture gap between RM-based BoN and Fusion, now explained by $\hat{\rho}_v \approx 0.12$.

6.2 Tree Search Failure: Diversity Collapse

While BoN fails at exploitation through poor selection, tree search fails at both exploitation *and* exploration: PRM-guided resampling collapses the candidate pool itself. Particle Filtering produces the least diverse final outputs across all benchmarks: for Qwen3.5-35B, PF's mean cosine distance ranges from 0.036 (WritingBench) to 0.069 (HealthBench), compared with 0.123–0.124 for BoN (Figure 4, left). Tracing pairwise similarity across 16 particles over generation depth, WritingBench particles remain near-identical throughout (similarity ≥ 0.997)—16 nominal particles are effectively a single trajectory (Figure 4, right). The mechanism is the exponential pruning sensitivity from Equation 8: a miscalibrated PRM prunes promising branches at every step, and the probability that the best candidate survives decays exponentially with depth. Naive parallel

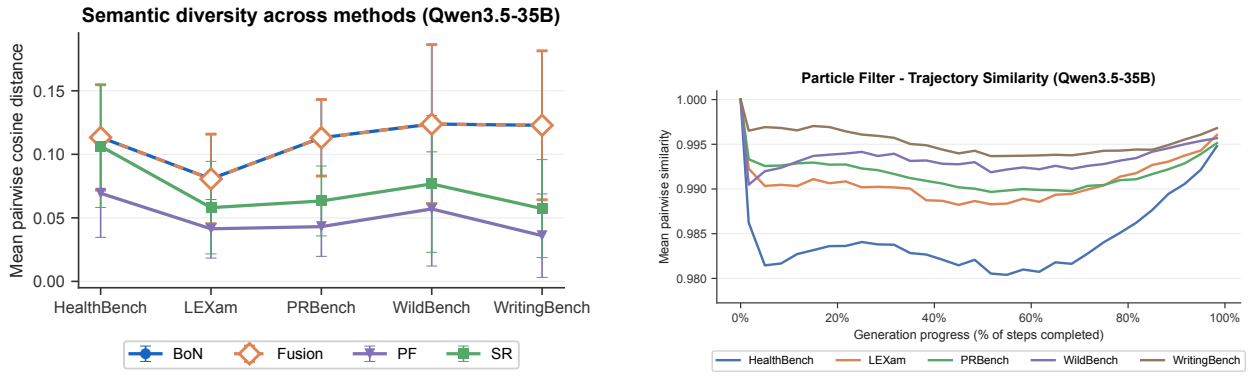

Figure 4 **Left:** Mean pairwise cosine distance across final outputs at High compute. **Right:** Particle trajectory similarity over normalised generation depth.

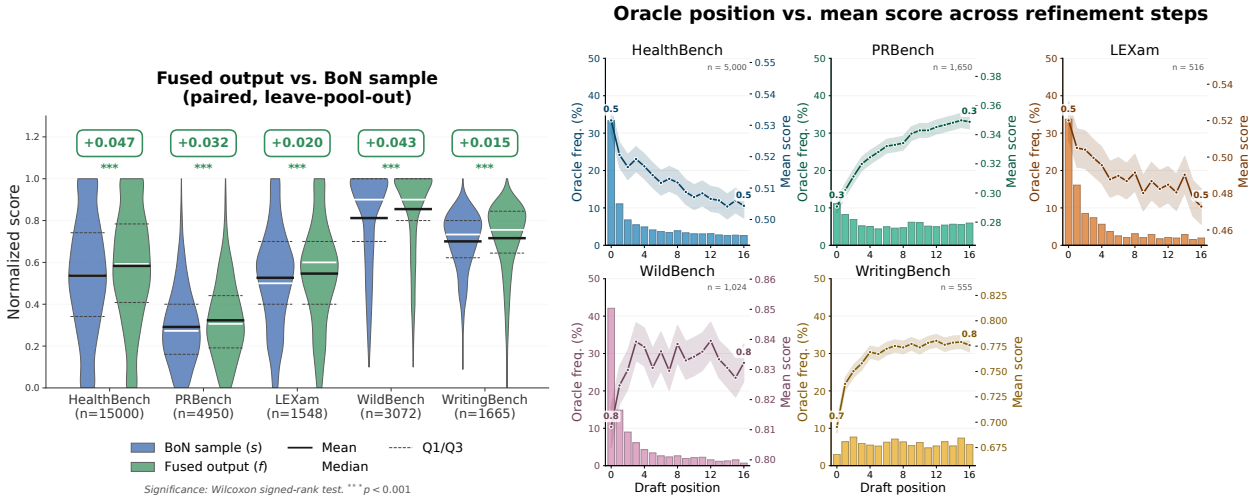

Figure 5 **Left:** Fusion output quality vs. a random candidate drawn from the candidates *not* included in the shared sub-pool common to both BoN and Fusion, per benchmark. **Right:** Oracle-position frequency (bars, left axis) and mean score trajectory (line, right axis) across SR iterations.

sampling preserves more diversity at equal cost.

6.3 Generative Methods: Fusion Synthesises, Sequential Refinement Regresses

Fusion: synthesis exceeds selection. Despite generating fewer independent candidates than BoN at equal compute, Fusion matches or exceeds BoN's oracle ceiling on all benchmarks (Figure 2), meaning the synthesised output frequently lifts the pool ceiling more than another i.i.d. sample. Figure 5 (left) shows that the fused output also consistently scores above a held-out i.i.d. candidate drawn from outside the shared sub-pool used by both BoN and Fusion—making this a fair comparison of synthesis quality against an independent sample of equivalent compute cost. Together, the two figures rule out a variance-only interpretation: synthesis raises mean candidate quality, not just the upper tail, so Fusion's ceiling gain reflects systematically better candidates rather than fatter-tailed sampling.

Sequential Refinement: refinement is benchmark-dependent. On the exploration side, SR's oracle ceiling falls below BoN's on three of five benchmarks (Figure 2), confirming that critique tokens displace more value than they add on most tasks. On the exploitation side, the per-iteration trajectory (Figure 5, right) reveals benchmark-dependent behaviour. On HealthBench and LEXam, refinement is counterproductive: the initial draft is most often the oracle and mean score declines with each iteration. On WildBench, mean score

rises despite the initial draft being the oracle 43% of the time; however, this gain traces to a single subtask (Coding & Debugging), which contributes more than the entire aggregate lift; excluding it produces a net regression (Appendix N). Only PRBench exhibits genuine improvement: oracle frequency is nearly uniform across iterations and mean score rises across all subtasks. WritingBench is discussed separately below.

The WritingBench exception is confounded by verbosity. WritingBench is the apparent exception: the oracle ceiling for SR exceeds BoN's and the best draft shifts to later iterations. However, the within-prompt length–score correlation is $\hat{\rho} = +0.33$ for SR (vs. $+0.20$ for BoN), and the judge discriminates more on length relative to substance than other benchmarks do (PRBench: 4.3× content-to-length discrimination; HealthBench: 2.5×; WritingBench: 0.7×). The fine-tuned judge released by the WritingBench authors shows an even stronger length–score correlation, confirming the verbosity bias is a property of the benchmark's evaluation design, not of any particular judge. Full analysis in Appendix O.

6.4 Generation and Exploitation Are Distinct Capabilities

Self-verifier quality is bounded by generator capability (Appendix E), but capability is not reducible to parameter count. To separate scale from family, we compare Fusion and SR within Qwen3.5 and across families (Qwen3.5 vs. OLMo3); full per-model tables are in Appendices I and H; full model-capacity decomposition in Appendix P.

Generation capability does not imply exploitation capability. The most revealing comparison is cross-family. Qwen3.5 and OLMo3 produce candidate pools with comparable headrooms on HealthBench and LEXam (Figure 18), and on WritingBench OLMo3 in fact shows *larger* headrooms. Yet headroom capture diverges sharply (Figure 19): at High compute, Qwen3.5 models (9B and 35B-A3B) close $\sim 36\%$ of headroom on average under Fusion, while OLMo3.1-32B-Think produces *negative* headroom on two of three benchmarks, meaning its synthesised output is worse than a random candidate. OLMo3 generates pools with as much (or more) oracle gap as Qwen3.5 but largely fails to convert that gap into a better answer.

Scaling generation capability within a family does not improve exploitation either. Within Qwen3.5, both Fusion and SR exhibit the same qualitative behaviour at every model size we tested. Fusion improves over the single-sample baseline on every benchmark and at every scale, but its advantage relative to a random candidate does not grow with model size — consistent with synthesis being a structural strategy (bypassing selection) rather than a capability-dependent one. SR's behaviour is benchmark-dependent — regression on HealthBench and LEXam, gains on WritingBench and PRBench — with comparable exploitation across model sizes (Figure 14).

7 Conclusion

We present the first systematic, compute-normalised comparison of five TTS families on open-ended generation, grounded in a unified exploration–exploitation framework, and find that exploitation — not exploration — is the bottleneck. Oracle quality rises steadily with compute, but no method converts more than a fraction into realised quality: RM-based selection fails structurally ($\hat{\rho}_v \approx 0.12$, both ORMs identical), tree search compounds this through diversity collapse, Fusion is the only consistently improving method yet captures only $\sim$40% of available headroom, and SR yields genuine gains on only one benchmark. These failures are not unique to open-ended generation — they arise whenever generators outpace RM training data and no scaling axis closes the gap.

Broader Impact and Limitations $\hat{\rho}_v$ provides a cheap diagnostic: practitioners can measure verifier-quality alignment on a small sample before committing inference budget. This matters most in high-stakes domains (medicine, law, finance), where RM-based TTS can actively degrade performance; our results reinforce that LLM-based evaluation is not a substitute for expert review, and deploying TTS in such settings without human oversight remains inadvisable.Our conclusions rest on two model families, and a single unified judge (Qwen3.5-397B-A17B), and do not cover all open-ended use cases. Our oracle estimator $\hat{O}^{\text{ideal}}$ assumes i.i.d. candidates—exact for BoN, but only approximate for Refinement, Fusion, and Particle Filtering; we expect it

to slightly underestimate the true oracle for the first two and slightly overestimate it for Particle Filtering (Appendix F).

Future work. Two open problems follow from the exploitation gap. First, training verifiers calibrated for open-ended evaluation: $\hat{\rho}_v$ provides a measurable target. Second, developing exploitation mechanisms beyond selection and synthesis: the high unrealised headroom represents a substantial opportunity.

Acknowledgements

We sincerely thank our colleague, Guglielmo Bonifazi, for his valuable input and insightful discussion points throughout the development of this work. We also express our gratitude to Shirsha Ray for her support in facilitating Kanak's collaboration with our team, which was instrumental in achieving these results.

References

[1] A. F. Akyürek, A. Gosai, C. B. C. Zhang, V. Gupta, J. Jeong, A. Gunjal, T. Rabbani, M. Mazzone, D. Randolph, M. M. Meymand, et al. PRBench: Large-scale expert rubrics for evaluating high-stakes professional reasoning. *arXiv preprint arXiv:2511.11562*, 2025.

[2] R. K. Arora, J. Wei, R. Soskin Hicks, P. Bowman, J. Quiñonero-Candela, F. Tsimpourlas, M. Sharman, M. Shah, A. Vallone, A. Beutel, J. Heidecke, and K. Singhal. HealthBench: Evaluating large language models towards improved human health. *arXiv preprint arXiv:2505.08775*, 2025.

[3] E. Beeching, L. Tunstall, and S. Rush. Scaling test-time compute with open models, 2024. URL https://huggingface.co/spaces/HuggingFaceH4/blogpost-scaling-test-time-compute.

[4] K. Chang, Y. Shi, C. Wang, H. Zhou, C. Hu, X. Liu, Y. Luo, Y. Ge, T. Xiao, and J. Zhu. Step-level verifier-guided hybrid test-time scaling for large language models, 2025. URL https://arxiv.org/abs/2507.15512.

[5] G. Dalal, A. Hallak, G. Chechik, and Y. Ziser. More test-time compute can hurt: Overestimation bias in llm beam search, 2026. URL https://arxiv.org/abs/2603.15377.

[6] K. Elangovan, J. C. L. Ong, L. Jin, B. J. J. Seng, Y. H. Kwan, L. S. Ng, R. J. Zhong, J. K. L. Ma, Y. H. Ke, N. Liu, K. M. Giacomini, and D. S. W. Ting. Development and evaluation of a lightweight large language model chatbot for medication enquiry. *PLOS Digital Health*, 4(9):e0000961, 2025. doi: 10.1371/journal.pdig.0000961. URL https://doi.org/10.1371/journal.pdig.0000961.

[7] Y. Fan, J. Ni, J. Merane, Y. Tian, Y. Hermstrüwer, Y. Huang, M. Akhtar, E. Salimbeni, F. Geering, O. Dreyer, D. Brunner, M. Leippold, M. Sachan, A. Stremitzer, C. Engel, E. Ash, and J. Niklaus. LEXam: Benchmarking legal reasoning on 340 law exams. *arXiv preprint arXiv:2505.12864*, 2025.

[8] D. Hendrycks, C. Burns, S. Kadavath, A. Arora, S. Basart, E. Tang, D. Song, and J. Steinhardt. Measuring mathematical problem solving with the math dataset, 2021. URL https://arxiv.org/abs/2103.03874.

[9] Y. Inoue, K. Misaki, Y. Imajuku, S. Kuroki, T. Nakamura, and T. Akiba. Wider or deeper? scaling llm inference-time compute with adaptive branching tree search, 2025. URL https://arxiv.org/abs/2503.04412.

[10] A. Khairi, D. D'souza, M. Fadaee, and J. Kreutzer. Making, not taking, the best of n. *arXiv preprint arXiv:2510.00931*, 2025.

[11] D. Li, S. Cao, C. Cao, X. Li, S. Tan, K. Keutzer, J. Xing, J. E. Gonzalez, and I. Stoica. S*: Test time scaling for code generation. *ArXiv*, abs/2502.14382, 2025. URL https://api.semanticscholar.org/CorpusID:276482470.

[12] H. Lightman, V. Kosaraju, Y. Burda, H. Edwards, B. Baker, T. Lee, J. Leike, J. Schulman, I. Sutskever, and K. Cobbe. Let's verify step by step, 2023. URL https://arxiv.org/abs/2305.20050.

[13] B. Y. Lin, Y. Deng, K. Chandu, F. Brahman, A. Ravichander, V. Pyatkin, N. Dziri, R. Le Bras, and Y. Choi. WildBench: Benchmarking LLMs with challenging tasks from real users in the wild. *arXiv preprint arXiv:2406.04770*, 2024.

[14] C. Y. Liu, L. Zeng, Y. Xiao, J. He, J. Liu, C. Wang, R. Yan, W. Shen, F. Zhang, J. Xu, Y. Liu, and Y. Zhou. Skywork-Reward-V2: Scaling preference data curation via human-ai synergy. *arXiv preprint arXiv:2507.01352*, 2025.

[15] R. Liu, J. Gao, J. Zhao, K. Zhang, X. Li, B. Qi, W. Ouyang, and B. Zhou. Can 1b llm surpass 405b llm? rethinking compute-optimal test-time scaling, 2025. URL https://arxiv.org/abs/2502.06703.

[16] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegreffe, U. Alon, N. Dziri, S. Prabhumoye, Y. Yang, et al. Self-refine: Iterative refinement with self-feedback. *Advances in neural information processing systems*, 36:46534–46594, 2023.

[17] L. Madaan, A. Didolkar, S. Gururangan, J. Quan, R. Silva, R. Salakhutdinov, M. Zaheer, S. Arora, and A. Goyal. Rethinking thinking tokens: Llms as improvement operators. *arXiv preprint arXiv:2510.01123*, 2025.

[18] S. Malik, V. Pyatkin, S. Land, J. Morrison, N. A. Smith, H. Hajishirzi, and N. Lambert. RewardBench 2: Advancing reward model evaluation. *arXiv preprint arXiv:2506.01937*, 2025.

[19] N. Muennighoff, Z. Yang, W. Shi, X. L. Li, L. Fei-Fei, H. Hajishirzi, L. Zettlemoyer, P. Liang, E. Candès, and T. B. Hashimoto. s1: Simple test-time scaling. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 20275–20321, 2025.

[20] I. Puri, S. Sudalairaj, G. Xu, K. Xu, and A. Srivastava. Rollout roulette: A probabilistic inference approach to inference-time scaling of llms using particle-based monte carlo methods. *arXiv preprint arXiv:2502.01618*, 2025.

[21] D. Rein, B. L. Hou, A. C. Stickland, J. Petty, R. Y. Pang, J. Dirani, J. Michael, and S. R. Bowman. Gpqa: A graduate-level google-proof q&a benchmark, 2023. URL https://arxiv.org/abs/2311.12022.

[22] J. Saad-Falcon, A. G. Lafuente, S. Natarajan, N. Maru, H. Todorov, E. Guha, E. K. Buchanan, M. Chen, N. Guha, C. Ré, and A. Mirhoseini. Archon: An architecture search framework for inference-time techniques, 2025. URL https://arxiv.org/abs/2409.15254.

[23] M. Sharma, C. B. C. Zhang, C. Bandi, C. Wang, A. Aich, H. Nghiem, T. Rabbani, Y. Htet, B. Jang, S. Basu, A. Balwani, D. Peskoff, M. Ayestaran, S. M. Hendryx, B. Kenstler, and B. Liu. Researchrubrics: A benchmark of prompts and rubrics for evaluating deep research agents, 2025. URL https://arxiv.org/abs/2511.07685.

[24] Y. Shi, H. Liu, Y. Hu, G. Song, X. Xu, Y. Ma, T. Tang, L. Zhang, Q. Chen, D. Feng, W. Lv, W. Wu, K. Yang, S. Yang, W. Wang, R. Shi, Y. Qiu, Y. Qi, J. Zhang, X. Sui, Y. Chen, Y. Zhang, A. Yang, B. Yu, D. Liu, J. Lin, W. Shen, B. Zhao, C. L. A. Clarke, and H. Wei. Plawbench: A rubric-based benchmark for evaluating llms in real-world legal practice, 2026. URL https://arxiv.org/abs/2601.16669.

[25] C. Snell, J. Lee, K. Xu, and A. Kumar. Scaling llm test-time compute optimally can be more effective than scaling model parameters. *arXiv preprint arXiv:2408.03314*, 2024.

[26] Z. Song, S. Tang, P. Ye, J. Fan, L. Bai, T. Chen, and W. Ouyang. Ctts: Collective test-time scaling, 2025. URL https://arxiv.org/abs/2508.03333.

[27] Q. Team. Qwen3.5: Accelerating productivity with native multimodal agents, February 2026. URL https://qwen.ai/blog?id=qwen3.5.

[28] Team OLMo, A. Ettinger, A. Bertsch, B. Kuehl, D. Graham, D. Heineman, D. Groeneveld, F. Brahman, F. Timbers, H. Ivison, et al. OLMo 3. *arXiv preprint arXiv:2512.13961*, 2025.

[29] Y. Wu, Z. Sun, S. Li, S. Welleck, and Y. Yang. Inference scaling laws: An empirical analysis of compute-optimal inference for problem-solving with language models. *arXiv preprint arXiv:2408.00724*, 2024.

[30] Y. Wu, J. Mei, M. Yan, C. Li, S. Lai, Y. Ren, Z. Wang, J. Zhang, M. Wu, Q. Jin, and F. Huang. WritingBench: A comprehensive benchmark for generative writing. *arXiv preprint arXiv:2503.05244*, 2025.

[31] T. Zeng et al. VersaPRM: Multi-domain process reward model via synthetic reasoning data. *arXiv preprint arXiv:2502.06737*, 2025.

[32] Q. Zhang, F. Lyu, Z. Sun, L. Wang, W. Zhang, W. Hua, H. Wu, Z. Guo, Y. Wang, N. Muennighoff, I. King, X. Liu, and C. Ma. A survey on test-time scaling in large language models: What, how, where, and how well?, 2025. URL https://arxiv.org/abs/2503.24235.

[33] L. Zhou, L. Pacchiardi, F. Martínez-Plumed, K. M. Collins, Y. Moros-Daval, S. Zhang, Q. Zhao, Y. Huang, L. Sun, J. E. Prunty, Z. Li, P. Sánchez-García, K. Jiang-Chen, P. A. M. Casares, J. Zu, J. Burden, B. Mehrbakhsh, D. Stillwell, M. Cebrian, J. Wang, P. Henderson, S. T. Wu, P. C. Kyllonen, L. Cheke, X. Xie, and J. Hernández-Orallo. General scales unlock AI evaluation with explanatory and predictive power. *Nature*, 652:58–67, 2026. doi: 10.1038/s41586-026-10303-2. URL https://doi.org/10.1038/s41586-026-10303-2.

[34] X. Zhu, X. Zhou, B. Zhu, H. Hu, M. Du, H. Zhang, H. Wang, and Z. Guo. Codescaler: Scaling code llm training and test-time inference via execution-free reward models, 2026. URL https://arxiv.org/abs/2602.17684.

A Compute Band Configurations

Table 5 Compute band configurations. Fusion High uses subset = 7 for Qwen3.5 and subset = 6 for OLMo3. Entries marked — were not run. $\bar{\ell}_t$: mean thinking tokens per BoN sample; $\bar{\ell}$: mean answer tokens per BoN sample. †Two tasks ran at $W = 3, k = 4$ instead of $W = 4, k = 4$ (see Table 48).

Band	BoN	Beam Search	PF	Sequential Refinement	Budget Forcing	Fusion
Low	$n = 2$	—	—	iter= 3	$2\bar{\ell}_t + \bar{\ell}$	—
Mid	$n = 4$	$W = 2, k = 2$	$N = 4$	iter= 6	$4\bar{\ell}_t + \bar{\ell}$	subset= 3
High	$n = 8$	$W = 2, k = 4$	$N = 8$	iter= 12	$8\bar{\ell}_t + \bar{\ell}$	subset= 6/7
XHigh	$n = 16$	$W = 4, k = 4^{\dagger}$	$N = 16$	iter= 16	$16\bar{\ell}_t + \bar{\ell}$	subset= 15

B Benchmark Details

LEXam. We use the `open_question` configuration of `LEXam-Benchmark/LEXam` (HuggingFace), drawn from law-school exams administered at a Swiss university. The original release contains 2,841 items across four legal areas (Private, Public, Interdisciplinary, Criminal) in English and German. We filter to `language == "en"`, retaining 516 items and dropping 2,325 German items. The English subset is dominated by international and comparative law (jurisdiction: International 429, Generic 73, Swiss 14), reflecting the language of instruction at the source institution. The pool is heavily skewed toward Private and Public law (85% of items combined), with Interdisciplinary (48) and Criminal (31) forming the long tail. Each item carries a single professor reference solution.

HealthBench. We use `openai/HealthBench` (5,000 items). Every example carries exactly one `theme` tag; the tags are non-overlapping and cover the full corpus, so we partition the dataset into seven sub-files used as separate evaluation tasks: `global_health` (1,097), `hedging` (1,071), `communication` (919), `context_seeking` (594), `emergency_referrals` (482), `health_data_tasks` (477), `complex_responses` (360). Each example carries on average 11.4 physician-authored rubric criteria, each tagged with a signed integer point value: positive points reward desired behaviors, negative points penalize undesired ones (e.g. recommending an unsafe action). The judge marks each criterion *met* or *not met*. An example's score is the sum of points across met criteria, divided by the sum of all *positive* points in the rubric (the maximum positive total). The dataset-level metric is the mean across examples, clipped to $[0, 1]$.

PRBench. We use the four pre-split configurations of `ScaleAI/PRBench`: `prbench_finance` (600), `prbench_finance_hard` (300), `prbench_legal` (500), `prbench_legal_hard` (250). Each item carries between 10 and 30 weighted rubric criteria (29,252 criteria across 1,650 items, average ~ 17 per item). Each criterion is tagged with one of six `weight_class` levels, but the per-criterion integer weight is not a fixed mapping from class to value: the dataset stores a specific weight per criterion drawn from the ranges in Table 6, so a single example's rubric can contain criteria with weights spanning $[-10, +10]$. We report the clipped score: per-example $s_i = \text{raw}_i / w^{+}_{\max,i}$ where $w^{+}_{\max,i}$ is the maximum positive weight present in example i's rubric, and the dataset-level score is $\max\big(0,\ \overline{s_i}\big)$.

Table 6 PRBench weight class to integer point ranges.

Weight class	Integer weight
critically important	$\{8, 9, 10\}$
important	$\{4, 5, 6, 7\}$
slightly important	$\{1, 2, 3\}$
slightly detrimental	$\{-3, -2, -1\}$
detrimental	$\{-7, -6, -5, -4\}$
critically detrimental	$\{-10, -9, -8\}$

WildBench. We use the `v2` configuration of `allenai/WildBench` (1,024 items). Items are tagged with one of 11 `primary_tag` categories, with a heavy long tail: Information seeking (182), Coding & Debugging (170), Creative Writing (146), Reasoning (133), Planning (116), Math (87), Editing (82), Data Analysis (33), Role playing (30), Brainstorming (24), Advice seeking (21). Each example carries a task-specific checklist of 6–33 items (avg ~ 11). *Deviation from upstream that affects absolute scores:* the WildBench paper macro-averages WB-Score across five task groups (equal weight per group regardless of group size); we micro-average across all 1,024 items, so larger categories dominate. This shifts our absolute WB-Scores relative to the official leaderboard but preserves relative ordering between TTS methods evaluated on the same item set, which is the comparison we make.

WritingBench. We use `X-PLUG/WritingBench` (`benchmark_all.jsonl`). The original release contains 1,000 items in Chinese and English; we filter to `lang == "en"`, retaining 555 items and dropping 445 Chinese items. The remaining items split fairly evenly across six domains: `finance_business` (115), `academic_engineering` (107), `politics_law` (99), `literature_arts` (96), `advertising_marketing` (74), `education` (64). Unlike the other rubric-only benchmarks, WritingBench rubrics are *dynamic per query*: every item ships with exactly five criteria, each consisting of a name, a free-text `criteria_description`, and five score-band descriptions (`1–2`, `3–4`, `5–6`, `7–8`, `9–10`). Each criterion is scored on 1–10; the example score is the mean across the five criteria.

Multi-turn handling. Three of the five benchmarks contain multi-turn items: HealthBench (41.7%, max 19 messages), PRBench (42.2%, max 19 messages), and WildBench (27.0%, max 9 messages); LEXam and WritingBench are single-turn only. Multi-turn conversations are passed to both the generator and the judge as ordered `[{role, content}]` message lists, following the standard OpenAI / vLLM chat-completion convention.

Native judges. Each benchmark ships with a recommended evaluation judge whose model and protocol we adopt verbatim from the upstream codebase (Table 7); these natives serve as the ground-truth comparator in our judge-agreement analysis (Sec D).

Table 7 Native judges adopted verbatim from each benchmark's upstream release. Calls/example is the average number of judge invocations per candidate completion under the native protocol; per-criterion designs (HealthBench, PRBench, WritingBench) fan out into many calls per example.

Benchmark	Native judge	Protocol	Calls/ex
LEXam	GPT-4o + Qwen3-32B + DeepSeek-V3	holistic, min-agg ensemble	3
HealthBench	GPT-4.1	per-criterion binary	~ 11
PRBench	o4-mini	per-criterion binary	~ 17
WildBench	GPT-4.1	holistic + checklist	1
WritingBench	Claude 3.7 Sonnet	per-criterion 1–10	5

C Hyperparameters and Prompt Templates

This appendix documents (i) the sampling and serving configuration used across all generation calls, (ii) method-specific algorithmic settings beyond the compute-band axis already reported in Appendix A, (iii) reward and process reward model configuration, and (iv) the prompt templates used by Sequential Refinement and Fusion. Methods not listed under prompt templates use only the dataset's prompt together with the system prompt below.

Sampling. All generation uses the sampling profile per generator family in Table 8, with seed 42 throughout. The system prompt is identical across all benchmarks and methods: `"Please complete the following user request."` Table 8 reports the canonical profile per generator. Fusion synthesis and Budget Forcing reduce `max_tokens` from 16,384 to 8,192. The serving context length depends on generator and method: OLMo3 runs at 65,536 throughout (the model's maximum supported context length); Qwen3.5 runs at 65,536 for BoN, Beam Search, Particle Filter, and Budget Forcing, raised to 131,072 for Sequential Refinement to accommodate accumulating revision history and for Fusion at subset sizes 3 and 7, and to 262,144 for Fusion at subset size 15 to fit the larger candidate pool. Sequential Refinement and Fusion strip everything up to and including the `</think>` tag from each draft before passing it to the next step.

Table 8 Canonical sampling profile per generator family.

Generator	Temperature	Top-p	Top-k	Presence penalty	Thinking mode
Qwen3.5 (9B, 35B-A3B)	1.0	0.95	20	1.5	enabled
OLMo3 (7B, 32B)	0.8	0.9	-1	0	enabled

Method configurations. Compute-band axes (n for BoN, W,k for Beam Search, N for Particle Filter, iteration count for Sequential Refinement, subset size for Fusion) are listed in Table 5. Table 9 records the remaining fixed hyperparameters per method.

Table 9 Method-specific algorithmic settings beyond the compute-band axis.

Method	Settings
BoN	XHigh pool (N=16) generated once; lower bands subsampled without replacement; selection = argmax ORM score.
Beam Search	W beams, k candidates/step (Table 5); delimiter `\n\n`; scored by min per-step PRM on full path; top-W kept globally; EOS = beam complete; PRM: VersaPRM-8B; depth 300 (500 for deterministic).
Particle Filter	N particles (Table 5); delimiter `\n\n`; loop order: score → resample → generate; multinomial resampling with replacement, weights = softmax($\mathbf{w}$, T=1.0); PRM: VersaPRM-8B; max iterations 300.
Sequential Refinement	Forced iterations (stop-check ignored); history window K=3 (draft, feedback) pairs; feedback & stop-check: thinking disabled, stop-check at T=0; `</think>` stripped from drafts before each prompt call.
Budget Forcing	2-phase decode: Phase 1 suppresses end-of-thinking tag and injects a wait-string to extend thinking; Phase 2 generates answer freely from accumulated context.
Fusion	Random subset without replacement from precomputed BoN-16 pool; candidates shuffled and `</think>`-stripped before synthesis; single-shot direct synthesis with thinking enabled.

BoN. Generate N independent completions, score each with an ORM, return the argmax. For compute bands Low through XHigh, a pool of N=16 completions is generated once per prompt (the XHigh run). Lower-band results are obtained by bootstrap subsampling: n candidates are drawn without replacement from the XHigh pool, scored by the ORM, and the argmax is selected. All four bands therefore share the same underlying generations; only the pool size visible to the reward model changes.

Beam Search. Maintain W candidate paths; at each step expand each beam by k continuations, score all $W \times k$ by PRM, keep the top-W. At each step every beam independently generates k candidate continuations, stopping at the step delimiter `\n\n` or EOS. All $W \times k$ candidates are scored by the PRM on the full accumulated path, using the minimum per-step positive probability as the sequence-level score. The top-W remaining candidates are retained as the next generation of beams. A beam is marked complete when generation stops at EOS rather than the delimiter; completed beams are frozen with their current score. The final output is the completed beam with the highest PRM score, or the highest-scoring active beam if no beam reached completion.

Particle Filter. Maintain N particles; at each step score all by PRM, stochastically resample with replacement, then generate one continuation each. The loop order follows SCORE → RESAMPLE → GENERATE. All N particles are scored by the PRM on their full accumulated paths; N replacement indices are then drawn from softmax($\mathbf{w}/T$) with T=1.0 via multinomial sampling with replacement. Finished particles (EOS) are frozen: they participate in resampling with their terminal score and, if re-selected, are copied as-is. PF terminates when all N particles reach EOS or after all iterations is exhausted. The final output is the highest-scoring completed (frozen) particle.

Sequential Refinement. Iteratively improve a single draft via two sequential calls per iteration: feedback, and refinement — all using the same generator. We define a window of maximum previous iterations present in the context at J=3, which deviates from the full-history formulation of Madaan et al. [16] to keep the prompt within context. The thinking blocks are stripped from drafts before each call. The feedback generation is run with thinking mode off with Qwen3.5.

Budget Forcing. Force the model to think longer by suppressing its end-of-thinking token and injecting a wait-string, then generate a final answer from the extended context. Phase 1 generates within a shared token budget; each time the End of Thinking token (e.g. `</think>`) is emitted, it is stripped and an injection token from {*"Wait", "Actually", "However", "Let me", "Also", "Well"*} is appended until the budget is exhausted, the suppression limit is reached, or the model halts naturally. The token is selected randomly.

Phase 2 generates the answer freely from the accumulated context. Evaluated at Low and Mid only; higher budgets caused degeneration into repetitive self-termination.

Fusion. Generate a pool of candidates independently, then synthesise a single fused output in one call rather than selecting among them. The candidate pool is shared with BoN; a random subset of size $s \in \{3, 7, 15\}$ is drawn without replacement, shuffled, and stripped of the thinking part before the synthesis call.

Reward models. Both ORMs (Skywork-Reward-V2-Llama-3.1-8B and Llama-3.1-70B-Instruct-RM-RB2) score each (prompt, completion) pair in a single classification forward pass. Each call returns one scalar score per candidate.

Process reward model. VersaPRM-8B returns per-step scores in a single forward pass with STEP pooling. The input format follows VersaPRM's training specification: the prompt is concatenated to the candidate's steps, separated by a space followed by four newlines, with steps obtained client-side by splitting on the configured delimiter `\n\n`. At each step boundary (tokenizer id 23535), the model emits a [neg, pos] logit pair over the $+/-$ discriminator tokens (ids $\{12, 10\}$) it was trained on; the positive probability is taken as the per-step score, and the sequence-level score reported to the search algorithm is the minimum across per-step positives, following the aggregation strategy reported as best-performing by Zeng et al. [31].

Sequential Refinement prompts. The initial draft (y_0) is generated with no custom template, using only the dataset prompt and the system prompt above. Each subsequent iteration consists of three generator calls in fixed order: (i) a *feedback* call that critiques the current draft; (ii) a *stop-check* call returning STOP or CONTINUE (computed every iteration but ignored under our forced-iteration setup); and (iii) a *refinement* call that produces the next draft from the task input and the recent (draft, feedback) history. The three templates are reproduced below verbatim, with placeholders: `{x}` is the original task input, `{y_t}` the current draft, `{fb_t}` the feedback on that draft, `{few_shot_examples}` an optional few-shot prefix (left empty in all our runs), and `{history}` the rendered sequence of prior drafts and feedbacks in the format: `## Draft 0\n{y_0}\n\n## Feedback 0\n{fb_0}\n\n## Draft 1\n`... (capped at the most recent K=3 pairs).

Feedback prompt (step i)

```
{few_shot_examples}### TASK INPUT
{x}

### CURRENT DRAFT
{y_t}

### FEEDBACK
Identify specific issues in the draft and propose concrete improvements
as bullet points:
```

Stop-check prompt (step ii)

```
{few_shot_examples}### TASK INPUT
{x}

### CURRENT DRAFT
{y_t}

### FEEDBACK ON THIS DRAFT
{fb_t}

### DECISION
Based on the feedback above:
```

```
- Answer STOP if the draft does not need further refinement.
- Answer CONTINUE if the feedback raises significant issues worth
  addressing.

You MUST output exactly one of the following two lines and nothing else:
<decision>STOP</decision>
<decision>CONTINUE</decision>
```

Refinement prompt (step iii)

```
{few_shot_examples}### TASK INPUT
{x}

### REVISION HISTORY
{history}

### INSTRUCTIONS
Revise the most recent draft by applying the most recent feedback.
- Fix every issue mentioned in the feedback.
- Preserve all task requirements and constraints.
- Do not introduce new unrelated content or regressions.
- Output ONLY the refined draft (no explanations, no scores, no
  feedback).

### REFINED OUTPUT
```

Fusion prompt. Fusion uses the direct-synthesis variant (a two-step compare-then-fuse variant exists in the implementation but is not used in this paper). `{generations}` is the candidate pool rendered as `## Generation 1`, `## Generation 2`, ..., and `{instruction}` is the original task input.

Fusion prompt (direct synthesis)

```
Based on the provided Task Input and Generated Texts, fuse them into a
better generation that combines the strength of each of them. The fused
generation should adequately respond to the task input, sound natural to
a native speaker, and be focused on conveying the most relevant and
accurate information in a responsible and ethical way.

### Generated Texts
{generations}

### Task Input
{instruction}

Output only the fused generation text, as if you were directly answering
the task yourself. Do not reference or mention the previous generations
(e.g., avoid phrases like "combining the best of both responses" or "as
mentioned in Generation 1").

Please provide your fused text.
```

Other methods. BoN, Beam Search, Particle Filter, and Budget Forcing use only each benchmark's native prompt together with the system prompt above; no custom user template is added.

LLM-Judge input tokens. Table 10 reports the total input tokens sent to the LLM-as-Judge across all the judge runs in our evaluation, aggregated by algorithm family × benchmark. Grand total: 27,540 M tokens (≈ 27.5 B).

Table 10 Total LLM-Judge input tokens (millions) by algorithm family and benchmark.

Algorithm	HealthBench	LEXam	PRBench	WildBench	WritingBench	Total
BoN (oracle, $n = 16$)	4,556	202	4,995	173	506	10,432
PF-High (oracle, $n = 16$)	1,519	74	1,704	54	176	3,527
Fusion	1,524	61	2,219	48	164	4,015
Beam Search	1,146	44	1,657	37	120	3,004
PF (normal)	1,146	44	1,657	37	120	3,004
Budget Forcing	764	29	1,105	25	80	2,003
Sequential Refinement	638	25	814	16	61	1,555
Total	11,293	479	14,151	390	1,227	27,540

Combined paper-level compute totals. Total compute across all experiments was approximately 23,800 GPU-hours on 8×B200 nodes Table 11

Table 11 Combined paper-level totals across Open-QA and verifiable evaluation tracks. Wall-clock hours measure end-to-end run duration; GPU-hours scale by node size.

	Tasks	Wall-clock (h)	GPU-hours
Open-QA	164	2,642.6	21,104
Verifiable	93	342.2	2,737
Total	257	2,984.8	≈23,800

D Judge Agreement Analysis

Why a unified judge. Running the native judges (Table 7, Sec B) across the full evaluation grid is prohibitive: per-criterion designs (HealthBench, PRBench, WritingBench) fan out into hundreds of thousands of GPT-4.1, o4-mini, and Claude 3.7 Sonnet calls, and consequently prohibitive cost. We therefore use **Qwen3.5-397B-A17B**, as the unified judge for all main-text results, and validate against each benchmark's native evaluation on a stratified sample.

Unified setup. All judge models receive each native prompt verbatim and evaluate the same candidate completions (with reasoning tags stripped), so the judge model is the sole variable across comparisons. All runs share the following sampling configuration:

Qwen3.5 Judge — Sampling Configuration

```
sampling:
  temperature: 0.7
  top_p:       0.8
  top_k:       20
  min_p:       0
  chat_template_kwargs:
    enable_thinking: false
```

Full config can be shared upon request.

Sample selection. We draw a stratified random sample over task splits per benchmark, preserving subpopulation proportions. The sampling rate is 15% of items for LEXam, WildBench, and WritingBench, and 5% for PRBench (reduced due to its high per-item criterion count, averaging ∼17.7 criteria per item). Both judges score every completion on identical inputs. For benchmarks with per-criterion evaluation (PRBench, WritingBench), the pair counts in Table 13 reflect the total number of criterion-level judgments (sampled items × average criteria per item); for LEXam and WildBench, which produce a single holistic score per item, the pair count equals the number of sampled items directly.

HealthBench: validation against physician annotations. HealthBench is the only benchmark in our suite that had public human expert annotations, enabling a stronger validation than unified-vs-native comparison. We evaluate our unified judge against the physician rubric annotations released with the benchmark. Table 12 reports Macro F1 for several judge models on this task. Qwen3.5-397B-A17B achieves a Macro F1 of 0.679, comparable to GPT-4.1 (0.709) and o4-mini (0.692). The inter-physician pairwise agreement across the 184 physicians with sufficient annotation data has a mean Macro F1 of 0.655 (std = 0.102, range $[0.04, 1.0]$). Our unified judge sits at the 57th percentile of this distribution, meaning it agrees with physicians better than the majority of individual physicians agree with each other.

Table 12 HealthBench judge agreement with physician annotations (Macro F1). The inter-physician baseline is the mean pairwise Macro F1 across 184 physicians with sufficient annotation data.

Judge	Macro F1
GPT-4.1	0.709
o4-mini	0.692
o3	0.681
Qwen3.5-397B-A17B (ours)	**0.679**
GPT-4.1 mini	0.661
Inter-physician mean	0.655 ($\pm$ 0.102)

Other benchmarks: unified vs. native judge. For the remaining four benchmarks, we compare the unified against each benchmark's native judge (Table 7). Table 13 reports the agreement.

Table 13 Criterion-level agreement between the unified (Qwen3.5-397B-A17B) and each benchmark's native judge. PRBench is evaluated on a 5% sample; the remaining benchmarks on a 15% sample. Metrics are reported in the form most natural to each native protocol and are not directly comparable across rows.

Benchmark	n pairs	Metric	Agreement
PRBench	1,186	κ / Macro F1	0.679 / 0.838
LEXam	77	Pearson r / MAE	0.813 / 22.2
WildBench	147	QWK / MAE	0.564 / 1.35
WritingBench	404	QWK / MAE	0.408 / 1.51

PRBench and LEXam show the strongest alignment. WildBench and WritingBench show lower per-pair agreement (QWK = 0.564 and 0.408), driven by the heterogeneity of evaluation criteria across task types and domains.

E Theory

Self-verifier methods (Fusion, Sequential Refinement). These methods use the generator itself to produce the final output, either by synthesising across candidates (Fusion) or by iteratively critiquing and rewriting (Sequential Refinement). Their headroom capture does not depend on an external verifier's correlation with true quality. Instead, by the data processing inequality, the mutual information between the self-verifier's

signal and true quality is bounded by the generator's own distributional quality:

$$I(\mathcal{V}_{=\theta}(y);\, Q_{\text{true}}(y)) \;\le\; I(\pi_\theta(y \mid x);\, Q_{\text{true}}(y))\,. \tag{5}$$

A more capable generator has higher mutual information with true quality, raising the ceiling on headroom capture for self-verifier methods. Critically, this bound is on *capability* (the ability to produce and assess high-quality outputs), not on parameter count alone.

Verifier correlation under noisy scoring. When the external verifier correlates imperfectly with true quality at rate $\rho_v = \text{Corr}[\mathcal{V}(y), \mathcal{V}^*(y)]$, the expected true quality of the BoN-selected candidate satisfies:

$$Q^{\text{BoN}}(N, \rho_v) \;\approx\; \mu \;+\; \rho_v \cdot \sigma \cdot \Phi^{-1}\!\left(\frac{N}{N+1}\right), \tag{6}$$

Substituting into the headroom capture Equation 2 with $Q^*(N) \approx \mu + \sigma\,\Phi^{-1}(N/(N+1))$ from Equation 3:

$$h^{\text{BoN}} \;=\; \frac{Q^{\text{BoN}} - \mu}{Q^* - \mu} \;=\; \frac{\rho_v \cdot \sigma \cdot \Phi^{-1}\!\left(\frac{N}{N+1}\right)}{\sigma \cdot \Phi^{-1}\!\left(\frac{N}{N+1}\right)} \;=\; \rho_v. \tag{7}$$

Headroom capture for BoN *is* the verifier correlation. When $\rho_v = 1$ the method captures all available quality; when $\rho_v = 0$ additional compute yields no benefit; when $\rho_v < 0$ more compute actively harms performance.

Tree search pruning sensitivity. Tree search is exponentially more sensitive to verifier miscalibration than BoN. Let ϵ_{prune} denote the per-step probability that the PRM incorrectly prunes the highest-quality surviving candidate. The probability that the best candidate survives to depth d decays as:

$$P(\text{best survives to depth } d) \;=\; (1 - \epsilon_{\text{prune}})^d. \tag{8}$$

At $\epsilon_{\text{prune}} = 0.33$ and $d = 4$, the best candidate survives with probability ≤ 0.20, meaning tree search is expected to return a suboptimal candidate more than 80% of the time even under moderate miscalibration. This formalises why Beam Search and Particle Filter perform strictly worse than naive parallel sampling on open-ended QA, where PRM miscalibration is severe (Section 6.2).

F Oracle Bias From Judge Noise

Setup. The judge here is the *final reference-based evaluation* — it scores candidates after the algorithm has already chosen what to do, and is not part of the algorithm itself. We start by studying the the BoN ceiling ("oracle") due to its simplicity of having independent candidates. Each of the n candidates are passed through the judge and then the max is taken. **Judge noise inflates that max upward.** For a given input, candidate $i \in \{1, \dots, n\}$ has true mean score μ_i (infinite-trial judge average). The judge realizes $X_i = \mu_i + \epsilon_i$ with $\epsilon_i \overset{\text{i.i.d.}}{\sim} \mathcal{N}(0, \sigma_J^2)$. **Notation.** Let $Z_1, \dots, Z_n \overset{\text{i.i.d.}}{\sim} \mathcal{N}(0,1)$ be a generic set of i.i.d. standard normals (not the candidate scores — introduced solely to define one constant), and

$$a_n := \mathbb{E}\big[\max_{i \in \{1,\dots,n\}} Z_i\big],$$

the expected maximum of n i.i.d. standard normals. Numerically: $a_2 \approx 0.564$, $a_4 \approx 1.029$, $a_8 \approx 1.424$, $a_{16} \approx 1.766$, growing like $\sqrt{2 \ln n}$. Two quantities of interest:

1. **Naive oracle** $O^{\text{naive}} = \max_i X_i$ — **overstates** the BoN ceiling.
2. **Ideal oracle** $O^{\text{ideal}} = \max_i \mu_i$ — true quality of the genuinely best candidate, i.e. what an infinite-trial judge would report.

F.1 Derivation of the expectations

Two facts used repeatedly:

- **(F1) Sum of independent normals.** If $A \sim \mathcal{N}(\alpha, \sigma_A^2)$ and $B \sim \mathcal{N}(\beta, \sigma_B^2)$ are independent, then $A + B \sim \mathcal{N}(\alpha + \beta, \sigma_A^2 + \sigma_B^2)$.
- **(F2) Affine equivariance of the max.** For constants a and $b > 0$ and any random variables $V_1, \ldots, V_n$, $\max_i(a + bV_i) = a + b \max_i V_i$, so $\mathbb{E}[\max_i(a + bV_i)] = a + b\,\mathbb{E}[\max_i V_i]$.

Ideal. Assume the distribution of true scores from our algorithm follows $\mu_i \overset{\text{i.i.d.}}{\sim} \mathcal{N}(\bar{\mu}, \tau^2)$, independent of the ϵ_i, so we can write $\mu_i = \bar{\mu} + \tau Z_i$ with $Z_i \overset{\text{i.i.d.}}{\sim} \mathcal{N}(0, 1)$. By (F2),

$$\mathbb{E}[O^{\text{ideal}}] = \mathbb{E}[\max_i \mu_i] = \mathbb{E}[\max_i(\bar{\mu} + \tau Z_i)] = \bar{\mu} + \tau\,\mathbb{E}[\max_i Z_i] = \boxed{\bar{\mu} + a_n\,\tau}.$$

Naive. Apply (F1) to $X_i = \mu_i + \epsilon_i$: since μ_i and ϵ_i are independent and Gaussian,

$$X_i \sim \mathcal{N}(\bar{\mu}, \tau^2 + \sigma_J^2),$$

and the X_i's are i.i.d. (the pairs (μ_i, ϵ_i) are mutually independent, so the sums are too). Standardize: $X_i = \bar{\mu} + \sqrt{\tau^2 + \sigma_J^2}\, Z_i'$ with $Z_i' \overset{\text{i.i.d.}}{\sim} \mathcal{N}(0, 1)$. By (F2),

$$\mathbb{E}[O^{\text{naive}}] = \mathbb{E}[\max_i X_i] = \bar{\mu} + \sqrt{\tau^2 + \sigma_J^2}\,\mathbb{E}[\max_i Z_i'] = \boxed{\bar{\mu} + a_n\sqrt{\tau^2 + \sigma_J^2}}.$$

Gap. Subtracting,

$$\mathbb{E}[O^{\text{naive}}] - \mathbb{E}[O^{\text{ideal}}] = a_n\big[\sqrt{\tau^2 + \sigma_J^2} - \tau\big] = a_n \cdot \frac{\sigma_J^2}{\sqrt{\tau^2 + \sigma_J^2} + \tau}$$

Can see that: $\sigma_J \to 0 \Rightarrow$ gap $\to 0$; $\tau = 0 \Rightarrow$ gap $= a_n\sigma_J$ (no true spread, all the apparent winner's lead is noise); $\tau \to \infty \Rightarrow$ gap $\to 0$ (a real quality gap drowns out the noise); grows with n via a_n.

Ideal per-entry estimator

The correction is just the closed-form bias evaluated at the per-entry candidate spread:

$$\hat{O}^{\text{ideal}}_{\text{entry}} = \max_i X_i - a_n\big[\sqrt{\hat{\tau}^2 + \sigma_J^2} - \hat{\tau}\big], \qquad \hat{\tau}^2 = \max\big(0,\ s_X^2 - \sigma_J^2\big),$$

where s_X^2 is the sample variance of the n candidate scores in this entry (so $\mathbb{E}[s_X^2] = \tau^2 + \sigma_J^2$ and $\hat{\tau}^2$ is a plug-in for τ^2). Under the Gaussian model with oracle τ, this is unbiased for $\mathbb{E}[O^{\text{ideal}}]$. With plug-in $\hat{\tau}^2$ the estimator has small finite-sample bias driven by the clip-at-zero (which biases $\hat{\tau}$ upward when true τ is small, hence biases the correction *downward* — i.e. corrects too little when τ is near 0; conservative).

Approximated per-benchmark correction We sampled 15-20% of prompts per benchmark with a minimum of 100, then generated a single response from Qwen3.5-35B for each, and ran the judge four times per generation to produce a within-entry variance estimate. We make two simplifying assumptions: (i) judge variance is constant across entries within a benchmark and (ii) constant across the models we evaluate. Under (i)–(ii) a single per-benchmark σ_J^2 suffices for every algorithm and model. The resulting estimates are listed in Table 14.

With σ_J^2 in hand, the per-entry estimator $\hat{O}^{\text{ideal}}_{\text{entry}}$ defined in Section F.1 can be evaluated directly: a_n is known, σ_J^2 is read from Table 14, and s_X^2 is the empirical variance of the candidate scores within the entry. For BoN this is unambiguous — the n independent generations are the candidate pool by construction — and Figure 6 shows the resulting adjustment across compute levels.

Benchmark	Square root of mean var (normalized)
HealthBench	0.076
PRBench	0.034
LEXam	0.055
WildBench	0.055
WritingBench	0.030

Table 14 Square root of the per-benchmark mean per-entry judge variance (micro-averaged across entries).

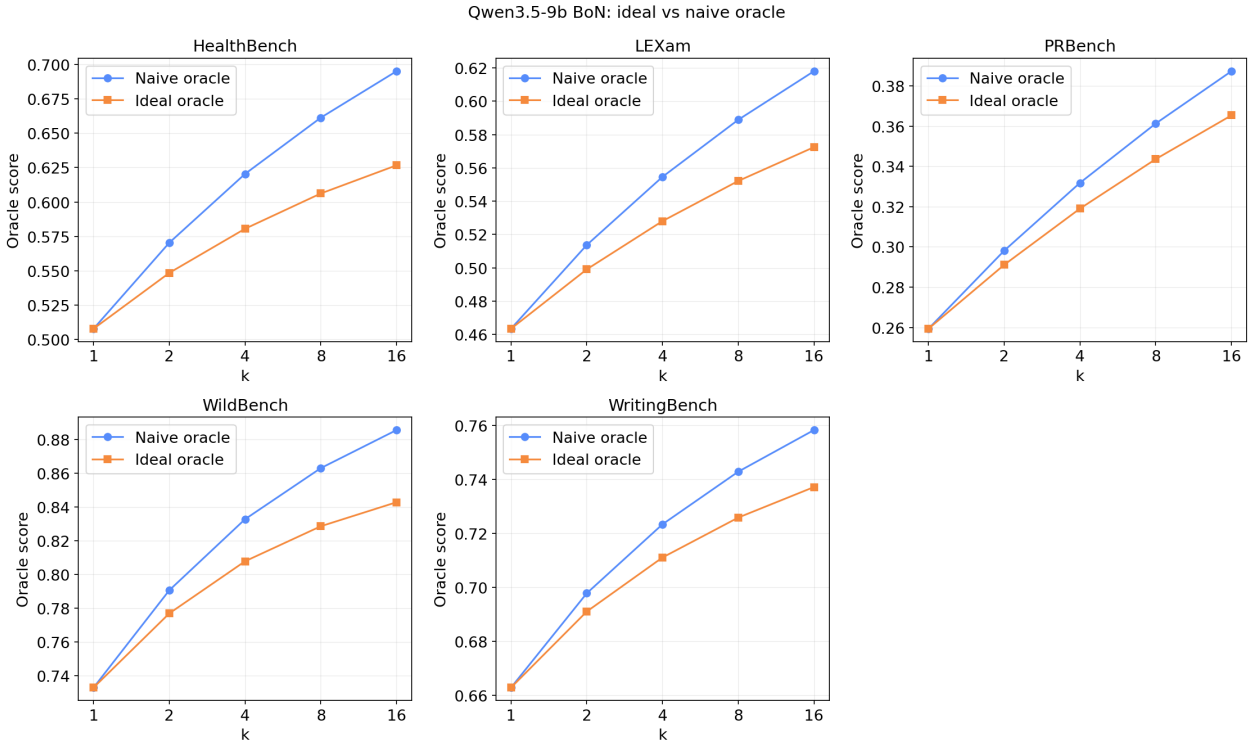

Figure 6 Naive (unadjusted) vs ideal oracle scores across compute for qwen3.5-9b BoN.

The other algorithms in our study do not contain i.i.d. candidate pools, so applying the same correction requires algorithm-specific choices. For simplicity, we assume the candidate pools of Refinement, Fusion, and Particle Filtering are all i.i.d. and then discuss the limitations in the next section. The candidate pools are defined as the following for compute C (assume corresponds to BoN@k):

- **Refinement.** The drafts in a trajectory that uses amount of compute C.
- **Fusion.** The set of k-1 i.i.d. samples along with the fused response.
- **Particle Filter.** The pool is the final k particles.

Limitations of the i.i.d. assumption. The closed-form bias is exact only when the n candidates are i.i.d. Gaussian. Below we work through the qualitative behaviour we expect from the i.i.d. plug-in correction under each algorithm's structural deviation from this assumption; the magnitude and even the direction of the residual error depend on the underlying generator's behaviour and should be read as hypotheses rather than guarantees.

- **Fusion.** When fusion works well, the fused response has a higher mean than the i.i.d. input samples. In the limit as the fused-mean minus input-mean grows, the naive oracle picks the fused sample with

probability approaching one, judge noise on that single chosen candidate averages out, and the true bias of the naive oracle approaches zero. Our estimator, however, plugs in s_X^2 from the (fused, inputs) pool: the fused-vs-input gap inflates s_X^2 as if it were within-pool spread, and the formula continues to subtract a positive correction. The reported ideal oracle is therefore expected to be a slight *lower bound* when the model is strong at fusing responses.

- **Refinement.** Refinements within a trajectory are expected to be correlated. A scenario of interest is monotonic improvement, where the last refinement consistently has the highest mean and the naive oracle picks it with probability approaching one — the same situation as Fusion, with the last refinement playing the role of the dominating fused response. The fused-vs-input mean gap is replaced by the first-vs-last drift across the trajectory, s_X^2 is inflated by that drift, and the formula continues to subtract a positive correction even though the true bias has approached zero. Assuming strong refinement capabilities, we'd expect the ideal oracle to be a slight *lower bound*.
- **Particle Filter.** Resampling concentrates particles into clusters of near-identical trajectories that share a true mean within each cluster. A scenario of interest is when resampling produces a few well-separated clusters of size c: the naive oracle picks the maximum within the best-mean cluster, which is inflated above the true ideal by the maximum of c judge-noise terms ($\approx a_c \sigma_J$). Our estimator's s_X^2, however, is dominated by the between-cluster mean gaps and inflates $\hat{\tau}$ well above the within-cluster noise scale, so the formula subtracts a much smaller correction ($\sim a_n \sigma_J^2/(2\hat{\tau})$) than the true within-cluster max-of-noise inflation. Therefore, we expect the ideal oracle to be a slight *upper bound*, opposite in direction from Fusion and Refinement.

F.2 Consistency of $h \approx \rho_v$ under judge noise

Looking at Eq. 4 , judge noise affects both sides of the identity $h \approx \rho_v$ symmetrically, so the linear relationship is preserved even without the oracle correction. To see this, note that the judge scores $J(y) = \mathcal{V}^*(y) + \epsilon_J$ with $\epsilon_J \sim \mathcal{N}(0, \sigma_J^2)$ enter two quantities:

Effect on measured headroom capture $\hat{h}$. The numerator $\hat{Q}(T) - \mu$ is the expected judge score of the RM-selected candidate minus baseline. Since RM selection is based on $\mathcal{V}(y)$ and judge noise ϵ_J is independent of $\mathcal{V}$, the noise averages out in expectation:

$$\hat{Q}(T) - \mu \;\approx\; \rho_v \cdot a_N \tau.$$

The denominator $\hat{Q}^*(T) - \mu$ is the expected maximum of N i.i.d. judge scores, which by the derivation in Section F.1 equals $a_N \sqrt{\tau^2 + \sigma_J^2}$. Therefore:

$$\hat{h} \;=\; \frac{\hat{Q}(T) - \mu}{\hat{Q}^*(T) - \mu} \;\approx\; \rho_v \cdot \frac{\tau}{\sqrt{\tau^2 + \sigma_J^2}}.$$

Effect on measured verifier correlation $\hat{\rho}_v$.

We measure $\hat{\rho}_v = \mathrm{Corr}[\mathcal{V}(y),\, J(y)]$, whereas the theoretical $\rho_v = \mathrm{Corr}[\mathcal{V}(y),\, \mathcal{V}^*(y)]$. Since $J(y) = \mathcal{V}^*(y) + \epsilon_J$ with ϵ_J independent of $\mathcal{V}$, standard attenuation gives:

$$\hat{\rho}_v \;=\; \rho_v \cdot \sqrt{\frac{\tau^2}{\tau^2 + \sigma_J^2}} \;=\; \rho_v \cdot \frac{\tau}{\sqrt{\tau^2 + \sigma_J^2}}.$$

Cancellation. Both quantities are attenuated by the same factor $\tau/\sqrt{\tau^2 + \sigma_J^2}$, so:

$$\hat{h} \;\approx\; \hat{\rho}_v,$$

and the identity $h \approx \rho_v$ holds at the level of *measured* quantities without any correction. The oracle correction in Section F.1 is therefore not required for the validity of Equation 4; it serves the separate purpose of obtaining unbiased estimates of true oracle quality $\mathbb{E}[O^{\mathrm{ideal}}]$.

Heterogeneous attenuation across benchmarks.

The attenuation factor $\tau/\sqrt{\tau^2+\sigma_J^2}$ varies across benchmarks because both τ (true spread of candidate quality) and σ_J (judge noise, Table 14) differ. This introduces benchmark-level offsets that appear as scatter around the identity line in Figure 3 (left), but do not bias the slope in expectation. The observed slope of 1.198 and $R^2 = 0.656$ are consistent with this prediction.

G Naive Oracle Results

Figures 7, 8, and 9 replicate Figures 2 and 3 using the naive oracle $\hat{O}^{\text{naive}} = \max_i J(y_i)$, which overstates true pool quality by selecting partly on judge noise. Despite the inflated ceiling, the conclusion is unchanged: oracle quality rises with compute while realised quality stagnates, and exploitation remains the binding constraint. The naive oracle inflates the apparent gap, making the exploitation failure look more severe, not less.

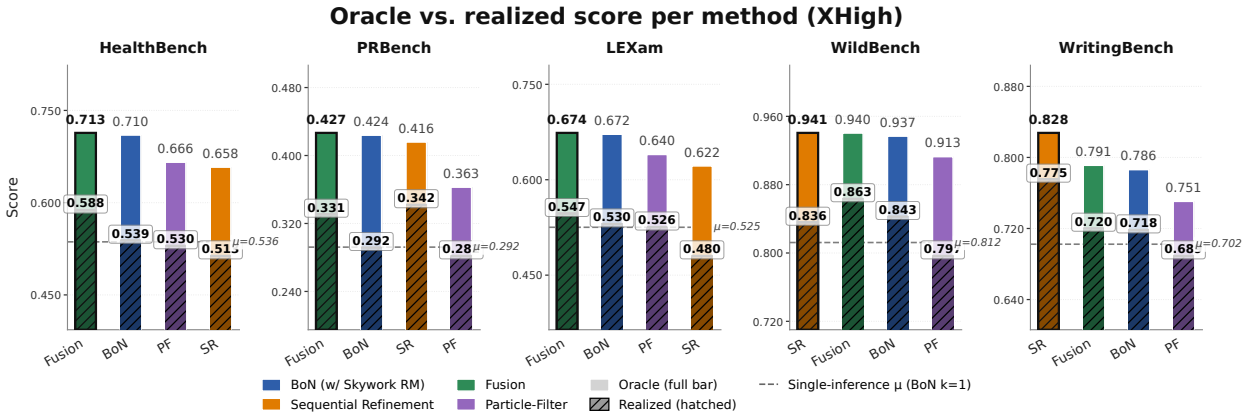

Figure 7 Oracle (full bar) vs. realised (hatched) quality per method at matched compute (**XHigh**), Qwen3.5-35B-A3B, using the **naive** (uncorrected) oracle.

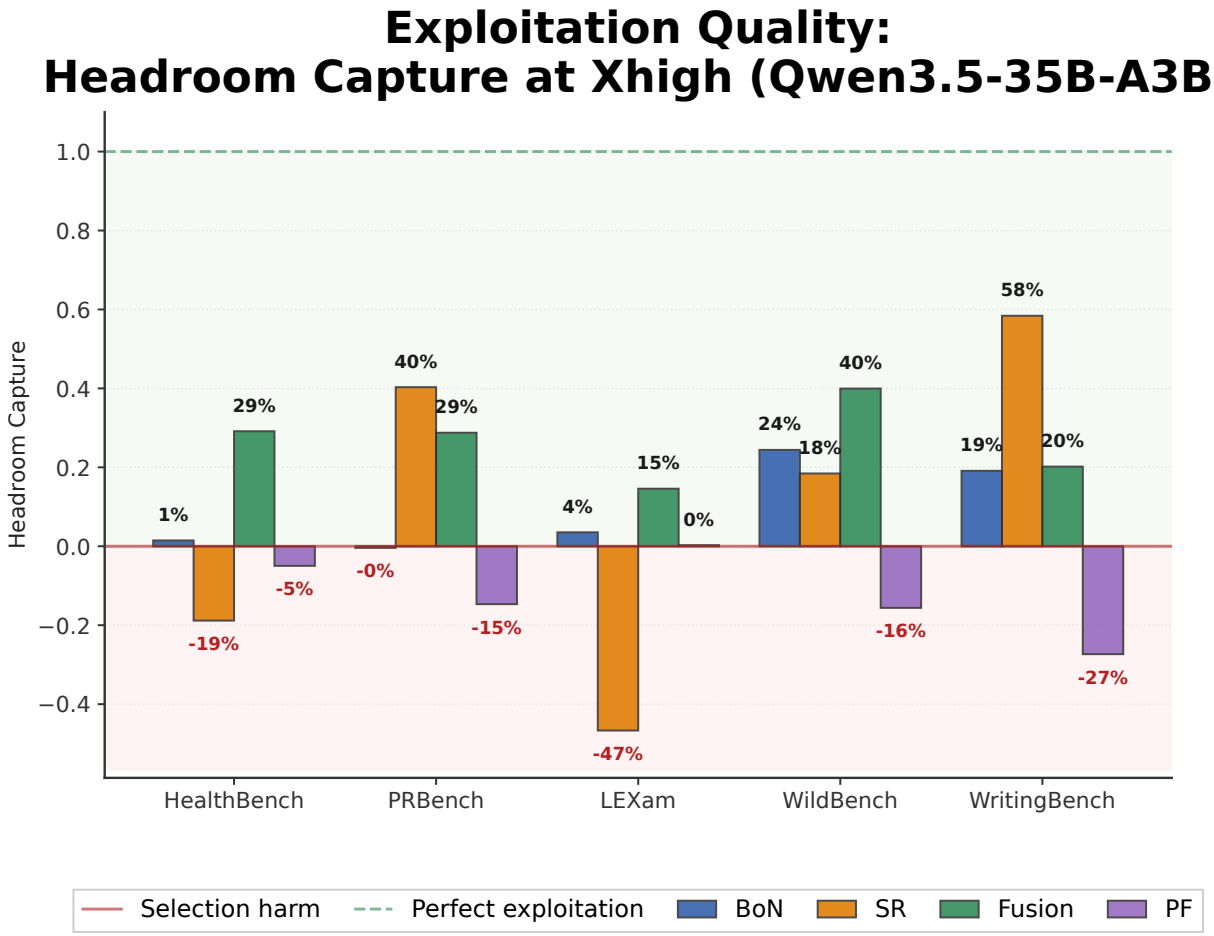

Figure 8 Headroom capture per method at XHigh compute, using the **naive** (uncorrected) oracle. Compare with Figure 3 (right) which uses the bias-corrected oracle.

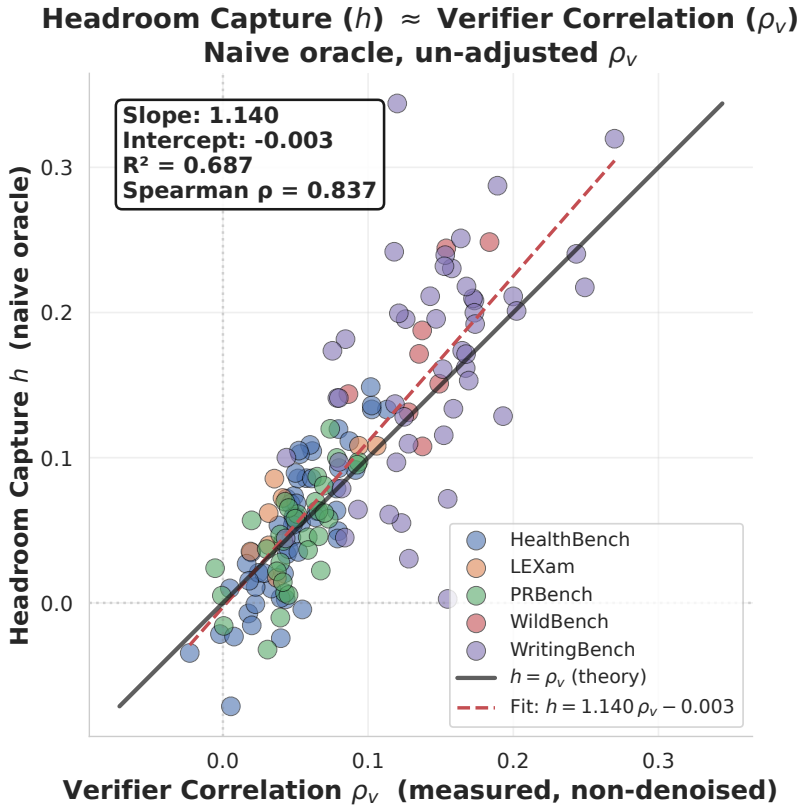

Figure 9 h vs. ρ_v across $N = 152$ task-level points using the **naive** (uncorrected) oracle and **measured** (non-denoised) ρ_v. The empirical fit closely tracks the identity line $y = x$, mirroring the linear relationship $\hat{h} \approx \hat{\rho}_v$ that Figure 3 (left) demonstrates under the bias-corrected oracle and denoised $\hat{\rho}_v$. Same conclusion holds under both conventions: headroom capture is bounded by verifier correlation regardless of whether we de-noise the oracle and the correlation estimate.

G.1 Headroom capture tables (Unadjusted oracles)

Table 15 Headroom capture for **Qwen3.5-35B**: $h = (\text{realized} - \mu)/(\text{oracle}_{\text{ideal}} - \mu)$, where μ is the BoN N=1 baseline (single-inference mean). $h = 1$ means full exploitation of the oracle gap; $h = 0$ matches μ; $h < 0$ underperforms μ. green = positive (gain captured), red = negative (below baseline). Bold when $|h| \geq 0.20$.

	HealthBench				PRBench				LEXam			
	Low	Mid	High	XHigh	Low	Mid	High	XHigh	Low	Mid	High	XHigh
BoN+Skywork	+0.047	+0.038	+0.030	+0.025	+0.038	+0.023	+0.014	-0.005	+0.047	+0.062	+0.065	+0.051
Fusion	—	**+0.670**	**+0.580**	**+0.495**	—	**+0.390**	**+0.357**	**+0.339**	—	**+0.355**	**+0.246**	**+0.209**
Particle Filter	—	—	—	-0.168	—	—	—	**-0.293**	—	—	—	+0.006
Self-Refine	**-1.096**	**-0.907**	**-0.553**	**-0.399**	**+0.455**	**+0.467**	**+0.495**	**+0.496**	**-1.148**	**-0.833**	**-0.959**	**-0.858**

	WildBench				WritingBench				Overall			
	Low	Mid	High	XHigh	Low	Mid	High	XHigh	Low	Mid	High	XHigh
BoN+Skywork	**+0.375**	**+0.391**	**+0.400**	**+0.413**	**+0.252**	**+0.261**	**+0.265**	**+0.273**	+0.152	+0.155	+0.155	+0.152
Fusion	—	**+0.547**	**+0.642**	**+0.662**	—	**+0.288**	+0.181	**+0.275**	—	**+0.450**	**+0.401**	**+0.396**
Particle Filter	—	—	—	**-0.392**	—	—	—	**-1.167**	—	—	—	**-0.403**
Self-Refine	**+0.353**	**+0.342**	**+0.286**	**+0.303**	**+0.869**	**+0.767**	**+0.792**	**+0.703**	-0.114	-0.033	+0.012	+0.049

Table 18 Headroom capture for **OLMo3-32B**: $h = (\text{realized} - \mu)/(\text{oracle}_{\text{ideal}} - \mu)$, where μ is the BoN N=1 baseline (single-inference mean). $h = 1$ means full exploitation of the oracle gap; $h = 0$ matches μ; $h < 0$ underperforms μ. green = positive (gain captured), red = negative (below baseline). Bold when $|h| \geq 0.20$.

	HealthBench				PRBench				LEXam			
	Low	Mid	High	XHigh	Low	Mid	High	XHigh	Low	Mid	High	XHigh
BoN+Skywork	+0.099	+0.096	+0.090	+0.086	+0.102	+0.082	+0.070	+0.076	+0.085	+0.065	+0.049	+0.025
Fusion	—	**-0.426**	**-0.326**	—	—	**-0.979**	**-1.016**	—	—	**+0.231**	**+0.228**	—

	WildBench				WritingBench				Overall			
	Low	Mid	High	XHigh	Low	Mid	High	XHigh	Low	Mid	High	XHigh
BoN+Skywork	**+0.237**	**+0.221**	**+0.216**	**+0.225**	**+0.336**	**+0.312**	**+0.292**	**+0.255**	+0.172	+0.155	+0.143	+0.133
Fusion	—	**-0.892**	—	—	—	**-0.833**	**-0.204**	—	—	**-0.580**	**-0.330**	—

Table 16 Headroom capture for **Qwen3.5-9B**: $h = (\text{realized} - \mu)/(\text{oracle}_{\text{ideal}} - \mu)$, where μ is the BoN N=1 baseline (single-inference mean). $h = 1$ means full exploitation of the oracle gap; $h = 0$ matches μ; $h < 0$ underperforms μ. green = positive (gain captured), red = negative (below baseline). Bold when $|h| \geq 0.20$.

	HealthBench				PRBench				LEXam			
	Low	Mid	High	XHigh	Low	Mid	High	XHigh	Low	Mid	High	XHigh
BoN+Skywork	+0.084	+0.080	+0.084	+0.089	+0.077	+0.071	+0.061	+0.050	+0.073	+0.058	+0.059	+0.056
Fusion	—	**+0.615**	**+0.512**	**+0.473**	—	**+0.301**	**+0.346**	**+0.313**	—	**+0.419**	**+0.445**	**+0.339**
Self-Refine	**-3.615**	**-1.681**	**-1.175**	**-0.918**	**+0.251**	**+0.323**	**+0.404**	**+0.329**	**-1.104**	**-0.970**	**-0.952**	**-1.147**

	WildBench				WritingBench				Overall			
	Low	Mid	High	XHigh	Low	Mid	High	XHigh	Low	Mid	High	XHigh
BoN+Skywork	**+0.365**	**+0.358**	**+0.355**	**+0.345**	+0.200	**+0.204**	**+0.211**	**+0.206**	+0.160	+0.154	+0.154	+0.149
Fusion	—	**+0.492**	**+0.506**	**+0.560**	—	**+0.247**	+0.179	+0.123	—	**+0.415**	**+0.398**	**+0.362**
Self-Refine	+0.058	-0.027	+0.047	+0.026	**+0.833**	**+0.770**	**+0.700**	**+0.630**	**-0.715**	**-0.317**	-0.195	**-0.216**

Table 17 Headroom capture for **OLMo3-7B**: $h = (\text{realized} - \mu)/(\text{oracle}_{\text{ideal}} - \mu)$, where μ is the BoN N=1 baseline (single-inference mean). $h = 1$ means full exploitation of the oracle gap; $h = 0$ matches μ; $h < 0$ underperforms μ. green = positive (gain captured), red = negative (below baseline). Bold when $|h| \geq 0.20$.

	HealthBench				PRBench				LEXam			
	Low	Mid	High	XHigh	Low	Mid	High	XHigh	Low	Mid	High	XHigh
BoN+Skywork	+0.177	+0.165	+0.160	+0.165	+0.170	+0.137	+0.119	+0.110	**+0.216**	+0.171	+0.152	+0.157
Fusion	—	**+0.207**	**+0.292**	—	—	+0.059	**+0.224**	—	—	**+0.353**	**+0.286**	—

	WildBench				WritingBench				Overall			
	Low	Mid	High	XHigh	Low	Mid	High	XHigh	Low	Mid	High	XHigh
BoN+Skywork	**+0.230**	+0.196	+0.184	+0.182	+0.158	+0.138	+0.126	+0.125	+0.190	+0.161	+0.148	+0.148
Fusion	—	-0.190	—	—	—	+0.083	+0.145	—	—	+0.102	**+0.237**	—

H Qwen3.5 Results

H.1 Qwen3.5-35B-A3B Oracle Results

Table 19 Oracle quality for **Qwen3.5-35B** across five benchmarks and four compute levels. Cell colour encodes delta from BoN oracle N=1 baseline: steel blue = large gain, pale blue = small gain over BoN N=1 baseline. Bold = gain ≥ 0.03 over baseline.

	HealthBench					PRBench					LEXam				
	N=1	Low	Mid	High	XHigh	N=1	Low	Mid	High	XHigh	N=1	Low	Mid	High	XHigh
BoN oracle	**0.536**	**0.571**	**0.599**	**0.620**	**0.637**	**0.292**	**0.325**	**0.354**	**0.380**	**0.403**	**0.525**	**0.560**	**0.587**	**0.609**	**0.626**
Fusion	—	—	**0.593**	**0.624**	**0.641**	—	—	**0.349**	**0.382**	**0.406**	—	—	**0.584**	**0.612**	**0.629**
Particle Filter	—	—	—	—	**0.575**	—	—	—	—	**0.327**	—	—	—	—	**0.580**
Self-Refine	—	0.551	0.558	**0.572**	**0.594**	—	0.313	**0.333**	**0.363**	**0.393**	—	0.543	0.551	**0.564**	**0.578**

	WildBench					WritingBench					Overall				
	N=1	Low	Mid	High	XHigh	N=1	Low	Mid	High	XHigh	N=1	Low	Mid	High	XHigh
BoN oracle	**0.812**	**0.846**	**0.867**	**0.879**	**0.886**	**0.702**	0.724	**0.740**	**0.752**	**0.761**	**0.574**	**0.605**	**0.629**	**0.648**	**0.663**
Fusion	—	—	**0.870**	**0.882**	**0.890**	—	—	**0.747**	**0.757**	**0.767**	—	—	**0.629**	**0.651**	**0.666**
Particle Filter	—	—	—	—	**0.852**	—	—	—	—	0.714	—	—	—	—	**0.610**
Self-Refine	—	**0.848**	**0.864**	**0.876**	**0.891**	—	**0.743**	**0.765**	**0.787**	**0.806**	—	0.600	**0.614**	**0.632**	**0.652**

H.2 Qwen3.5-9B Realised Results

Table 20 Realised quality for **Qwen3.5-9B** across benchmarks and four compute levels. Cell colour encodes delta from the single-sample baseline (BoN@1): green = improvement, red = regression. Bold = gain ≥ 0.03 over baseline.

	HealthBench					PRBench					LEXam				
	N=1	Low	Mid	High	XHigh	N=1	Low	Mid	High	XHigh	N=1	Low	Mid	High	XHigh
BoN+Skywork	**0.508**	0.511	0.514	0.516	0.518	**0.259**	0.262	0.264	0.265	0.265	**0.463**	0.466	0.467	0.469	0.470
BoN+Llama70B	**0.508**	0.511	0.514	0.517	0.519	**0.259**	0.261	0.262	0.263	0.262	**0.463**	0.468	0.471	0.475	0.475
Fusion	—	—	**0.548**	**0.560**	**0.566**	—	—	0.275	**0.290**	**0.293**	—	—	0.487	**0.502**	**0.501**
Beam Search	—	—	0.508	0.505	0.506	—	—	0.254	0.253	0.254	—	—	0.475	0.480	0.463
Particle Filter	—	—	0.503	0.506	0.502	—	—	0.251	0.251	0.248	—	—	0.471	0.465	0.467
Sequential Refinement	—	0.479	0.472	0.464	0.455	—	0.264	0.269	0.283	0.287	—	0.444	0.437	0.429	0.413
Budget Forcing	—	0.519	0.524	—	—	—	0.258	0.258	—	—	—	0.466	0.465	—	—

	WildBench					WritingBench					Overall				
	N=1	Low	Mid	High	XHigh	N=1	Low	Mid	High	XHigh	N=1	Low	Mid	High	XHigh
BoN+Skywork	**0.733**	0.749	0.760	**0.767**	**0.771**	**0.663**	0.668	0.673	0.676	0.678	**0.525**	0.531	0.535	0.538	0.540
BoN+Llama70B	**0.733**	0.744	0.750	0.753	0.753	**0.663**	0.669	0.673	0.676	0.678	**0.525**	0.531	0.534	0.537	0.537
Fusion	—	—	**0.771**	**0.783**	**0.798**	—	—	0.675	0.675	0.672	—	—	0.551	**0.562**	**0.566**
Beam Search	—	—	0.733	0.746	0.731	—	—	0.662	0.654	0.657	—	—	0.527	0.528	0.522
Particle Filter	—	—	0.720	0.721	0.718	—	—	0.654	0.661	0.650	—	—	0.520	0.521	0.517
Sequential Refinement	—	0.736	0.731	0.737	0.736	—	**0.696**	**0.710**	**0.724**	**0.731**	—	0.524	0.524	0.528	0.524
Budget Forcing	—	0.752	0.752	—	—	—	0.631	0.617	—	—	—	0.525	0.523	—	—

H.3 Qwen3.5-9B Oracle Results

Table 21 Oracle quality for **Qwen3.5-9B** across five benchmarks and four compute levels. Cell colour encodes delta from BoN oracle N=1 baseline: steel blue = large gain, pale blue = small gain over BoN N=1 baseline. Bold = gain ≥ 0.03 over baseline.

	HealthBench					PRBench					LEXam				
	N=1	Low	Mid	High	XHigh	N=1	Low	Mid	High	XHigh	N=1	Low	Mid	High	XHigh
BoN oracle	**0.508**	**0.549**	**0.581**	**0.606**	**0.627**	**0.259**	**0.291**	**0.319**	**0.344**	**0.365**	**0.463**	**0.499**	**0.528**	**0.552**	**0.572**
Fusion	—	—	**0.574**	**0.609**	**0.631**	—	—	**0.312**	**0.347**	**0.368**	—	—	**0.521**	**0.549**	**0.573**
Self-Refine	—	0.516	0.529	**0.545**	**0.566**	—	0.276	0.289	**0.317**	**0.343**	—	0.481	0.490	**0.499**	**0.508**

	WildBench					WritingBench					Overall				
	N=1	Low	Mid	High	XHigh	N=1	Low	Mid	High	XHigh	N=1	Low	Mid	High	XHigh
BoN oracle	**0.733**	**0.777**	**0.808**	**0.829**	**0.843**	**0.663**	0.691	**0.711**	**0.726**	**0.737**	**0.525**	**0.561**	**0.589**	**0.611**	**0.629**
Fusion	—	—	**0.811**	**0.832**	**0.849**	—	—	**0.710**	**0.729**	**0.740**	—	—	**0.586**	**0.613**	**0.632**
Self-Refine	—	**0.777**	**0.795**	**0.826**	**0.843**	—	**0.702**	**0.724**	**0.750**	**0.770**	—	0.551	**0.565**	**0.587**	**0.606**

I OLMo3 Results

Tables 22 and 23 report realised quality for OLMo3-32B and OLMo3-7B respectively. Tables 24–25 report the corresponding oracle scores. Sequential Refinement was not evaluated on OLMo3 due to compute constraints.

The OLMo3 results serve as the primary negative case for our central thesis: generation capability does not imply exploitation capability.

Fusion severely degrades OLMo3-32B. Fusion—the only method to consistently improve Qwen3.5—produces large regressions on OLMo3-32B. On PRBench, realised quality drops from 0.211 (baseline) to 0.164 at High (−22% relative). WildBench drops from 0.721 to 0.653, HealthBench from 0.480 to 0.450, and WritingBench from 0.569 to 0.516. Only LEXam shows marginal improvement (0.395 → 0.414). Overall, OLMo3-32B Fusion scores 0.441 at Mid and 0.394 at High, well below the 0.475 baseline, and the damage worsens with additional compute. This constitutes *negative* headroom capture: the synthesised output is systematically worse than a random candidate from the same pool. The contrast with Qwen3.5-35B-A3B, where Fusion captures ∼40% of headroom and improves on every benchmark, demonstrates that exploitation is a distinct capability from generation.

OLMo3-7B Fusion is mixed but not broken. The smaller OLMo3-7B does not exhibit the same Fusion failure. At High compute it improves over baseline on HealthBench (+3.0pp), LEXam (+2.3pp), PRBench (+1.2pp), and WritingBench (+1.2pp), while regressing on WildBench at Mid (−1.7pp). This partial success suggests that the 32B failure is not a simple property of the OLMo3 architecture, but reflects how scaling interacts with synthesis capability in this model family.

Budget Forcing degenerates on OLMo3. Budget Forcing produces the largest regressions of any method. OLMo3-32B drops from 0.475 overall to 0.393 at Mid; individual benchmarks are worse: PRBench 0.211 → 0.157, WritingBench 0.569 → 0.421. OLMo3-7B follows the same pattern (overall 0.356 → 0.288). Additional thinking tokens produce degenerate outputs rather than deeper reasoning.

Oracle quality confirms the dissociation. OLMo3-7B oracle quality rises steadily with compute (Table 24): overall from 0.356 at N=1 to 0.474 at XHigh, with Fusion oracle tracking BoN oracle closely across compute levels. The candidate pool quality is proportionally comparable to Qwen3.5, confirming that the divergence in realised quality is driven entirely by exploitation, not exploration.

Table 22 Realised quality for **OLMo3-32B** across five benchmarks and four compute levels. Cell colour encodes delta from the single-sample baseline (BoN@1): green = improvement, red = regression. Bold = gain ≥ 0.03 over baseline.

	HealthBench					PRBench					LEXam				
	N=1	Low	Mid	High	XHigh	N=1	Low	Mid	High	XHigh	N=1	Low	Mid	High	XHigh
BoN+Skywork	**0.480**	0.484	0.487	0.489	0.490	**0.211**	0.213	0.215	0.215	0.217	**0.395**	0.398	0.399	0.399	0.398
BoN+Llama70B	**0.480**	0.483	0.486	0.487	0.488	**0.211**	0.212	0.214	0.215	0.218	**0.395**	0.399	0.401	0.405	0.408
Fusion	—	—	0.454	0.450	—	—	—	0.174	0.164	—	—	—	0.409	0.414	—
Beam Search	—	—	0.479	0.480	—	—	—	0.211	0.212	—	—	—	0.393	0.386	—
Particle Filter	—	—	0.479	0.480	0.477	—	—	0.210	0.207	0.210	—	—	0.390	0.406	0.395
Sequential Refinement	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Budget Forcing	—	0.445	0.417	—	—	—	0.183	0.157	—	—	—	0.391	0.370	—	—

	WildBench					WritingBench					Overall				
	N=1	Low	Mid	High	XHigh	N=1	Low	Mid	High	XHigh	N=1	Low	Mid	High	XHigh
BoN+Skywork	**0.721**	0.733	0.739	0.744	0.749	**0.569**	0.582	0.590	0.595	0.596	**0.475**	0.482	0.486	0.488	0.490
BoN+Llama70B	**0.721**	0.733	0.741	0.746	**0.752**	**0.569**	0.579	0.586	0.590	0.595	**0.475**	0.481	0.486	0.488	0.492
Fusion	—	—	0.653	—	—	—	—	0.516	0.550	—	—	—	0.441	0.394	—
Beam Search	—	—	0.707	0.709	—	—	—	0.568	0.557	—	—	—	0.472	0.469	—
Particle Filter	—	—	0.707	0.709	0.713	—	—	0.573	0.568	0.566	—	—	0.472	0.474	0.472
Sequential Refinement	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Budget Forcing	—	0.676	0.599	—	—	—	0.501	0.421	—	—	—	0.439	0.393	—	—

Table 23 Realised quality for **OLMo3-7B** across five benchmarks and four compute levels. Cell colour encodes delta from the single-sample baseline (BoN@1): green = improvement, red = regression. Bold = gain ≥ 0.03 over baseline.

	HealthBench					PRBench					LEXam				
	N=1	Low	Mid	High	XHigh	N=1	Low	Mid	High	XHigh	N=1	Low	Mid	High	XHigh
BoN+Skywork	**0.335**	0.342	0.348	0.353	0.358	**0.142**	0.145	0.147	0.148	0.150	**0.281**	0.288	0.291	0.294	0.298
BoN+Llama70B	**0.335**	0.342	0.346	0.350	0.354	**0.142**	0.144	0.146	0.147	0.150	**0.281**	0.289	0.294	0.297	0.298
Fusion	—	—	0.349	**0.365**	—	—	—	0.144	0.154	—	—	—	0.300	0.304	—
Beam Search	—	—	0.332	0.328	0.329	—	—	0.142	0.139	0.141	—	—	0.278	0.278	0.272
Particle Filter	—	—	0.334	0.340	0.336	—	—	0.139	0.141	0.140	—	—	0.273	0.281	0.279
Sequential Refinement	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Budget Forcing	—	0.313	0.300	—	—	—	0.116	0.096	—	—	—	0.262	0.258	—	—

	WildBench					WritingBench					Overall				
	N=1	Low	Mid	High	XHigh	N=1	Low	Mid	High	XHigh	N=1	Low	Mid	High	XHigh
BoN+Skywork	**0.556**	0.568	0.575	0.581	**0.586**	**0.466**	0.472	0.475	0.477	0.479	**0.356**	0.363	0.367	0.371	0.374
BoN+Llama70B	**0.556**	0.567	0.574	0.578	0.577	**0.466**	0.471	0.474	0.476	0.479	**0.356**	0.363	0.367	0.370	0.372
Fusion	—	—	0.539	—	—	—	—	0.471	0.478	—	—	—	0.361	0.326	—
Beam Search	—	—	0.549	0.553	0.546	—	—	0.470	0.473	0.467	—	—	0.354	0.354	0.351
Particle Filter	—	—	0.550	0.550	0.546	—	—	0.463	0.469	0.469	—	—	0.352	0.356	0.354
Sequential Refinement	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Budget Forcing	—	0.489	0.435	—	—	—	0.372	0.349	—	—	—	0.310	0.288	—	—

I.1 Oracle Results Olmo-7B

Table 24 Oracle quality for **OLMo3-7B** across five benchmarks and four compute levels. Cell colour encodes delta from BoN oracle N=1 baseline: steel blue = large gain, pale blue = small gain over BoN N=1 baseline. Bold = gain ≥ 0.03 over baseline.

	HealthBench					PRBench					LEXam				
	N=1	Low	Mid	High	XHigh	N=1	Low	Mid	High	XHigh	N=1	Low	Mid	High	XHigh
BoN oracle	**0.335**	**0.376**	**0.413**	**0.445**	**0.473**	**0.142**	0.161	**0.180**	**0.197**	**0.213**	**0.281**	**0.313**	**0.342**	**0.368**	**0.391**
Fusion	—	—	**0.403**	**0.438**	—	—	—	**0.175**	**0.199**	—	—	—	**0.335**	**0.361**	—

	WildBench					WritingBench					Overall				
	N=1	Low	Mid	High	XHigh	N=1	Low	Mid	High	XHigh	N=1	Low	Mid	High	XHigh
BoN oracle	**0.556**	**0.610**	**0.655**	**0.691**	**0.720**	**0.466**	**0.503**	**0.530**	**0.553**	**0.571**	**0.356**	**0.393**	**0.424**	**0.451**	**0.474**
Fusion	—	—	**0.641**	—	—	—	—	**0.529**	**0.552**	—	—	—	**0.416**	**0.387**	—

I.2 Oracle results Olmo3-32B

Table 25 Oracle quality for **OLMo3-32B** across five benchmarks and four compute levels. Cell colour encodes delta from BoN oracle N=1 baseline: steel blue = large gain, pale blue = small gain over BoN N=1 baseline. Bold = gain ≥ 0.03 over baseline.

	HealthBench					PRBench					LEXam				
	N=1	Low	Mid	High	XHigh	N=1	Low	Mid	High	XHigh	N=1	Low	Mid	High	XHigh
BoN oracle	**0.480**	**0.521**	**0.554**	**0.580**	**0.601**	**0.211**	0.236	**0.259**	**0.281**	**0.300**	**0.395**	**0.429**	**0.459**	**0.483**	**0.501**
Fusion	—	—	**0.540**	**0.571**	—	—	—	**0.248**	**0.256**	—	—	—	**0.456**	**0.478**	—

	WildBench					WritingBench					Overall				
	N=1	Low	Mid	High	XHigh	N=1	Low	Mid	High	XHigh	N=1	Low	Mid	High	XHigh
BoN oracle	**0.721**	**0.769**	**0.804**	**0.828**	**0.846**	**0.569**	**0.608**	**0.637**	**0.660**	**0.678**	**0.475**	**0.513**	**0.543**	**0.566**	**0.585**
Fusion	—	—	**0.797**	—	—	—	—	**0.631**	**0.660**	—	—	—	**0.535**	0.491	—

J Headroom capture tables

J.1 Qwen3.5-35B-A3B results

Table 26 Headroom capture for **Qwen3.5-35B**: $h = (\text{realized} - \mu)/(\text{oracle}_{\text{ideal}} - \mu)$, where μ is the BoN N=1 baseline (single-inference mean). $h = 1$ means full exploitation of the oracle gap; $h = 0$ matches μ; $h < 0$ underperforms μ. green = positive (gain captured), red = negative (below baseline). Bold when $|h| \geq 0.20$.

	HealthBench				PRBench				LEXam			
	XLow	Low	Mid	XHigh	XLow	Low	Mid	XHigh	XLow	Low	Mid	XHigh
BoN+Skywork	+0.047	+0.038	+0.030	+0.025	+0.038	+0.023	+0.014	-0.005	+0.047	+0.062	+0.065	+0.051
Fusion	—	**+0.670**	**+0.580**	**+0.495**	—	**+0.390**	**+0.357**	**+0.339**	—	**+0.355**	**+0.246**	**+0.209**
Particle Filter	—	—	—	-0.168	—	—	—	**-0.293**	—	—	—	+0.006
Self-Refine	**-1.096**	**-0.907**	**-0.553**	**-0.399**	**+0.455**	**+0.467**	**+0.495**	**+0.496**	**-1.148**	**-0.833**	**-0.959**	**-0.858**

	WildBench				WritingBench				Overall			
	XLow	Low	Mid	XHigh	XLow	Low	Mid	XHigh	XLow	Low	Mid	XHigh
BoN+Skywork	**+0.375**	**+0.391**	**+0.400**	**+0.413**	**+0.252**	**+0.261**	**+0.265**	**+0.273**	+0.152	+0.155	+0.155	+0.152
Fusion	—	**+0.547**	**+0.642**	**+0.662**	—	**+0.288**	+0.181	**+0.275**	—	**+0.450**	**+0.401**	**+0.396**
Particle Filter	—	—	—	-0.392	—	—	—	**-1.167**	—	—	—	-0.403
Self-Refine	**+0.353**	**+0.342**	**+0.286**	**+0.303**	**+0.869**	**+0.767**	**+0.792**	**+0.703**	-0.114	-0.033	+0.012	+0.049

J.2 Qwen3.5-9B results

Table 27 Headroom capture for **Qwen3.5-9B**: $h = (\text{realized} - \mu)/(\text{oracle}_{\text{ideal}} - \mu)$, where μ is the BoN N=1 baseline (single-inference mean). $h = 1$ means full exploitation of the oracle gap; $h = 0$ matches μ; $h < 0$ underperforms μ. green = positive (gain captured), red = negative (below baseline). Bold when $|h| \geq 0.20$.

	HealthBench				PRBench				LEXam			
	XLow	Low	Mid	XHigh	XLow	Low	Mid	XHigh	XLow	Low	Mid	XHigh
BoN+Skywork	+0.084	+0.080	+0.084	+0.089	+0.077	+0.071	+0.061	+0.050	+0.073	+0.058	+0.059	+0.056
Fusion	—	**+0.615**	**+0.512**	**+0.473**	—	**+0.301**	**+0.346**	**+0.313**	—	**+0.419**	**+0.445**	**+0.339**
Self-Refine	**-3.615**	**-1.681**	**-1.175**	**-0.918**	**+0.251**	**+0.323**	**+0.404**	**+0.329**	**-1.104**	**-0.970**	**-0.952**	**-1.147**

	WildBench				WritingBench				Overall			
	XLow	Low	Mid	XHigh	XLow	Low	Mid	XHigh	XLow	Low	Mid	XHigh
BoN+Skywork	**+0.365**	**+0.358**	**+0.355**	**+0.345**	+0.200	**+0.204**	**+0.211**	**+0.206**	+0.160	+0.154	+0.154	+0.149
Fusion	—	**+0.492**	**+0.506**	**+0.560**	—	**+0.247**	+0.179	+0.123	—	**+0.415**	**+0.398**	**+0.362**
Self-Refine	+0.058	-0.027	+0.047	+0.026	**+0.833**	**+0.770**	**+0.700**	**+0.630**	**-0.715**	**-0.317**	-0.195	**-0.216**

J.3 Olmo3-32B results

Table 28 Headroom capture for **OLMo3-32B**: $h = (\text{realized} - \mu)/(\text{oracle}_{\text{ideal}} - \mu)$, where μ is the BoN N=1 baseline (single-inference mean). $h = 1$ means full exploitation of the oracle gap; $h = 0$ matches μ; $h < 0$ underperforms μ. green = positive (gain captured), red = negative (below baseline). Bold when $|h| \geq 0.20$.

	HealthBench				PRBench				LEXam			
	XLow	Low	Mid	XHigh	XLow	Low	Mid	XHigh	XLow	Low	Mid	XHigh
BoN+Skywork	+0.099	+0.096	+0.090	+0.086	+0.102	+0.082	+0.070	+0.076	+0.085	+0.065	+0.049	+0.025
Fusion	—	**-0.426**	**-0.326**	—	—	**-0.979**	**-1.016**	—	—	**+0.231**	**+0.228**	—

	WildBench				WritingBench				Overall			
	XLow	Low	Mid	XHigh	XLow	Low	Mid	XHigh	XLow	Low	Mid	XHigh
BoN+Skywork	**+0.237**	**+0.221**	**+0.216**	**+0.225**	**+0.336**	**+0.312**	**+0.292**	**+0.255**	+0.172	+0.155	+0.143	+0.133
Fusion	—	**-0.892**	—	—	—	**-0.833**	-0.204	—	—	**-0.580**	**-0.330**	—

J.4 Olmo3-7B results

Table 29 Headroom capture for **OLMo3-7B**: $h = (\text{realized} - \mu)/(\text{oracle}_{\text{ideal}} - \mu)$, where μ is the BoN N=1 baseline (single-inference mean). $h = 1$ means full exploitation of the oracle gap; $h = 0$ matches μ; $h < 0$ underperforms μ. green = positive (gain captured), red = negative (below baseline). Bold when $|h| \geq 0.20$.

	HealthBench				PRBench				LEXam			
	XLow	Low	Mid	XHigh	XLow	Low	Mid	XHigh	XLow	Low	Mid	XHigh
BoN+Skywork	+0.177	+0.165	+0.160	+0.165	+0.170	+0.137	+0.119	+0.110	**+0.216**	+0.171	+0.152	+0.157
Fusion	—	**+0.207**	**+0.292**	—	—	+0.059	**+0.224**	—	—	**+0.353**	**+0.286**	—

	WildBench				WritingBench				Overall			
	XLow	Low	Mid	XHigh	XLow	Low	Mid	XHigh	XLow	Low	Mid	XHigh
BoN+Skywork	**+0.230**	+0.196	+0.184	+0.182	+0.158	+0.138	+0.126	+0.125	+0.190	+0.161	+0.148	+0.148
Fusion	—	-0.190	—	—	—	+0.083	+0.145	—	—	+0.102	**+0.237**	—

K Verifier Correlation and Headroom Capture by RM

Tables 30 and 31 report $\hat{\rho}_v$ (denoised) and headroom capture $\hat{h}$ (ideal oracle) for every (generator, RM, benchmark) combination ($N = 152$, 4 generators $\times$ 2 RMs $\times$ 19 tasks aggregated to benchmark level). Comparing the two tables cell-by-cell validates $\hat{h} \approx \hat{\rho}_v$ (Equation 4) across all combinations (regression: $\hat{h} = 1.198\,\hat{\rho}_v - 0.011$, $R^2 = 0.66$, $\rho = 0.81$, $p < 10^{-36}$).

Table 30 Verifier correlation $\hat{\rho}_v$ (Spearman between RM scores and judge scores), averaged over tasks within each benchmark ($N = 152$, 4 generators $\times$ 2 RMs $\times$ 19 tasks). Both RMs produce near-identical profiles (Skywork mean $\hat{\rho}_v = 0.120$, Llama mean $\hat{\rho}_v = 0.107$, computed at task level before benchmark aggregation), confirming the failure is structural rather than model-specific.

Generator	RM	Health.	LEXam	PR	Wild	Writing
Qwen3.5-9B	Skywork V2	0.063	0.045	0.064	0.255	0.197
	Llama-3.1-70B	0.068	0.058	0.050	0.178	0.189
Qwen3.5-35B	Skywork V2	0.036	0.028	0.023	0.260	0.220
	Llama-3.1-70B	0.067	0.046	0.031	0.146	0.173
Olmo-3-7B	Skywork V2	0.127	0.136	0.113	0.180	0.123
	Llama-3.1-70B	0.119	0.154	0.101	0.166	0.095
Olmo-3.1-32B	Skywork V2	0.078	0.053	0.082	0.177	0.251
	Llama-3.1-70B	0.064	0.051	0.073	0.180	0.189

Table 31 Headroom capture $\hat{h} = (\text{BoN@16} - \mu) \,/\, (\text{Oracle@16} - \mu)$ for the same generator–RM–benchmark combinations. **Bold** negative values indicate the RM selects *worse* than random. Comparing cell-by-cell with Table 30 confirms $\hat{h} \approx \hat{\rho}_v$ (regression: $\hat{h} = 1.20\,\hat{\rho}_v - 0.011$, $R^2 = 0.66$, $\rho = 0.81$, $p < 10^{-36}$), validating Equation 4.

Generator	RM	Health.	LEXam	PR	Wild	Writing
Qwen3.5-9B	Skywork V2	0.080	0.056	0.041	0.345	0.208
	Llama-3.1-70B	0.092	0.102	0.015	0.182	0.182
Qwen3.5-35B	Skywork V2	0.010	0.051	0.002	0.413	0.275
	Llama-3.1-70B	0.056	0.090	0.046	0.243	0.253
Olmo-3-7B	Skywork V2	0.139	0.157	0.119	0.182	0.123
	Llama-3.1-70B	0.127	0.157	0.110	0.130	0.126
Olmo-3.1-32B	Skywork V2	0.066	0.025	0.083	0.225	0.243
	Llama-3.1-70B	0.059	0.123	0.090	0.246	0.232

L Deterministic Benchmark Results: MATH-500 and GPQA Diamond

To contextualise our open-ended QA findings, we report results on two standard deterministic benchmarks: **MATH-500** [8] (competition mathematics, exact-match verified) and **GPQA Diamond** [21] (graduate-level science, multiple-choice). These benchmarks admit binary verification, making them the canonical setting where TTS was developed and where RM-based methods are expected to perform well.

We evaluate the same generator (Qwen3.5-35B-A3B) and the same TTS methods at matched compute levels, using the same token-budget normalisation as the main experiments. This allows a direct comparison of method rankings and headroom capture between deterministic and open-ended settings.

Table 32 Realised quality on **deterministic benchmarks** for **Qwen3.5-35B-A3B**. Cell colour encodes delta from the single-sample baseline (BoN@1): green = improvement, red = regression. Bold = $|\Delta| \geq 0.03$. Italic Pass@K row reports the BoN-pool oracle. Particle Filter and Budget Forcing on GPQA were not run.

	MATH-500					GPQA Diamond				
	N=1	Low	Mid	High	XHigh	N=1	Low	Mid	High	XHigh
BoN+Skywork	**0.922**	0.930	0.933	0.935	0.936	**0.823**	0.841	0.851	0.852	0.843
BoN+Llama70B	**0.922**	0.929	0.932	0.934	0.938	**0.823**	0.836	0.842	0.841	0.838
Fusion	—	—	0.936	0.934	0.932	—	—	**0.859**	0.833	0.823
Beam Search	—	—	0.942	0.940	0.934	—	—	0.818	0.803	0.818
Particle Filter	—	—	0.912	**0.892**	**0.890**	—	—	—	—	—
SR	—	0.928	0.930	0.928	0.936	—	0.828	0.823	0.798	0.838
Budget Forcing	—	0.930	0.938	—	—	—	—	—	—	—
BoN Oracle (Pass@K)	*0.922*	*0.935*	*0.942*	*0.948*	*0.954*	*0.823*	*0.870*	*0.909*	*0.938*	*0.960*

Headroom is benchmark-fixed. Qwen3.5-35B-A3B already reaches the single-sample pass@1 of 0.922 on MATH-500 and 0.823 on GPQA Diamond, leaving only 3.2 pp and 13.7 pp of headroom up to oracle Pass@16. RM-based BoN at XHigh captures 50% of available headroom on MATH-500 and 15% on GPQA, versus ∼15% performed on open-ended QA (Section 6.1). Appendix 10 reproduces this scaling with the Skywork-RM paper's much weaker reference generator (single-sample ∼0.42 on both benchmarks), where BoN+Skywork captures ∼43% of headroom on MATH and ∼22% on GPQA — within a few points of our 50%/15%. RM efficacy is therefore approximately benchmark-fixed: stronger generators shrink the absolute headroom rather than improve the rate at which RMs can exploit it.

M Skywork-Reward-V2 Reproduction on Deterministic Benchmarks

We verified that our BoN pipeline reproduces the scaling behaviour reported in the original Skywork-Reward-V2 paper [14] with the ORM model **Skywork-Reward-V2-Llama-3.1-8B**. We run Best-of-N sampling with PPE Correctness scoring on **MATH-500** and **GPQA**, matching the authors' evaluation protocol.

Figure 10 shows realised accuracy (solid lines) and oracle accuracy (dashed) as a function of N. Our results replicate the paper's findings: realised accuracy scales positively with N on both benchmarks, reaching ∼0.66 on MATH-500 and ∼0.55 on GPQA at N=32, consistent with the curves reported for Skywork-Reward-V2-Llama-3.1-8B in Figure 4 of Liu et al. [14].

The large gap between oracle and realised quality visible in both panels, the oracle approaching 1.0 while realised quality plateaus well below 0.7, already hints at the exploitation bottleneck we characterise in the main text. Here, however, the RM is at least positively correlated with correctness ($\hat{\rho}_v > 0$), so realised quality does improve with compute, unlike the near-zero correlations we observe on open-ended QA (Section 6.1).

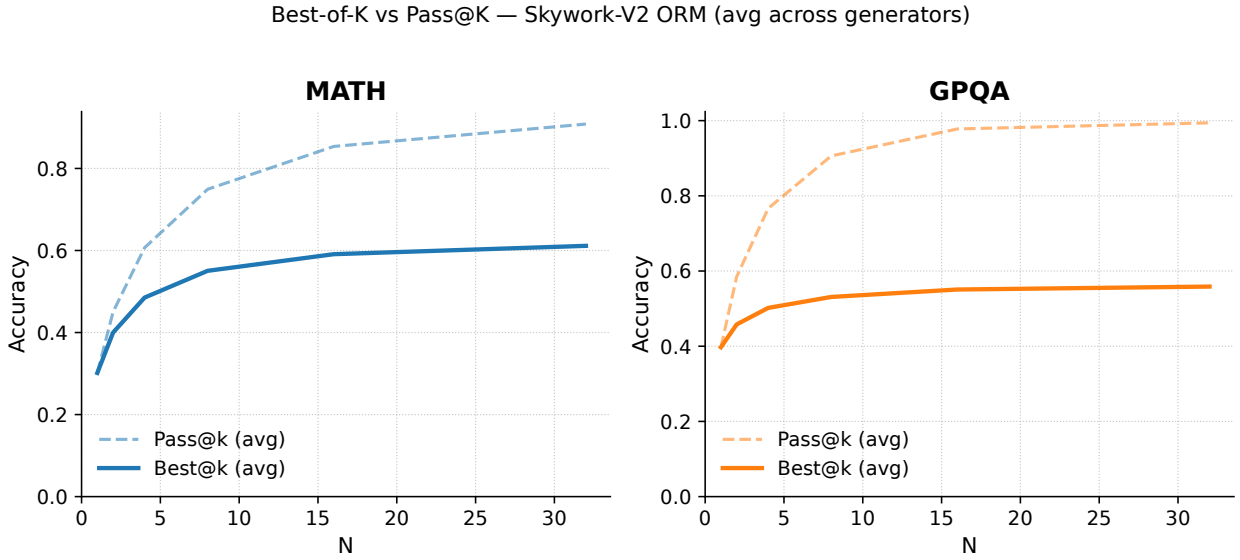

Figure 10 Reproduction of BoN scaling from the Skywork-Reward-V2 paper [14] on MATH-500 (left) and GPQA Diamond (right). Dashed grey: oracle accuracy. Solid coloured: realised accuracy under Skywork-Reward-V2-Llama-3.1-8B selection. Results match the authors' reported curves, validating our BoN pipeline before deployment on open-ended benchmarks.

N WildBench Per-Category Decomposition

Sequential Refinement's aggregate gains on WildBench mask extreme heterogeneity across task categories. We decompose the total realised lift (sum of per-item score changes from the initial to the final draft) by `primary_tag`.

Coding & Debugging accounts for 16.6% of items (170/1,024) but contributes 105.5% of the total realised gain: a mean lift of +27.06 per item, compared with −0.28 per item across the remaining 854 items. Excluding Coding & Debugging, Sequential Refinement produces a net regression on WildBench. The aggregate improvement is therefore not a general property of iterative refinement on open-ended chat tasks, but is driven almost entirely by a single near-deterministic subtask where successive drafts converge toward correct code.

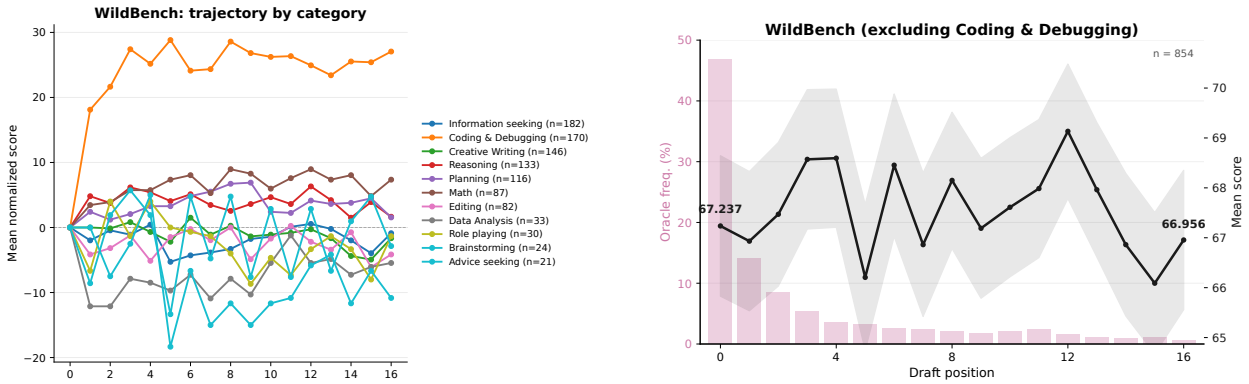

Figure 11 **Left:** Mean score trajectory across Sequential Refinement iterations per WildBench task category. Coding & Debugging rises steeply; most other categories are flat or decline. **Right:** Oracle and mean score trajectories on WildBench with Coding & Debugging excluded. Without this subtask, mean quality regresses across iterations while the oracle continues to rise—confirming the exploitation failure pattern observed on other benchmarks.

O Verbosity Analysis

Response length is not a neutral quantity in open-ended evaluation: if the judge rewards longer responses, methods that produce progressively longer outputs will appear to improve even when content quality is flat or declining. This appendix documents the verbosity bias in two steps. Section O.1 quantifies the length–score relationship under our unified judge (Qwen3.5-397B-A17B). Section O.2 shows that the WritingBench native judge amplifies this bias substantially, confirming it is a property of the benchmark's evaluation design rather than of any particular judge.

O.1 Unified Judge: Length–Score Correlation

The token length distributions in Figure 12 reveal a structural difference between Sequential-Refinement and all other methods: response length grows monotonically across iterations on every benchmark. This matters because if the judge rewards length, Sequential Refinement's apparent gains on WritingBench reflect elaboration rather than quality improvement.

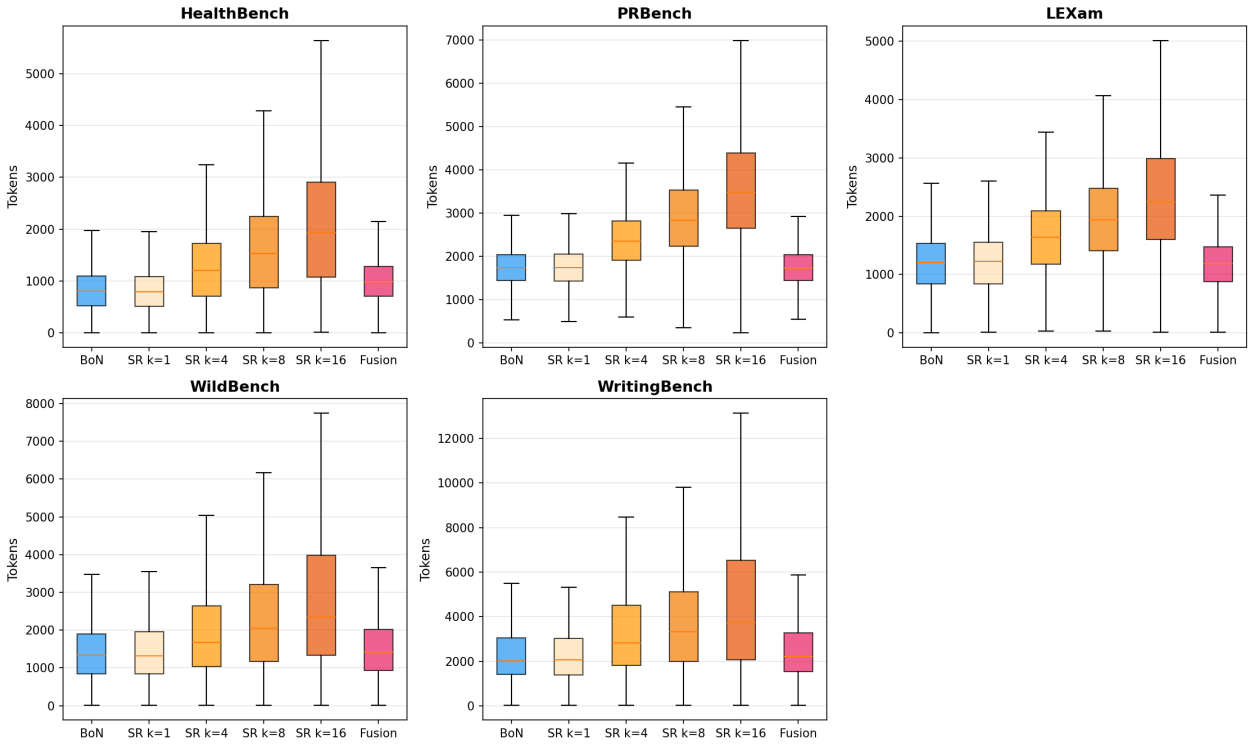

Figure 12 Token length distributions per method and benchmark for Qwen3.5-35B-A3B. **BoN** (blue) and **Fusion** (pink) show stable length distributions across compute levels. **Sequential Refinement** (orange gradient, SR k=1 through k=16) shows monotonically increasing median length on every benchmark, with the steepest growth on WritingBench and PRBench.

Tables 33–37 quantify the relationship between length and judge score within prompts, separately for BoN and Sequential Refinement. The key findings are:

- **WritingBench** shows the strongest within-prompt length–score correlation under both methods ($\hat{\rho} = +0.198$ for BoN, $+0.352$ for Sequential Refinement), and is the only benchmark where $\mathrm{CV}_{\mathrm{len}}$ exceeds $\mathrm{CV}_{\mathrm{score}}$ under Sequential Refinement (ratio $= 0.36$). Length growth across iterations is the primary driver of score gains on this benchmark.
- **HealthBench and PRBench** are content-driven: $\mathrm{CV}_{\mathrm{score}}$ substantially exceeds $\mathrm{CV}_{\mathrm{len}}$ under BoN (ratios of 1.57 and 3.17 respectively). Sequential Refinement's regressions on these benchmarks reflect genuine

quality degradation, not a scoring artifact.

- **LEXam** is the only benchmark where Sequential Refinement shows a *negative* length–score correlation ($\hat{\rho} = -0.060$, $p < 10^{-3}$), consistent with iterative conditioning producing longer but worse responses on knowledge-intensive legal tasks.

Table 33 Within-prompt Spearman rank correlation between response length and judge score for **BoN** candidates (16 per prompt), Qwen3.5-35B-A3B, unified judge. $\hat{\rho}$ = mean per-prompt Spearman correlation; 95% CI is bootstrapped; %ρ>0 = fraction of prompts with a positive correlation; p-values from a one-sample t-test and Wilcoxon signed-rank test against $H_0 : \rho = 0$.

Benchmark	n	$\hat{\rho}$ [95% CI]	%ρ>0	t-test p	Wilcoxon p
HealthBench	4,619	+0.133 [+0.124, +0.142]	67.5%	7.76×10^{-180}	8.20×10^{-167}
PRBench	1,634	+0.112 [+0.099, +0.126]	64.7%	6.07×10^{-57}	1.87×10^{-51}
LEXam	502	+0.014 [−0.011, +0.039]	52.0%	2.73×10^{-1}	3.81×10^{-1}
WildBench	917	+0.044 [+0.024, +0.063]	57.6%	1.93×10^{-5}	4.72×10^{-6}
WritingBench	554	+0.198 [+0.174, +0.222]	74.5%	4.97×10^{-46}	4.10×10^{-39}

Table 34 Within-prompt length variance for **BoN** candidates (16 per prompt), Qwen3.5-35B-A3B. CV = coefficient of variation (std/mean) of token lengths within a prompt; %CV<0.1 and %CV<0.2 report the fraction of prompts with low length dispersion; mean and median std are in tokens. PRBench shows the lowest length variance (mean CV = 0.083), consistent with its structured rubric format constraining response length. WritingBench and WildBench show the highest variance, reflecting open-ended prompts that admit responses of widely varying length.

Benchmark	n	mean CV	med CV	%CV<0.1	%CV<0.2	mean std	med std
HealthBench	5,000	0.150	0.127	29.7%	83.2%	101.3	88.0
PRBench	1,650	0.083	0.076	81.2%	98.6%	141.6	128.8
LEXam	516	0.119	0.096	54.3%	90.9%	113.5	110.7
WildBench	1,023	0.186	0.131	30.2%	73.4%	295.2	160.7
WritingBench	555	0.145	0.107	45.2%	76.2%	392.7	244.0

Table 35 Score variability versus length variability for **BoN** candidates (16 per prompt), Qwen3.5-35B-A3B, unified judge. $\mathrm{CV}_{\mathrm{score}}$ and $\mathrm{CV}_{\mathrm{len}}$ are the per-prompt coefficient of variation of judge scores and token lengths respectively; mean ratio = mean($\mathrm{CV}_{\mathrm{score}}/\mathrm{CV}_{\mathrm{len}}$). A ratio >1 indicates score variation is driven by content; a ratio <1 indicates length is the dominant axis of variation. PRBench (ratio = 3.17) and HealthBench (ratio = 1.57) are content-driven. WritingBench (ratio = 0.55) and WildBench (ratio = 0.69) are length-driven even under BoN, before any iterative elaboration.

Benchmark	n	$\mathrm{CV}_{\mathrm{score}}$	$\mathrm{CV}_{\mathrm{len}}$	mean ratio	%ratio>2	%both<0.05
HealthBench	5,000	0.329	0.150	1.57	37.8%	0.1%
PRBench	1,650	0.356	0.083	3.17	76.5%	0.2%
LEXam	516	0.217	0.119	1.65	39.1%	0.0%
WildBench	1,023	0.137	0.186	0.69	8.0%	0.4%
WritingBench	555	0.084	0.145	0.55	5.2%	1.4%

Table 36 Within-prompt Spearman rank correlation between response length and judge score for **Sequential Refinement** drafts ($k = 1, \ldots, 16$ per prompt), Qwen3.5-35B-A3B, unified judge. Columns as in Table 33. Compared with BoN, Sequential Refinement shows a substantially stronger length–score correlation on WritingBench (+0.352 vs. +0.198) and PRBench (+0.252 vs. +0.112), confirming that iterative elaboration amplifies the length–score relationship. LEXam is the only benchmark with a significantly *negative* correlation ($\hat{\rho} = -0.060$), consistent with refinement producing longer but worse responses on knowledge-intensive tasks.

Benchmark	n	$\hat{\rho}$ [95% CI]	%ρ>0	t-test p	Wilcoxon p
HealthBench	4,703	+0.059 [+0.047, +0.069]	56.2%	1.26×10^{-23}	2.94×10^{-23}
PRBench	1,634	+0.252 [+0.232, +0.272]	72.2%	6.68×10^{-116}	1.75×10^{-99}
LEXam	505	−0.060 [−0.092, −0.026]	41.2%	2.46×10^{-4}	2.29×10^{-4}
WildBench	925	+0.023 [−0.001, +0.046]	52.0%	5.23×10^{-2}	3.82×10^{-2}
WritingBench	555	+0.352 [+0.319, +0.385]	80.7%	1.12×10^{-73}	1.21×10^{-55}

Table 37 Score variability versus length variability for **Sequential Refinement** drafts (16 iterations per prompt), Qwen3.5-35B-A3B, unified judge. Columns as in Table 35. Compared with BoN, Sequential Refinement shows higher $\mathrm{CV}_{\mathrm{len}}$ on every benchmark, reflecting monotonic length growth across iterations. On WritingBench, $\mathrm{CV}_{\mathrm{len}}$ substantially exceeds $\mathrm{CV}_{\mathrm{score}}$ (ratio = 0.36), confirming that length growth is the primary driver of score gains. On HealthBench, the ratio drops from 1.57 (BoN) to 0.91 (Sequential Refinement): iterative conditioning increases length variance more than score variance, a sign of elaboration without quality improvement.

Benchmark	n	$\mathrm{CV}_{\mathrm{score}}$	$\mathrm{CV}_{\mathrm{len}}$	mean ratio	%ratio>2	%both<0.05
HealthBench	5,000	0.358	0.257	0.91	19.3%	0.0%
PRBench	1,650	0.297	0.256	0.84	15.6%	0.0%
LEXam	516	0.237	0.218	0.86	13.2%	0.0%
WildBench	1,024	0.126	0.256	0.46	5.8%	0.2%
WritingBench	555	0.083	0.198	0.36	2.3%	0.5%

O.2 Native WritingBench Judge: Bias Amplification

The unified judge already reveals a strong verbosity bias on WritingBench. To confirm this is a property of the benchmark's evaluation design and not of our judge choice, we repeat the analysis using the WritingBench native judge, **WritingBench-Critic-Model-Qwen-7B** [30], a Qwen2.5-7B-Instruct model fine-tuned on 50K writing evaluation examples. Wu et al. report 83% human agreement for this critic model on a 300-query, 5-annotator pairwise evaluation [30].

The native judge amplifies the verbosity signal substantially on WritingBench while leaving all other benchmarks unchanged: the BoN Spearman correlation rises from +0.198 to +0.370; the Sequential Refinement correlation rises from +0.352 to +0.539; $\mathrm{CV}_{\mathrm{score}}$ on WritingBench collapses from 0.084 to 0.034 (BoN) and from 0.083 to 0.038 (Sequential Refinement), meaning the native judge discriminates on length. These results suggest strongly that the verbosity bias is intrinsic to WritingBench's evaluation design, not an artefact of our unified judge.

Table 38 Within-prompt Spearman rank correlation between response length and judge score for **BoN** candidates (16 per prompt), Qwen3.5-35B-A3B, **native WritingBench judge**. WritingBench $\hat{\rho}$ rises from +0.198 (unified judge, Table 33) to +0.370; all other benchmarks are stable, confirming the amplification is WritingBench-specific.

Benchmark	n	$\hat{\rho}$ [95% CI]	%ρ>0	t-test p	Wilcoxon p
HealthBench	4,619	+0.133 [+0.124, +0.142]	67.5%	7.76×10^{-180}	8.20×10^{-167}
PRBench	1,634	+0.112 [+0.099, +0.125]	64.7%	6.07×10^{-57}	1.87×10^{-51}
LEXam	502	+0.014 [−0.010, +0.038]	52.0%	2.73×10^{-1}	3.81×10^{-1}
WildBench	917	+0.044 [+0.023, +0.064]	57.6%	1.93×10^{-5}	4.72×10^{-6}
WritingBench	555	+0.370 [+0.347, +0.394]	87.9%	6.24×10^{-121}	7.28×10^{-77}

Table 39 Score variability versus length variability for **BoN** candidates (16 per prompt), Qwen3.5-35B-A3B, **native WritingBench judge**. Under the native judge, WritingBench CV_{score} collapses to 0.034 (vs. 0.084 under the unified judge, Table 35), yielding a ratio of 0.25: score variation is almost entirely explained by length. Length statistics are identical to Table 34 since they are judge-independent.

Benchmark	n	CV_{score}	CV_{len}	mean ratio	%ratio>2	%both<0.05
HealthBench	5,000	0.329	0.150	1.57	37.8%	0.1%
PRBench	1,650	0.356	0.083	3.17	76.5%	0.2%
LEXam	516	0.217	0.119	1.65	39.1%	0.0%
WildBench	1,023	0.137	0.186	0.69	8.0%	0.4%
WritingBench	555	0.034	0.145	0.25	0.0%	2.0%

Table 40 Within-prompt Spearman rank correlation between response length and judge score for **Sequential Refinement** drafts ($k = 1, \ldots, 16$ per prompt), Qwen3.5-35B-A3B, **native WritingBench judge**. WritingBench $\hat{\rho}$ rises to +0.539 (vs. +0.352 under the unified judge, Table 36), with 92.2% of prompts showing a positive correlation. All other benchmarks are stable.

Benchmark	n	$\hat{\rho}$ [95% CI]	%ρ>0	t-test p	Wilcoxon p
HealthBench	4,703	+0.059 [+0.047, +0.070]	56.2%	1.26×10^{-23}	2.94×10^{-23}
PRBench	1,634	+0.252 [+0.232, +0.272]	72.2%	6.68×10^{-116}	1.75×10^{-99}
LEXam	505	−0.060 [−0.091, −0.029]	41.2%	2.46×10^{-4}	2.29×10^{-4}
WildBench	925	+0.023 [+0.000, +0.045]	52.0%	5.23×10^{-2}	3.82×10^{-2}
WritingBench	551	+0.539 [+0.513, +0.566]	92.2%	1.07×10^{-159}	1.82×10^{-82}

Table 41 Score variability versus length variability for **Sequential Refinement** drafts (16 iterations per prompt), Qwen3.5-35B-A3B, **native WritingBench judge**. WritingBench $CV_{score} = 0.038$ (vs. 0.083 under the unified judge, Table 37), with a ratio of 0.17: under the native judge, virtually all score variation in Sequential Refinement on WritingBench is attributable to response length.

Benchmark	n	CV_{score}	CV_{len}	mean ratio	%ratio>2	%both<0.05
HealthBench	5,000	0.358	0.257	0.91	19.3%	0.0%
PRBench	1,650	0.297	0.256	0.84	15.6%	0.0%
LEXam	516	0.237	0.218	0.86	13.2%	0.0%
WildBench	1,024	0.126	0.256	0.46	5.8%	0.2%
WritingBench	555	0.038	0.198	0.17	0.5%	0.4%

O.3 WritingBench: Judge Robustness Analysis

To assess whether our WritingBench judge agreement (QWK 0.408, Section 4) materially affects our conclusions, we re-score the same TTS outputs with the official fine-tuned Critic model released by the WritingBench authors [30] and recompute realized and oracle scores under both judges (Figure 13). For BoN with either RM (Skywork, Llama70B), headroom capture is judge-invariant (0.19 vs. 0.19), indicating that conclusions about parallel selection methods transfer cleanly across judges. For Sequential Refinement the capture ratio rises from 0.58 to 0.75 under Critic model, but the decomposition shows this is driven by a shrinking denominator (oracle$-\mu$ drops from 0.125 to 0.081) rather than an improved numerator (realized$-\mu$ actually *decreases* slightly, from 0.073 to 0.061). This denominator compression is the mechanical signature of Critic model's stronger length bias (Appendix O) interacting with SR's length-growing trajectory: length-favored late iterations (both realized and oracle) are pulled closer together, inflating the ratio without genuine selection improvement. The result confirms that our methods-level conclusions are stable across judges where it matters, and where they diverge the divergence is explained by a known structural bias of the benchmark rather than by miscalibration of our judge.

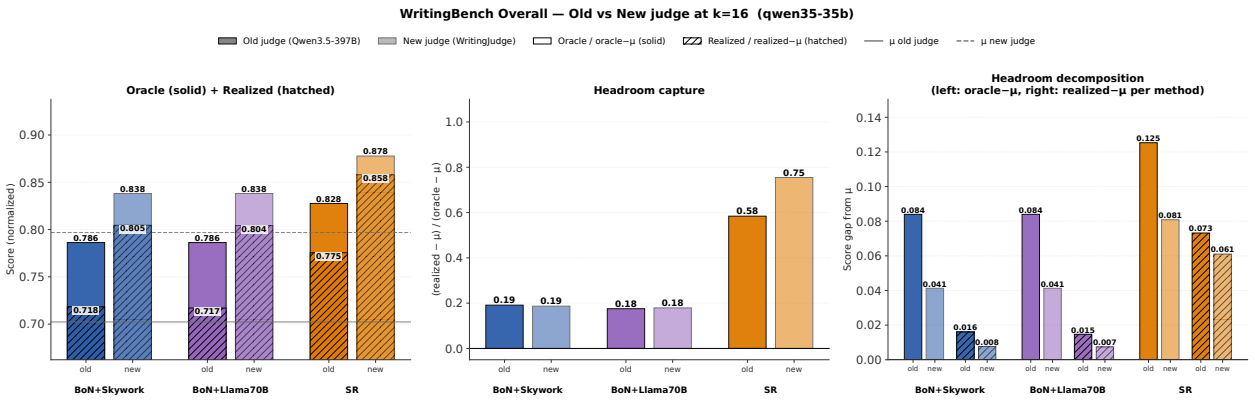

Figure 13 WritingBench old vs. new judge at k=16 (Qwen3.5-35B, micro-averaged across 555 samples). *Left:* Oracle (solid) and realized (hatched) scores for BoN+Skywork, BoN+Llama70B, and Self-Refine under our judge (Qwen3.5-397B, dark) and the authors' fine-tuned WritingBench-Critic-Model-Qwen-7B (light); horizontal lines show the candidate-mean μ for each judge. *Center:* Headroom capture (realized $-\ \mu$)/(oracle $-\ \mu$). *Right:* Decomposition into denominator (oracle$-\mu$, solid) and numerator (realized$-\mu$, hatched).

P Effect of Model Capacity on Self-Verifier Algorithms

We study self-verifier methods along two axes: compute and model capacity. The model-capacity axis spans the Qwen3.5 dense instruct family at three sizes — 4B, 9B, and 27B —, along with Olmo3-7b and Olmo3.1-32b where only fusion was run, holding all other settings fixed. We first report the mean absolute score gain on each benchmark in Figure 14. Because moving from 0.80 → 0.85 is arguably harder than 0.60 → 0.65, we also report the relative error reduction — the fraction of the gap between a BoN@1 baseline and a perfect score that is closed — in Figure 15. Both metrics point to the same conclusions:

1. **Fusion ability is model-family-dependent**: Qwen3.5 shows fusion consistently helping across benchmarks, with gains that are roughly stable across model scales. Increasing compute further improves performance monotonically. However, Olmo3.1-32b fails badly with Olmo3-7b still struggling to hit the BoN@1 baseline.
2. **Sequential Refinement performance is benchmark-dependent:** it yields clear gains on 2/5 benchmarks but strictly hurts performance on 2/5. There are no consistent model capacity scaling trends.

To isolate the *exploration* quality of sequential refinement from its exploitation cost, we compare its oracle score (the maximum score within the pool) against the BoN oracle. When comparing the oracles with matched compute in Figure 16, BoN oracle outperforms on 3/5 benchmarks, has roughly equivalent performance on WildBench, and is outperformed on WritingBench although this is known to have a length bias that benefits Refinement (Appendix O.1).

Next, we ask whether the oracle underperformance is due to (1) fewer candidates from the additional exploitation cost, or (2) lower variance in the candidate pool relative to i.i.d. sampling. To test this, we match k self-revisions against k i.i.d. samples without compute normalization, asking whether iterative refinement produces a higher-quality pool (in terms of max score) even if exploitation were free. Figure 17 shows that any oracle advantage appears tied to model size, but in many cases sequential refinement drafts produce a *worse* oracle pool than i.i.d. sampling at matched k: two benchmarks show strictly negative effects, two show small effects, and WritingBench is the only benchmark with a large positive effect. As before, we attribute the WritingBench result to a length bias in the judge combined with sequential refinement's tendency to increase verbosity; see Appendix O.1 for the supporting analysis.

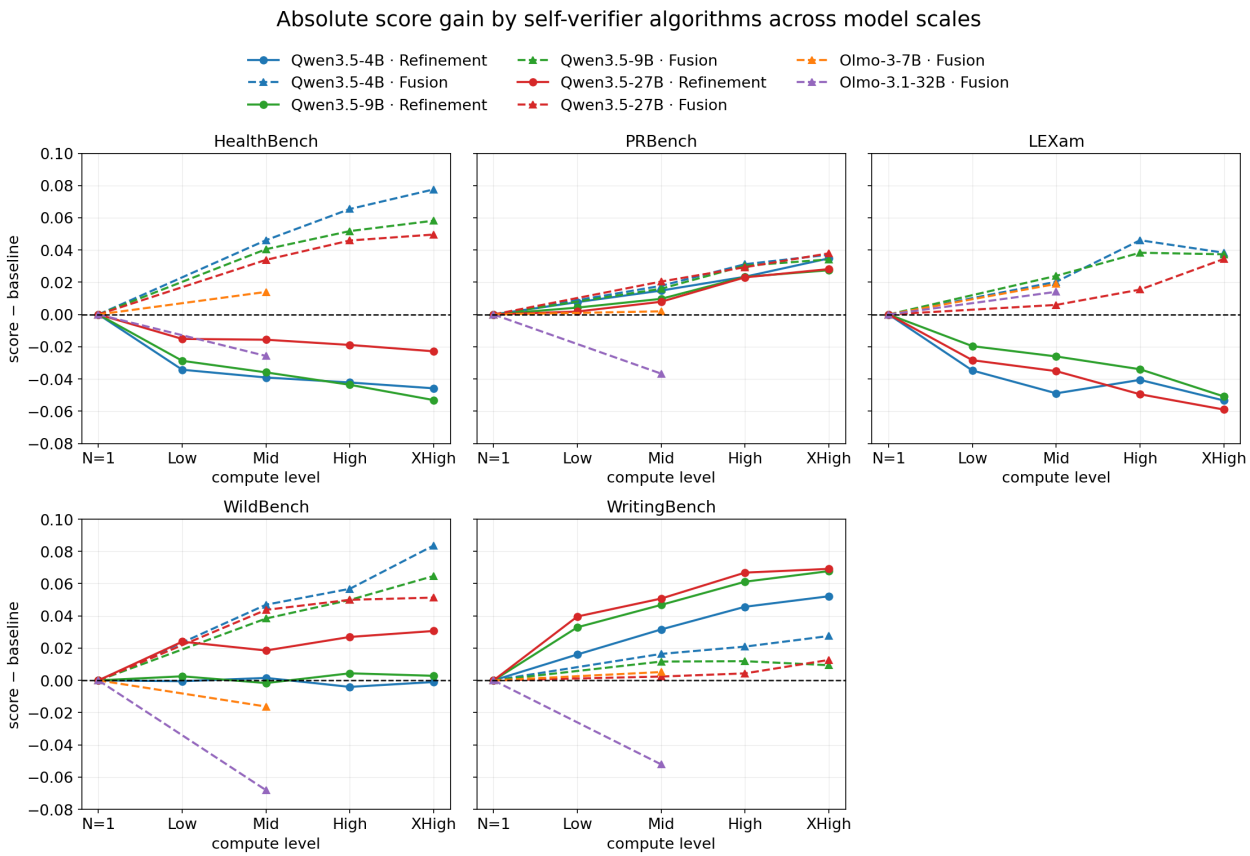

Figure 14 Absolute score gain over the BoN@1 baseline across compute levels, by model family and benchmark.

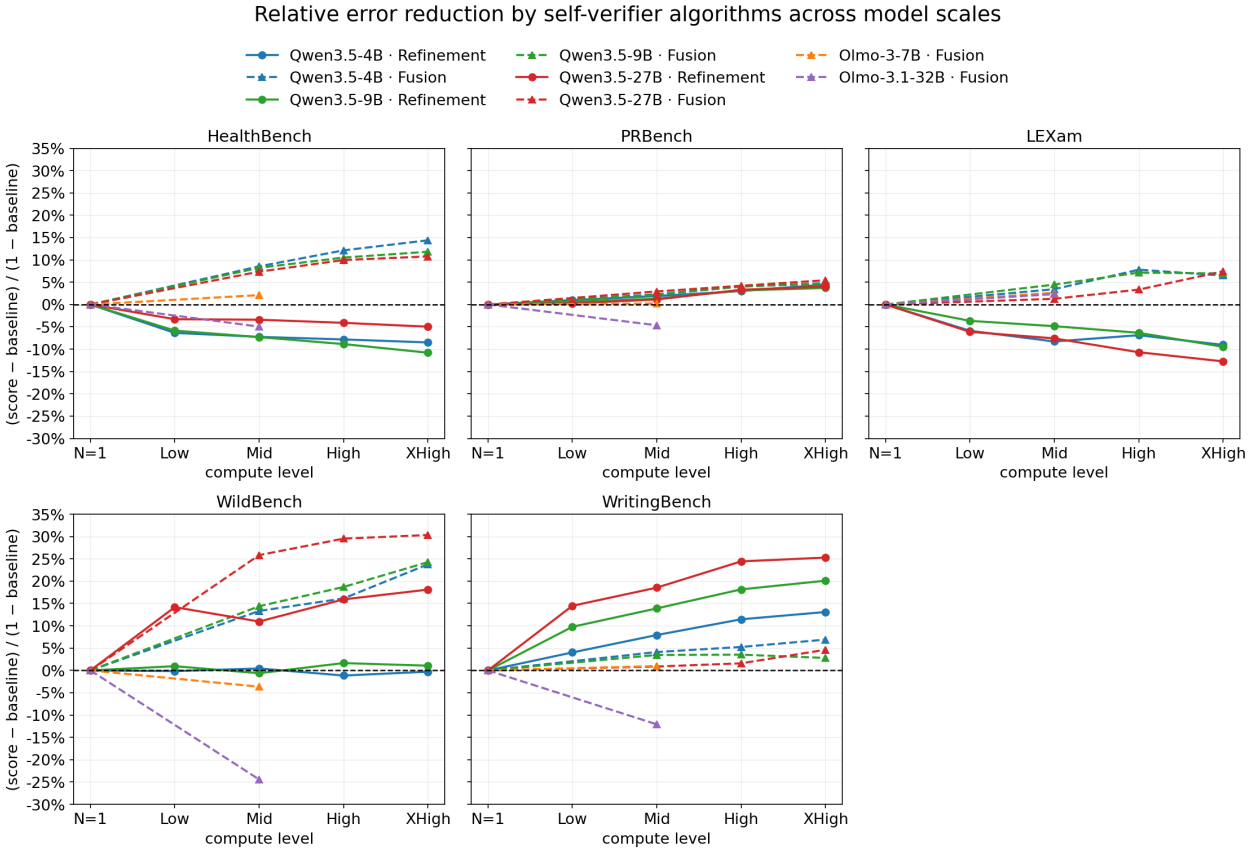

Figure 15 Fraction of remaining gap to a perfect score, $(p_k - p_1)/(1 - p_1)$, closed at compute level k, faceted by benchmark. p_1 is the BoN@1 baseline.

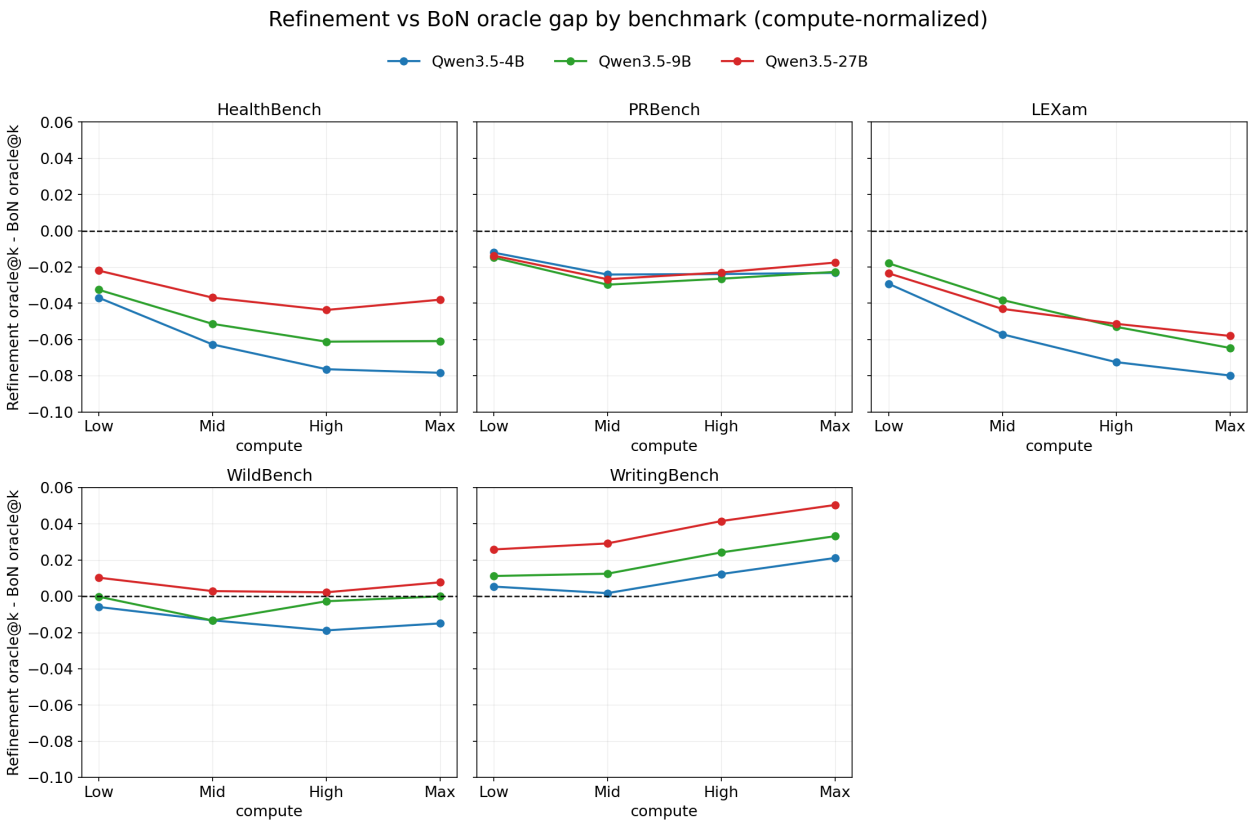

Figure 16 Sequential Refinement oracle vs BoN oracle across compute levels, by model family and benchmark.

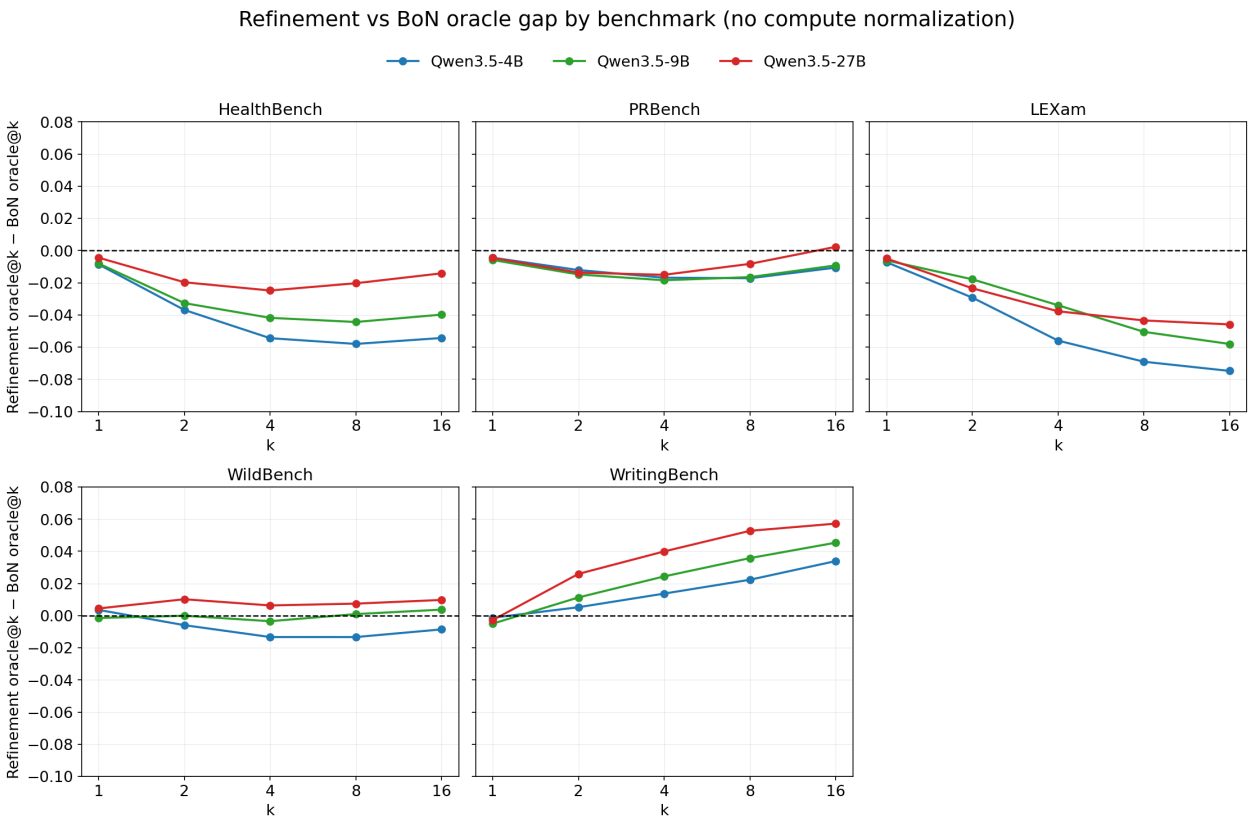

Figure 17 Sequential Refinement oracle vs BoN oracle across compute levels, by model family and benchmark. Note that x-axis refers to number of candidates in the pool so compare k self-revisions to k i.i.d. samples here.

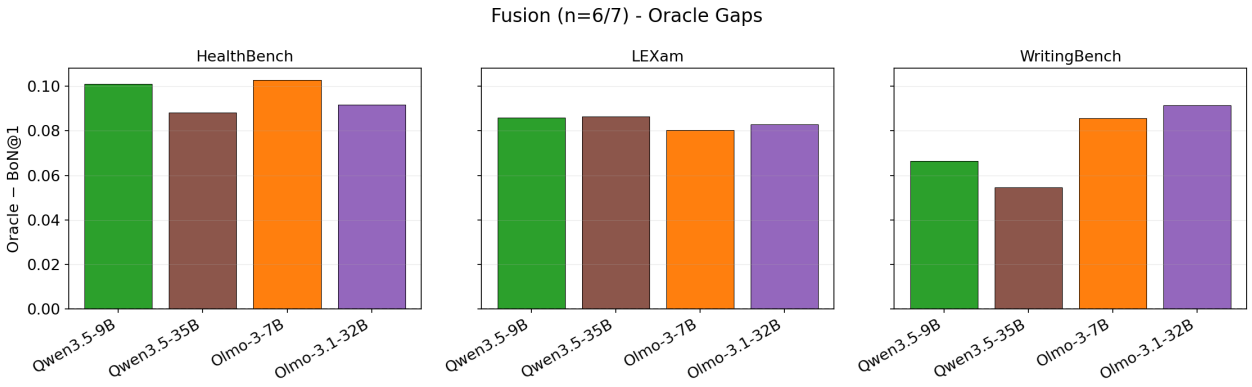

Figure 18 Oracle gaps over a BoN@1 baseline across models for fusion at medium compute (k=8). Note that Olmo models used 6 candidates instead of 7 due to context length limits. Additionally, context-length issues persisted for Olmo on PrBench and WildBench so they are not included here.

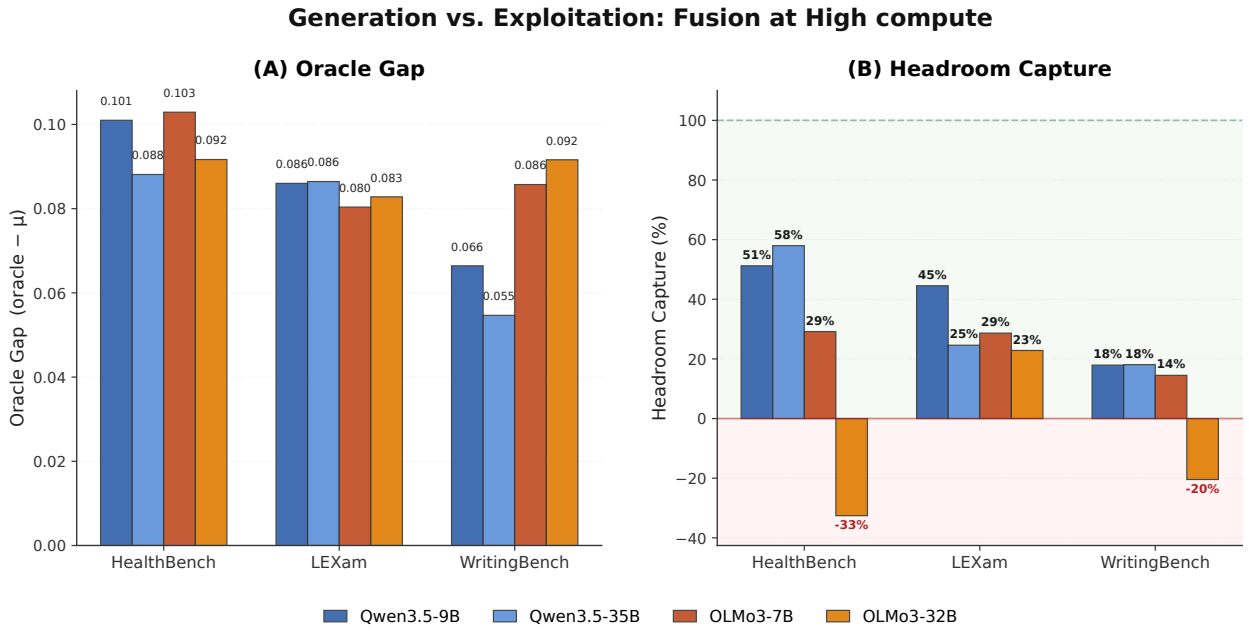

Figure 19 Oracle gap and headroom capture comparison between different families

Q Task Demand Profiles

To characterise what each benchmark truly measures, we applied the DeLeAn rubric from the ADeLe framework [33] to annotate instances across our five benchmarks along 18 demand dimensions spanning primordial capabilities, domain knowledge, and extraneous factors. The resulting demand profiles are visualised as radar plots, where the inner ring reflects low-scoring instances (score 0) and the outer ring reflects high-scoring instances (score 5), with colour intensity encoding instance frequency at each demand level.

This characterisation serves a dual purpose. Beyond describing what each benchmark measures, it allows us to shed light on *which primordial capabilities* each TTS method is principled to engage, grounding our empirical findings in the cognitive skill demands of each benchmark. We note that this mapping is interpretive rather than predictive: the demand profile identifies the skills a benchmark stresses, but whether a given TTS method benefits those skills depends on the exploitation bottleneck identified in Section 6. Crucially, the profiles do not yield a clean mapping from demand dimensions to method success—the same dimension can be associated with regression on one benchmark and gains on another, depending on how the benchmark operationalises that demand and how the judge evaluates it.

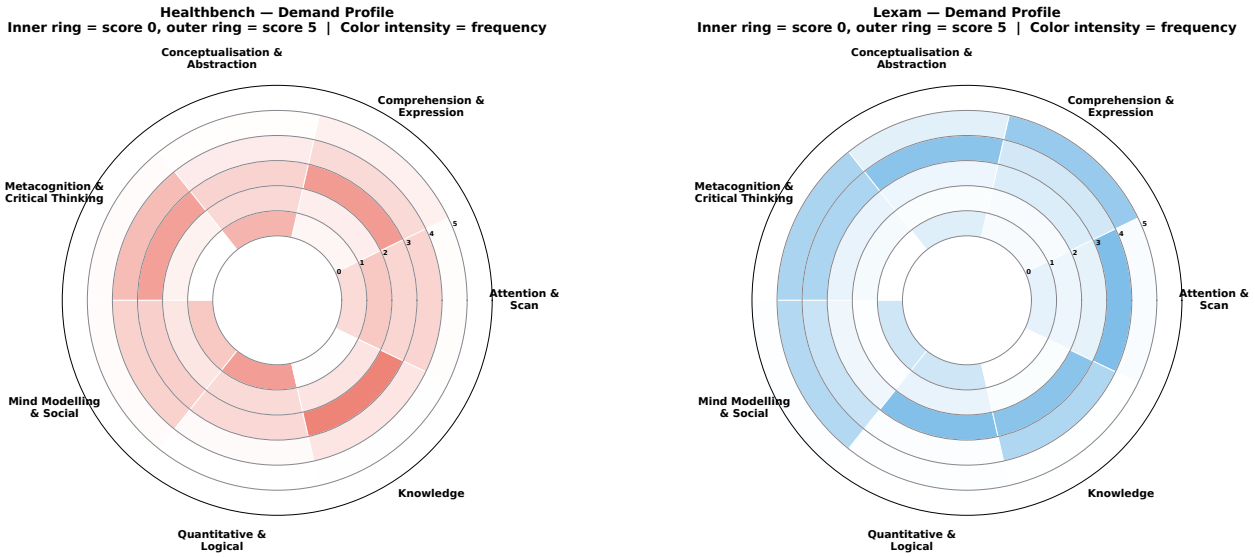

Figure 20 Demand profiles for HealthBench (left) and LEXam (right)

HealthBench is dominated by *Metacognition & Critical Thinking* (MC), with a strong secondary load on *Comprehension & Expression* (CE) (Figure 20, left). *Mind Modelling & Social* (MS) contributes moderately, while *Conceptualisation & Abstraction* (CL), *Knowledge* (KN), *Attention & Scan* (AS), and *Quantitative & Logical* (QL) are comparatively absent. Notably, the low KN load is counterintuitive for a medical benchmark, but reflects that HealthBench instances stress the *process* of reasoning over medical queries—self-monitoring and structured response generation against physician rubrics—rather than raw factual recall. The dominance of MC is consistent with Sequential Refinement's strong regression here (−2.3pp at XHigh): the first draft already engages the primary metacognitive demand, and iterative critique-and-rewrite saturates rather than improves it.

LEXam is dominated by *Comprehension & Expression* (CE), with *Attention & Scan* (AS) as the second most prominent dimension—markedly higher than in HealthBench (Figure 20, right). *Metacognition & Critical Thinking* (MC) and *Knowledge* (KN) contribute moderately, while *Conceptualisation & Abstraction* (CL), *Mind Modelling & Social* (MS), and *Quantitative & Logical* (QL) are largely absent. The prominence of AS reflects the need to locate and integrate specific information across legal texts, while CE dominance is consistent with the interpretive demands of legal reasoning. LEXam exhibits the strongest Sequential Refinement regression of all benchmarks (−4.5pp at XHigh), despite having a different demand profile from HealthBench. This suggests the failure mode is not uniquely tied to MC dominance: tasks requiring precise comprehension and targeted information extraction (CE+AS) are equally ill-suited to iterative refinement, since conditioning on prior drafts compounds misinterpretations rather than correcting them.

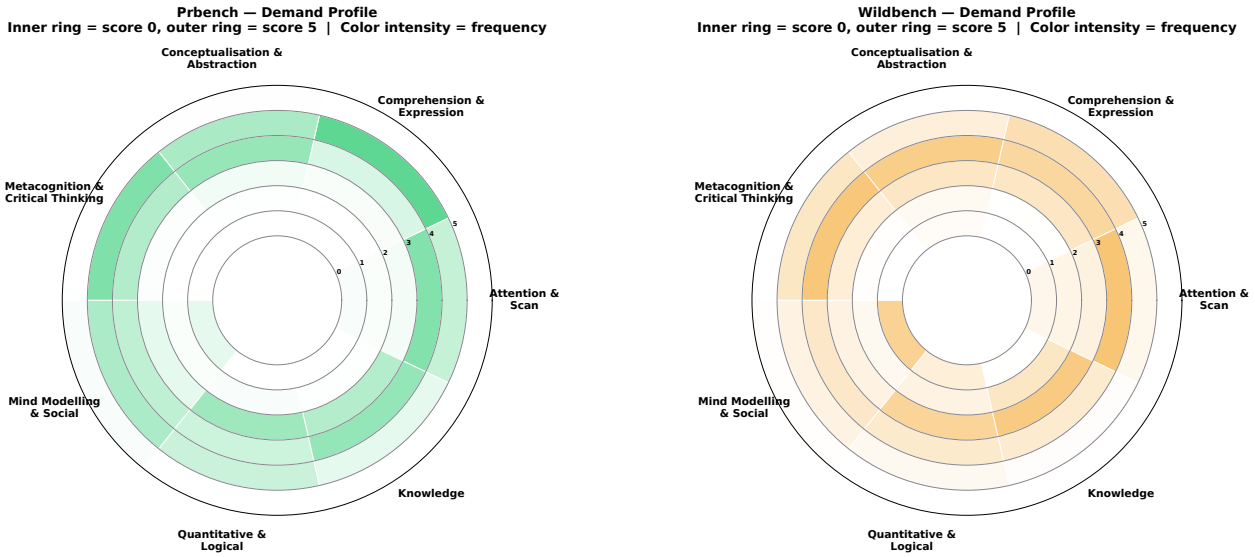

Figure 21 Demand profiles for PRBench (left) and WildBench (right)

PRBench is co-dominated by *Comprehension & Expression* (CE) and *Metacognition & Critical Thinking* (MC), with a moderate contribution from *Conceptualisation & Abstraction* (CL) and *Mind Modelling & Social* (MS) (Figure 21, left). *Attention & Scan* (AS), *Knowledge* (KN), and *Quantitative & Logical* (QL) are comparatively absent. Sequential Refinement achieves its largest non-WritingBench gains on PRBench (+3.5pp at XHigh), which is noteworthy given that PRBench shares its two dominant dimensions (CE, MC) with HealthBench, where Sequential Refinement strongly regresses. This apparent contradiction suggests that the demand profile alone does not determine method success: the key difference is likely how these dimensions are operationalised and evaluated. PRBench's rubric-based scoring rewards elaboration and structured reasoning across multiple criteria, whereas HealthBench's physician rubrics penalise deviation from precise clinical content. The CE+MC demand can therefore support either regression or gain depending on whether the judge rewards precision or elaboration.

WildBench is dominated by *Metacognition & Critical Thinking* (MC), with *Comprehension & Expression* (CE) as a strong second dimension (Figure 21, right). *Attention & Scan* (AS) is notably the third most prominent dimension, more so than in PRBench. *Conceptualisation & Abstraction* (CL) contributes moderately, while *Mind Modelling & Social* (MS), *Knowledge* (KN), and *Quantitative & Logical* (QL) are comparatively absent. WildBench's MC dominance makes it superficially similar to HealthBench, yet Sequential Refinement gains modestly here (+2.4pp at XHigh) rather than regressing. This again points to evaluation operationalisation as the mediating factor: WildBench uses an LLM-generated holistic checklist judge that is more tolerant of elaborated responses, whereas HealthBench's physician rubrics are precision-anchored. The MC demand is present in both, but its interaction with the evaluation mechanism produces opposite outcomes.

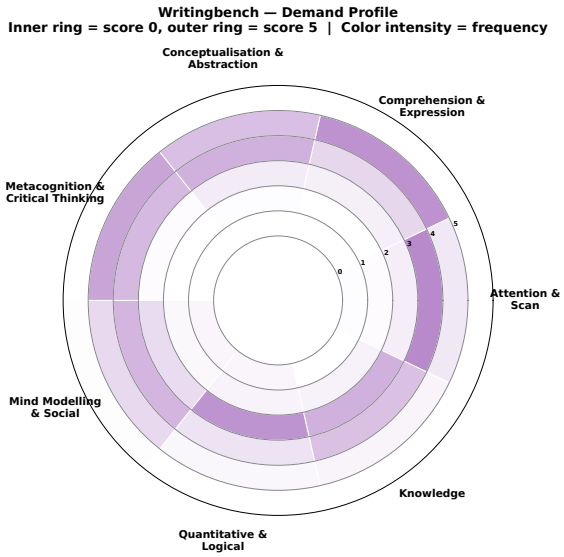

Figure 22 WritingBench demand profile

WritingBench is distinctively loaded on *Comprehension & Expression* (CE) and *Conceptualisation & Abstraction* (CL), which together dominate the profile with the highest frequency at outer demand levels (Figure 22). *Metacognition & Critical Thinking* (MC) contributes moderately, while *Mind Modelling & Social* (MS) and *Attention & Scan* (AS) are present but secondary. *Knowledge* (KN) and *Quantitative & Logical* (QL) are comparatively absent. The CE+CL dominance, with near-absent knowledge and precision-retrieval demands, makes WritingBench the most distinctive profile among the five benchmarks. Tasks primarily requiring expression and conceptualisation have no fixed correct answer, which could plausibly reward iterative elaboration. However, as we show in Section 6.3 and Appendix O, Sequential Refinement's large gains here (+7.3pp) are substantially confounded by the WritingBench judge's verbosity bias. The demand profile alone is insufficient to explain the magnitude: CE+CL makes WritingBench a plausible candidate for refinement benefit, but the verbosity confound means we cannot attribute the observed gains to genuine skill improvement without verbosity-controlled evaluation.

Cross-benchmark demand structure and method fit. The five profiles reveal that demand dimensions alone do not predict method success. The same dimensions—MC and CE—appear as dominant in HealthBench (Sequential Refinement regresses −2.3pp), PRBench (Sequential Refinement gains +3.5pp), and WildBench (Sequential Refinement gains +2.4pp). The differentiating factor is not the demand profile but how each benchmark operationalises and evaluates those demands. Precision-sensitive judges — whether rubric-based (HealthBench) or reference-guided (LEXam) — penalise the elaboration that iterative refinement produces: HealthBench's physician rubrics reward only predefined clinical criteria, while LEXam's reference-guided LLM judge penalises the compounded misinterpretations that conditioning on prior drafts introduces. Checklist and holistic judges (PRBench, WildBench, WritingBench) are, by contrast, more tolerant of or actively reward elaborated responses. This implies that the demand profile is useful for characterising *what* a benchmark tests, but predicting TTS method success requires additionally knowing *how*the benchmark evaluates outputs. Fusion, by contrast, improves uniformly across all five profiles, confirming that its gains are structural — bypassing verifier quality entirely — and not contingent on any particular demand configuration or evaluation style.

These profiles make explicit what the per-method analyses implied: demand dimensions identify *what* a benchmark stresses, but they do not determine method success on their own. The same MC+CE profile supports opposite Sequential Refinement outcomes depending on whether the judge rewards precision or elaboration. This evaluation-operationalisation interaction is orthogonal to model capability.

R Token usage

Table 42 BoN n=16 total completion tokens and ±27% compute band (min–max) per benchmark.

Benchmark	Qwen3.5-9B		Qwen3.5-35B-A3B		OLMo-3-7B		OLMo-3.1-32B	
	Tokens	Band [min–max]	Tokens	Band [min–max]	Tokens	Band [min–max]	Tokens	Band [min–max]
healthbench_communication	39.6M	[28.9M–50.3M]	35.7M	[26.1M–45.4M]	32.9M	[24.0M–41.8M]	34.0M	[24.8M–43.2M]
healthbench_complex_responses	17.3M	[12.6M–22.0M]	16.2M	[11.8M–20.6M]	14.1M	[10.3M–17.9M]	16.0M	[11.7M–20.3M]
healthbench_context_seeking	19.3M	[14.1M–24.5M]	17.8M	[13.0M–22.6M]	15.6M	[11.4M–19.8M]	15.6M	[11.4M–19.8M]
healthbench_emergency_referrals	15.9M	[11.6M–20.1M]	14.5M	[10.6M–18.4M]	12.6M	[9.2M–16.0M]	12.1M	[8.8M–15.3M]
healthbench_global_health	41.3M	[30.1M–52.4M]	37.5M	[27.4M–47.7M]	34.5M	[25.2M–43.9M]	34.7M	[25.3M–44.0M]
healthbench_health_data_tasks	20.8M	[15.2M–26.4M]	19.7M	[14.4M–25.1M]	17.3M	[12.7M–22.0M]	20.1M	[14.7M–25.5M]
healthbench_hedging	39.2M	[28.6M–49.8M]	35.7M	[26.1M–45.3M]	29.9M	[21.8M–38.0M]	31.5M	[23.0M–40.0M]
lexam_open	31.1M	[22.7M–39.5M]	27.8M	[20.3M–35.4M]	24.0M	[17.5M–30.4M]	25.8M	[18.8M–32.8M]
prbench_finance	48.2M	[35.2M–61.2M]	44.6M	[32.5M–56.6M]	45.6M	[33.3M–58.0M]	44.3M	[32.3M–56.2M]
prbench_finance_hard	24.5M	[17.9M–31.1M]	22.8M	[16.6M–28.9M]	22.9M	[16.7M–29.1M]	22.2M	[16.2M–28.2M]
prbench_legal	32.6M	[23.8M–41.4M]	30.7M	[22.4M–38.9M]	24.7M	[18.0M–31.3M]	26.3M	[19.2M–33.4M]
prbench_legal_hard	17.1M	[12.5M–21.7M]	16.0M	[11.7M–20.4M]	12.5M	[9.2M–15.9M]	13.4M	[9.8M–17.1M]
WildBench	88.7M	[64.8M–112.7M]	80.0M	[58.4M–101.6M]	82.0M	[59.9M–104.2M]	85.7M	[62.6M–108.9M]
writingbench_academic_engineering	7.9M	[5.8M–10.0M]	8.1M	[5.9M–10.2M]	8.1M	[5.9M–10.4M]	9.9M	[7.2M–12.6M]
writingbench_advertising_marketing	5.0M	[3.7M–6.4M]	5.0M	[3.6M–6.3M]	5.0M	[3.6M–6.3M]	6.4M	[4.7M–8.2M]
writingbench_education	4.4M	[3.2M–5.6M]	4.3M	[3.2M–5.5M]	4.8M	[3.5M–6.0M]	5.5M	[4.0M–7.0M]
writingbench_finance_business	9.3M	[6.8M–11.8M]	9.3M	[6.8M–11.8M]	9.2M	[6.7M–11.7M]	10.5M	[7.7M–13.4M]
writingbench_literature_arts	9.2M	[6.7M–11.6M]	9.0M	[6.6M–11.5M]	7.1M	[5.2M–9.0M]	8.6M	[6.3M–10.9M]
writingbench_politics_law	7.3M	[5.3M–9.2M]	7.0M	[5.1M–8.9M]	6.9M	[5.1M–8.8M]	8.7M	[6.4M–11.1M]

Table 43 Particle Filter (Mid (n=4)) total completion tokens per benchmark.

Benchmark	Qwen3.5-9B	Qwen3.5-35B-A3B	OLMo-3-7B	OLMo-3.1-32B
healthbench_communication	10.2M	9.0M	8.3M	8.6M
healthbench_complex_responses	4.5M	4.3M	3.4M	3.9M
healthbench_context_seeking	5.0M	4.6M	3.9M	3.9M
healthbench_emergency_referrals	4.1M	3.7M	3.1M	3.0M
healthbench_global_health	10.5M	9.4M	8.7M	8.9M
healthbench_health_data_tasks	5.5M	5.0M	4.3M	5.1M
healthbench_hedging	10.3M	9.2M	7.5M	7.9M
lexam_open	8.0M	7.0M	6.0M	6.4M
prbench_finance	12.3M	11.3M	12.3M	11.6M
prbench_finance_hard	6.3M	5.8M	6.1M	5.9M
prbench_legal	8.2M	7.7M	6.1M	6.6M
prbench_legal_hard	4.4M	4.0M	3.0M	3.3M
WildBench	22.6M	20.6M	22.5M	23.7M
writingbench_academic_engineering	2.0M	2.0M	2.1M	2.5M
writingbench_advertising_marketing	1.4M	1.3M	1.2M	1.7M
writingbench_education	1.1M	1.1M	1.2M	1.4M
writingbench_finance_business	2.4M	2.3M	2.3M	2.7M
writingbench_literature_arts	2.3M	2.2M	1.8M	2.3M
writingbench_politics_law	1.8M	1.7M	1.8M	2.3M

Table 44 Particle Filter (High (n=8)) total completion tokens per benchmark.

Benchmark	Qwen3.5-9B	Qwen3.5-35B-A3B	OLMo-3-7B	OLMo-3.1-32B
healthbench_communication	20.3M	18.2M	16.5M	17.1M
healthbench_complex_responses	9.1M	8.6M	6.8M	8.0M
healthbench_context_seeking	10.0M	9.1M	7.8M	8.0M
healthbench_emergency_referrals	8.2M	7.3M	6.4M	6.1M
healthbench_global_health	21.2M	18.9M	17.5M	17.7M
healthbench_health_data_tasks	10.9M	10.2M	8.3M	9.9M
healthbench_hedging	20.2M	18.3M	15.0M	16.1M
lexam_open	15.9M	14.0M	11.8M	12.8M
prbench_finance	24.3M	22.4M	24.7M	23.7M
prbench_finance_hard	12.5M	11.4M	12.6M	11.9M
prbench_legal	16.4M	15.3M	12.3M	13.2M
prbench_legal_hard	8.7M	8.0M	6.1M	6.9M
WildBench	45.4M	40.8M	44.7M	47.7M
writingbench_academic_engineering	3.9M	4.1M	3.9M	5.1M
writingbench_advertising_marketing	2.7M	2.6M	2.5M	3.4M
writingbench_education	2.3M	2.2M	2.3M	2.9M
writingbench_finance_business	4.6M	4.6M	4.6M	5.2M
writingbench_literature_arts	4.4M	4.6M	3.7M	4.6M
writingbench_politics_law	3.6M	3.6M	3.5M	4.3M

Table 45 Particle Filter (XHigh (n=16)) total completion tokens per benchmark.

Benchmark	Qwen3.5-9B	Qwen3.5-35B-A3B	OLMo-3-7B	OLMo-3.1-32B
healthbench_communication	40.5M	36.1M	33.0M	35.0M
healthbench_complex_responses	18.1M	17.0M	13.7M	15.9M
healthbench_context_seeking	20.0M	18.2M	16.0M	16.2M
healthbench_emergency_referrals	16.5M	14.7M	12.9M	12.5M
healthbench_global_health	42.3M	37.8M	35.2M	36.3M
healthbench_health_data_tasks	21.9M	20.5M	16.8M	20.4M
healthbench_hedging	40.7M	36.5M	30.3M	32.8M
lexam_open	31.5M	27.9M	23.6M	25.5M
prbench_finance	48.1M	44.5M	49.4M	47.5M
prbench_finance_hard	24.7M	22.6M	25.0M	23.5M
prbench_legal	33.0M	30.7M	24.5M	26.3M
prbench_legal_hard	17.0M	16.0M	12.2M	13.5M
WildBench	89.6M	81.5M	89.8M	95.2M
writingbench_academic_engineering	7.9M	8.2M	8.6M	10.4M
writingbench_advertising_marketing	5.1M	5.3M	5.1M	6.9M
writingbench_education	4.4M	4.4M	4.8M	5.6M
writingbench_finance_business	9.3M	9.4M	9.1M	10.8M
writingbench_literature_arts	8.9M	9.0M	6.8M	9.0M
writingbench_politics_law	7.5M	7.2M	6.9M	8.9M

Table 46 Beam Search (Mid (bw=2, n=2)) total completion tokens per benchmark.

Benchmark	Qwen3.5-9B	Qwen3.5-35B-A3B	OLMo-3-7B	OLMo-3.1-32B
healthbench_communication	10.2M	9.0M	8.0M	8.4M
healthbench_complex_responses	4.7M	4.2M	3.4M	4.1M
healthbench_context_seeking	5.0M	4.6M	3.8M	3.8M
healthbench_emergency_referrals	4.1M	3.7M	2.9M	2.9M
healthbench_global_health	10.5M	9.3M	8.4M	8.5M
healthbench_health_data_tasks	5.5M	5.1M	3.9M	5.0M
healthbench_hedging	10.3M	9.2M	7.3M	7.7M
lexam_open	7.9M	7.0M	5.9M	6.4M
prbench_finance	12.3M	11.2M	12.4M	11.9M
prbench_finance_hard	6.3M	5.7M	6.2M	5.9M
prbench_legal	8.1M	7.6M	6.0M	6.4M
prbench_legal_hard	4.3M	4.0M	3.1M	3.3M
WildBench	24.1M	20.5M	22.5M	23.9M
writingbench_academic_engineering	2.1M	2.1M	2.1M	2.5M
writingbench_advertising_marketing	1.3M	1.3M	1.2M	1.7M
writingbench_education	1.2M	1.2M	1.2M	1.4M
writingbench_finance_business	2.4M	2.4M	2.3M	2.7M
writingbench_literature_arts	2.4M	2.3M	1.9M	2.2M
writingbench_politics_law	1.9M	1.7M	1.7M	2.2M

Table 47 Beam Search (High (bw=2, n=4)) total completion tokens per benchmark.

Benchmark	Qwen3.5-9B	Qwen3.5-35B-A3B	OLMo-3-7B	OLMo-3.1-32B
healthbench_communication	19.8M	17.7M	15.7M	16.4M
healthbench_complex_responses	9.4M	8.8M	6.6M	7.9M
healthbench_context_seeking	9.7M	8.8M	7.4M	7.5M
healthbench_emergency_referrals	8.1M	7.2M	5.7M	5.5M
healthbench_global_health	20.6M	18.4M	16.4M	16.6M
healthbench_health_data_tasks	10.7M	10.2M	8.0M	9.6M
healthbench_hedging	20.4M	18.2M	14.1M	15.1M
lexam_open	16.0M	13.7M	11.8M	12.8M
prbench_finance	24.2M	22.1M	25.0M	23.5M
prbench_finance_hard	12.5M	11.2M	12.3M	12.0M
prbench_legal	16.1M	15.0M	11.9M	12.9M
prbench_legal_hard	8.5M	7.8M	6.0M	6.4M
WildBench	46.6M	41.6M	45.0M	47.2M
writingbench_academic_engineering	4.0M	4.1M	4.0M	5.2M
writingbench_advertising_marketing	2.7M	2.6M	2.4M	3.3M
writingbench_education	2.3M	2.2M	2.3M	2.8M
writingbench_finance_business	4.7M	4.7M	4.6M	5.2M
writingbench_literature_arts	5.1M	4.6M	3.6M	4.4M
writingbench_politics_law	3.8M	3.5M	3.3M	4.5M

Table 48 Beam Search (XHigh (bw=4, n=4)) total completion tokens per benchmark. †Run at bw=3, n=4.

Benchmark	Qwen3.5-9B	Qwen3.5-35B-A3B	OLMo-3-7B	OLMo-3.1-32B
healthbench_communication	40.4M	34.9M	32.0M	34.0M
healthbench_complex_responses	21.6M	19.8M	14.0M	16.7M
healthbench_context_seeking	21.0M	18.1M	15.1M	15.5M
healthbench_emergency_referrals	16.6M	14.4M	11.3M	11.8M
healthbench_global_health	42.6M	36.6M	33.3M	34.9M
healthbench_health_data_tasks	23.3M	21.4M	16.6M	20.4M
healthbench_hedging	45.3M	38.4M	29.6M	32.2M
lexam_open	32.1M	27.9M	23.6M	26.8M
prbench_finance	48.6M	44.3M	50.1M	47.7M
prbench_finance_hard	24.5M	22.5M	25.7M	23.8M
prbench_legal	32.8M	30.0M	24.3M	25.9M
prbench_legal_hard	16.8M	15.7M	12.1M	13.2M
WildBench	100.0M	89.6M	92.8M	99.2M
writingbench_academic_engineering	9.1M	9.2M	8.8M	11.0M
writingbench_advertising_marketing	4.1M†	5.6M	5.0M	6.6M
writingbench_education	5.1M	5.1M	4.6M	6.2M
writingbench_finance_business	10.4M	10.1M	9.3M	11.1M
writingbench_literature_arts	10.8M	7.3M†	7.0M	9.0M
writingbench_politics_law	8.1M	8.1M	7.0M	9.3M

Table 49 Sequential Refinement (iter=16) total completion tokens per benchmark.

Benchmark	Qwen3.5-9B	Qwen3.5-35B-A3B	OLMo-3-7B	OLMo-3.1-32B
healthbench_communication	83.0M	82.6M	–	–
healthbench_complex_responses	33.0M	32.4M	–	–
healthbench_context_seeking	45.6M	44.7M	–	–
healthbench_emergency_referrals	36.4M	36.0M	–	–
healthbench_global_health	91.6M	88.3M	–	–
healthbench_health_data_tasks	40.1M	40.2M	–	–
healthbench_hedging	85.2M	83.1M	–	–
lexam_open	48.7M	48.1M	–	–
prbench_finance	71.3M	74.8M	–	–
prbench_finance_hard	35.8M	38.0M	–	–
prbench_legal	54.5M	58.2M	–	–
prbench_legal_hard	27.5M	29.5M	–	–
WildBench	120.6M	121.5M	–	–
writingbench_academic_engineering	14.3M	15.3M	–	–
writingbench_advertising_marketing	7.7M	7.8M	–	–
writingbench_education	7.5M	7.8M	–	–
writingbench_finance_business	17.0M	17.7M	–	–
writingbench_literature_arts	13.6M	14.1M	–	–
writingbench_politics_law	11.7M	12.0M	–	–

Table 50 Fusion (Mid (s=3)) total completion tokens per benchmark.

Benchmark	Qwen3.5-9B	Qwen3.5-35B-A3B	OLMo-3-7B	OLMo-3.1-32B
healthbench_communication	7.4M	6.7M	6.2M	6.4M
healthbench_complex_responses	3.2M	3.0M	2.6M	3.0M
healthbench_context_seeking	3.6M	3.3M	2.9M	2.9M
healthbench_emergency_referrals	3.0M	2.7M	2.4M	2.3M
healthbench_global_health	7.7M	7.0M	6.5M	6.5M
healthbench_health_data_tasks	3.9M	3.7M	3.2M	3.8M
healthbench_hedging	7.3M	6.7M	5.6M	5.9M
lexam_open	5.8M	5.2M	4.5M	4.8M
prbench_finance	9.0M	8.4M	8.6M	8.3M
prbench_finance_hard	4.6M	4.3M	4.3M	4.2M
prbench_legal	6.1M	5.7M	4.6M	4.9M
prbench_legal_hard	3.2M	3.0M	2.4M	2.5M
WildBench	16.6M	15.0M	15.4M	16.1M
writingbench_academic_engineering	1.5M	1.5M	1.5M	1.9M
writingbench_advertising_marketing	940K	934K	930K	1.2M
writingbench_education	825K	809K	892K	1.0M
writingbench_finance_business	1.7M	1.7M	1.7M	2.0M
writingbench_literature_arts	1.7M	1.7M	1.3M	1.6M
writingbench_politics_law	1.4M	1.3M	1.3M	1.6M

Table 51 Fusion (High, s=6or7) total completion tokens per benchmark.

Benchmark	Qwen3.5-9B	Qwen3.5-35B-A3B	OLMo-3-7B	OLMo-3.1-32B
healthbench_communication	14.9M	13.4M	12.3M	12.8M
healthbench_complex_responses	6.5M	6.1M	5.3M	6.0M
healthbench_context_seeking	7.2M	6.7M	5.8M	5.8M
healthbench_emergency_referrals	5.9M	5.4M	4.7M	4.5M
healthbench_global_health	15.5M	14.1M	13.0M	13.0M
healthbench_health_data_tasks	7.8M	7.4M	6.5M	7.5M
healthbench_hedging	14.7M	13.4M	11.2M	11.8M
lexam_open	11.7M	10.4M	9.0M	9.7M
prbench_finance	18.1M	16.7M	17.1M	16.6M
prbench_finance_hard	9.2M	8.5M	8.6M	8.3M
prbench_legal	12.2M	11.5M	9.3M	9.9M
prbench_legal_hard	6.4M	6.0M	4.7M	5.0M
WildBench	33.3M	30.0M	30.8M	32.1M
writingbench_academic_engineering	3.0M	3.0M	3.1M	3.7M
writingbench_advertising_marketing	1.9M	1.9M	1.9M	2.4M
writingbench_education	1.7M	1.6M	1.8M	2.1M
writingbench_finance_business	3.5M	3.5M	3.4M	4.0M
writingbench_literature_arts	3.4M	3.4M	2.7M	3.2M
writingbench_politics_law	2.7M	2.6M	2.6M	3.3M

Table 52 Fusion (XHigh (s=15)) total completion tokens per benchmark.

Benchmark	Qwen3.5-9B	Qwen3.5-35B-A3B	OLMo-3-7B	OLMo-3.1-32B
healthbench_communication	37.1M	33.5M	–	–
healthbench_complex_responses	16.2M	15.2M	–	–
healthbench_context_seeking	18.1M	16.7M	–	–
healthbench_emergency_referrals	14.9M	13.6M	–	–
healthbench_global_health	38.7M	35.2M	–	–
healthbench_health_data_tasks	19.5M	18.5M	–	–
healthbench_hedging	36.7M	33.5M	–	–
lexam_open	29.2M	26.1M	–	–
prbench_finance	45.2M	41.8M	–	–
prbench_finance_hard	23.0M	21.3M	–	–
prbench_legal	30.6M	28.7M	–	–
prbench_legal_hard	16.0M	15.0M	–	–
WildBench	83.2M	75.0M	–	–
writingbench_academic_engineering	7.4M	7.5M	–	–
writingbench_advertising_marketing	4.7M	4.7M	–	–
writingbench_education	4.1M	4.0M	–	–
writingbench_finance_business	8.7M	8.7M	–	–
writingbench_literature_arts	8.6M	8.5M	–	–
writingbench_politics_law	6.8M	6.6M	–	–